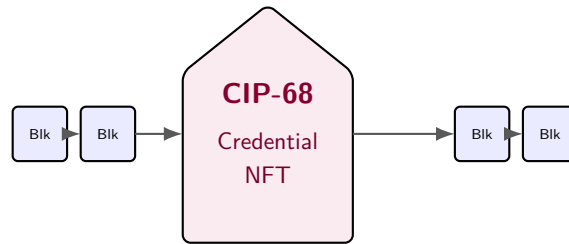


Cardano Credential Verification

A Complete Study Guide

*Comprehensive Tutorial for the Whitepaper
“Blockchain-Based Professional Credential Verification:
A Cardano Smart Contract Architecture for License Management
and Digital Signatures” (v3.1)*



Timothy E. Smith, PE

Virobit Research

February 2026

Open Source — MIT License

https://github.com/virobit/license_management_using_cardano

Contents

Preface	7
1 Blockchain Fundamentals	8
1.1 What Is a Blockchain?	8
1.1.1 Blocks	8
1.1.2 Chains and Immutability	9
1.2 Consensus Mechanisms	9
1.2.1 Proof of Work (PoW)	9
1.2.2 Proof of Stake (PoS) and Cardano's Ouroboros	10
1.3 Public Key Cryptography	10
1.3.1 The Mailbox Analogy	10
1.3.2 Digital Signatures	11
1.3.3 Keys, Addresses, and Wallets	11
1.4 Hashing	12
1.4.1 One-Way Functions	12
1.4.2 SHA-256 and Blake2b	13
1.4.3 Practical Examples	13
1.5 Transaction Models: Account vs UTxO	14
1.5.1 The Account Model (Ethereum)	14
1.5.2 The UTxO Model (Bitcoin and Cardano)	14
1.5.3 Cardano's Extended UTxO (EUTxO)	14
1.6 What Is Cardano?	15
1.6.1 Generational Context	15
1.6.2 Architecture and Timing	16
1.6.3 Native Multi-Asset Support	16
1.6.4 Formal Verification Heritage	16
2 Cardano's Extended UTxO Model and Architecture	18
2.1 The Extended UTxO (eUTxO) Model	18
2.1.1 From Bitcoin's UTxO to Cardano's eUTxO	18
2.1.2 Datums: Data Attached to UTxOs	19
2.1.3 Redeemers: Data Provided When Spending	19
2.1.4 Script Context: The Transaction in Full	20
2.1.5 Determinism: The Key Advantage	20
2.2 Native Multi-Asset Framework	20
2.2.1 Tokens Without Smart Contracts	20

2.2.2	Policy ID and Asset Name	21
2.2.3	Multi-Asset Bundles in UTxOs	22
2.3	Minimum UTxO (minUTxO) Deposit	22
2.3.1	The Problem: Ledger Bloat	22
2.3.2	The Apartment Deposit Analogy	23
2.3.3	Why This Matters for Credential Systems	23
2.4	Slots, Epochs, and Time	23
2.4.1	Cardano’s Time Model	23
2.4.2	Slot-Based Transaction Validity	24
2.4.3	Converting Between Slots and Wall-Clock Time	24
2.5	Transaction Fees	25
2.5.1	Deterministic Fee Model	25
2.5.2	Plutus Script Execution Fees	25
2.5.3	Comparison with Ethereum Gas	25
2.6	Addresses	26
2.6.1	Address Types	26
2.6.2	Key Hashes: PKH and SKH	26
2.6.3	Bech32 Encoding	27
2.7	UTxO Selection Strategies	27
2.7.1	The UTxO Selection Problem	27
2.7.2	LargestFirstSelector	28
2.7.3	RandomImproveSelector	28
2.7.4	Practical Implications	28
2.8	Chapter Summary	29
3	Cardano Improvement Proposals (CIPs)	30
3.1	What Are CIPs?	30
3.2	CIP-25: NFT Metadata Standard	31
3.2.1	The Problem	31
3.2.2	How It Works	31
3.2.3	How the Credential System Uses CIP-25	32
3.3	CIP-68: Datum Metadata Standard	33
3.3.1	The Problem CIP-68 Solves	33
3.3.2	The Dual-Token Pattern	33
3.3.3	How Revocation Works	34
3.3.4	Technical Details	34
3.3.5	Verification Flow	35
3.3.6	CIP-25 vs. CIP-68: Comparison	35
3.3.7	Why CIP-68 Matters for Credentials	36
3.4	CIP-30: dApp–Wallet Web Bridge	37
3.4.1	The Problem	37
3.4.2	How It Works	37
3.4.3	How the Credential System Uses CIP-30	37
3.5	CIP-31: Reference Inputs	38
3.5.1	The Problem	38
3.5.2	How It Works	39
3.5.3	How the Credential System Uses CIP-31	39
3.6	CIP-32: Inline Datums	39

3.6.1	The Problem	39
3.6.2	How It Works	40
3.6.3	How the Credential System Uses CIP-32	40
3.7	CIP-33: Reference Scripts	41
3.7.1	The Problem	41
3.7.2	How It Works	41
3.7.3	How the Credential System Uses CIP-33	41
3.8	CIP-1852 / BIP-44: HD Wallet Derivation	41
3.8.1	The Problem	41
3.8.2	How It Works	42
3.8.3	How the Credential System Uses CIP-1852	43
3.9	How the CIPs Work Together	44
3.10	Chapter Summary	45
4	Smart Contracts on Cardano	47
4.1	What is a Smart Contract?	47
4.1.1	The Core Idea	47
4.1.2	Cardano Validators Are Not Ethereum Smart Contracts	47
4.1.3	Why This Matters for Credentials	48
4.2	Native Scripts (Phase 1)	48
4.2.1	Overview	48
4.2.2	The Five Script Types	49
4.2.3	Why Native Scripts for Minting Policies?	50
4.3	Plutus V2 Validators (Phase 2)	50
4.3.1	Beyond Simple Scripts	50
4.3.2	The Three Inputs to a Validator	51
4.3.3	Deterministic Execution	52
4.4	Execution Units (exUnits)	52
4.4.1	CPU Steps and Memory Units	52
4.4.2	How exUnits Affect Fees	53
4.4.3	Deterministic Cost Estimation	53
4.5	Minting Policies	53
4.5.1	What is a Minting Policy?	53
4.5.2	Policy ID = Script Hash	54
4.5.3	The Credential System's Minting Policy	54
4.5.4	Verification Without Trust	55
4.6	The SignatureCollectionValidator	55
4.6.1	The Problem: Multi-Party Signing on eUTxO	55
4.6.2	The Solution: Independent Per-Signer UTxOs	56
4.6.3	The Three Redeemer Actions	56
4.6.4	The SignerDatum	57
4.6.5	Concurrency in Detail: The COLLECT → FINALIZE Flow	58
4.6.6	What if Something Goes Wrong?	59
4.7	The DuesEnforcementContract	59
4.7.1	Linking License Validity to Payments	59
4.7.2	Contract Parameters	60
4.7.3	Two Redeemer Actions	60
4.7.4	Grace Period Mechanics	60

4.7.5	The DuesContractDatum	61
4.8	Aiken: The Future	61
4.8.1	The Problem with PlutusTx	61
4.8.2	Enter Aiken	62
4.8.3	Migration Roadmap (v4.0)	62
5	The Credential Verification System	64
5.1	System Overview	64
5.1.1	Layer 1: Cardano Blockchain	64
5.1.2	Layer 2: Off-Chain Application Layer	65
5.1.3	Layer 3: Local SQLite Database	65
5.1.4	Communication: The Blockfrost API	65
5.2	Token Taxonomy	65
5.2.1	Token 1: License NFT (CIP-68)	67
5.2.2	Token 2: Signature Token (Fungible)	67
5.2.3	Token 3: Validity Token (Time-Bounded)	67
5.2.4	Token 4: Work Product (Logical Composite)	68
5.3	Wallet Roles	68
5.3.1	Authority	68
5.3.2	Licensee	70
5.3.3	Signer	70
5.3.4	Observer	70
5.4	Credential Lifecycle	70
5.4.1	Step (a): Policy Creation	71
5.4.2	Step (b): License NFT Minting (CIP-68)	71
5.4.3	Step (c): Signature and Validity Token Minting	72
5.4.4	Step (d): Token Holding	72
5.4.5	Step (e): Signing (Token Deposit)	72
5.4.6	Step (f): Verification	73
5.4.7	Step (g): Renewal	73
5.4.8	Step (h): Revocation	73
5.5	Revocation Mechanism	73
5.5.1	The Problem: eUTxO Ownership	75
5.5.2	The Broken Approach: Direct Token Burn	75
5.5.3	The CIP-68 Solution: Datum-Based Revocation	75
5.5.4	The minUTxO Economics	76
5.6	Verification Flow	76
5.6.1	Step-by-Step Verification	76
5.6.2	Permissionless Verification	78
5.6.3	Verification Without a Full Node	79
5.7	Cost Economics	79
5.7.1	Comparison to Ethereum	79
5.7.2	Annual Cost Projection	80
5.7.3	Locked ADA and Capital Efficiency	80
5.8	Chapter Summary	80
6	Multi-Party Document Signing, Privacy, and Legal Compliance	82
6.1	The Signing Problem	82

6.2	The Six-Phase Signing Workflow	83
6.2.1	Phase 1: Setup	83
6.2.2	Phase 2: Credential Verification	84
6.2.3	Phase 3: Signing (Token Deposit)	84
6.2.4	Phase 4: Collection	84
6.2.5	Phase 5: Finalization	85
6.2.6	Phase 6: Verification	85
6.3	The eUTxO Concurrency Solution	86
6.3.1	Understanding the Problem	86
6.3.2	The Independent UTxO Solution	87
6.3.3	Why This Works in Cardano’s Model	88
6.4	SignerDatum Structure	88
6.5	The Privacy Architecture	90
6.5.1	On-Chain Data (Public, Immutable)	90
6.5.2	Off-Chain Data (Private, Erasable)	90
6.6	AES-256-GCM Encryption	91
6.7	GDPR and Crypto-Shredding	92
6.7.1	The GDPR Dilemma	92
6.7.2	The Solution: Crypto-Shredding	93
6.7.3	Why Hashes Cannot Be Reversed	94
6.8	ESIGN Act (US Federal)	95
6.8.1	Requirement 1: Intent to Sign	95
6.8.2	Requirement 2: Consent to Conduct Business Electronically	95
6.8.3	Requirement 3: Association of Signature with Record	95
6.8.4	Requirement 4: Record Retention	96
6.9	UETA (US State Law)	96
6.9.1	Blockchain-Specific State Amendments	96
6.10	eIDAS 2.0 (European Union)	97
6.10.1	Three Signature Tiers	97
6.10.2	Qualified Electronic Ledger (Article 45l)	98
6.11	Putting It All Together: Compliance Summary	99
7	The Software and the v4.0 Roadmap	101
7.1	Package Overview	101
7.2	Architecture	102
7.2.1	core.py — The Main Module	102
7.2.2	tx_utils.py — Transaction Utilities	103
7.2.3	schema.py — Database Schema	103
7.2.4	__init__.py — Package Exports	103
7.3	Key API Functions	103
7.3.1	generate_wallet	103
7.3.2	mint_license_nft	104
7.3.3	update_license_status	105
7.3.4	sign_document	105
7.3.5	verify_signature	106
7.3.6	pay_dues	106
7.4	Database Schema	107
7.5	Configuration	107

7.6	Testing	110
7.7	Aiken Smart Contracts	110
7.8	CIP-33 Reference Scripts	111
7.9	Validity Token Consolidation	112
7.10	HSM Authority Keys	113
7.11	KERI / Multi-Sig Upgrade	114
7.12	Security Audit Results	114
	7.12.1 v3.0 Audit: Initial Findings	115
	7.12.2 v3.1 Audit: Remediation Verification	115
	7.12.3 Final Audit: Testnet Clearance	115
7.13	Glossary of Terms	116

Preface

This study guide is a companion to the research paper “*Blockchain-Based Professional Credential Verification: A Cardano Smart Contract Architecture for License Management and Digital Signatures*” (v3.1, February 2026). While the paper presents the architecture for an academic audience, this guide is written for **anyone who wants to understand how the system works**, regardless of technical background.

No prior knowledge of blockchain, cryptography, or Cardano is assumed. Every concept referenced in the whitepaper is explained from first principles, with diagrams, analogies, and code examples. By the end of this guide, you will understand:

- How blockchains work and why they are useful for credential verification
- How Cardano’s unique architecture (eUTxO, native tokens, Plutus) differs from other blockchains
- What every Cardano Improvement Proposal (CIP) referenced in the paper does
- How smart contracts enforce credential rules on Cardano
- How the complete credential lifecycle works — from issuance through revocation
- How multi-party document signing is implemented
- How privacy and legal compliance (GDPR, ESIGN, eIDAS) are achieved
- How the `cardano-license` software package works
- What the v4.0 roadmap delivers

Each chapter builds on the previous one, so we recommend reading in order. Key terms are **highlighted in blue** when first introduced, and a comprehensive glossary is provided at the end.

This guide was prepared with assistance from Claude Code (Anthropic). All technical claims, architectural decisions, and editorial judgments are the sole responsibility of the author.

Chapter 1

Blockchain Fundamentals

This chapter introduces the foundational concepts you need before we touch a single line of Cardano code. Every idea is explained from scratch—no prior knowledge of blockchain, cryptography, or distributed systems is assumed. By the end of this chapter you will understand *what* a blockchain is, *why* it works, and *how* Cardano improves on earlier designs. These building blocks are essential for the credential-verification system we will construct in later chapters.

1.1 What Is a Blockchain?

Imagine a notebook that sits in the centre of a town square. Every time someone makes a deal—say, Alice sells a bicycle to Bob—they walk to the square and write the transaction in the notebook for everyone to see. Once a page is full, a town clerk stamps it with a unique seal, tears it out, and staples it to the previous page. The seal on each new page includes a reference to the seal on the page before it. If anyone tried to rip out a page in the middle and replace it, the seals would no longer match and the entire town would immediately notice.

A **blockchain** is the digital version of that notebook. Instead of pages we have *blocks*; instead of a staple we have a *cryptographic hash* (explained in Section 1.4); and instead of one notebook in a square, thousands of identical copies are maintained by computers all over the world.

1.1.1 Blocks

A **block** is a bundle of data. In a financial blockchain the data is a list of transactions (“Alice sent 10 tokens to Bob”), but blocks can hold any kind of information—credential issuances, votes, supply-chain events, or software licence grants. Every block also carries a small header that contains bookkeeping metadata: a timestamp, a reference to the previous block, and the result of running all the block’s data through a hash function (Section 1.4).

Each block can be thought of as a sealed envelope. Once the envelope is sealed (the block is added to the chain), its contents cannot be changed without breaking the seal. Because every subsequent envelope references the seal of the one before it, altering a single envelope would require re-sealing every envelope that comes after—a task that is computationally infeasible on a well-established chain.

1.1.2 Chains and Immutability

The word *chain* refers to the way blocks are linked. Block 2 contains the hash of Block 1; Block 3 contains the hash of Block 2; and so on. This creates an unbroken lineage all the way back to Block 0 (the *genesis block*). If someone modifies even a single byte inside Block 1, its hash changes, which means Block 2’s “previous hash” field no longer matches, which invalidates Block 3, and so on. The entire chain after the tampered block collapses like a row of dominoes.

This property is called **immutability**: once data is recorded in a block and enough subsequent blocks have been added on top, it is extraordinarily difficult to alter. Immutability is one of the key reasons blockchains are attractive for credential verification—a licence recorded on-chain cannot be silently edited or back-dated.

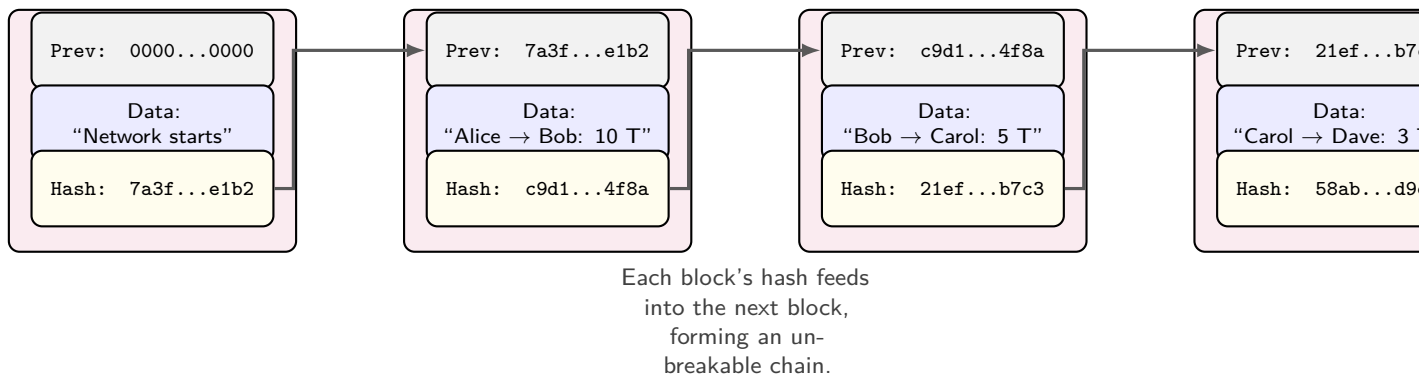


Figure 1.1: A blockchain of four blocks. Each block stores data plus the hash of the preceding block. Changing Block 1 would invalidate every subsequent hash.

1.2 Consensus Mechanisms

If thousands of computers around the world each hold a copy of the blockchain, who decides which new block gets added next? What stops a malicious actor from broadcasting a fake block that credits themselves with a million tokens? These questions are answered by a **consensus mechanism**—a set of rules that all participants (called *nodes*) follow to agree on the next valid block.

Without consensus, a distributed ledger would quickly descend into chaos. Different nodes would have different versions of reality, and there would be no way to tell which one is “true.” Consensus mechanisms solve this by making it expensive or statistically improbable for any single party to dominate the block-production process.

1.2.1 Proof of Work (PoW)

Bitcoin, the first blockchain, uses **Proof of Work**. In PoW, nodes (called *miners*) compete to solve a mathematical puzzle. The puzzle is simple to describe but computationally exhausting: find a number (called a *nonce*) such that when you hash the block header together with that nonce, the resulting hash starts with a certain number of leading zeros. There is no shortcut; miners must try billions of nonce values per second until one of them stumbles on a valid answer.

The miner who finds the answer first broadcasts the new block to the network, and the other miners verify (verification is fast—just one hash computation) and accept it. The winning miner is rewarded with newly created cryptocurrency. The difficulty of the puzzle is automatically adjusted so that, on average, one new block is found every ten minutes (in Bitcoin’s case).

PoW is remarkably secure—attacking it would require controlling more than half of the entire network’s computing power, which for Bitcoin would cost billions of dollars in hardware and electricity. The downside is precisely that electricity cost: Bitcoin’s annual energy consumption rivals that of some small countries. This is the primary motivation for alternative consensus mechanisms.

1.2.2 Proof of Stake (PoS) and Cardano’s Ouroboros

Cardano uses **Proof of Stake**, specifically a protocol called **Ouroboros**. Instead of burning electricity to solve puzzles, PoS selects the next block producer based on how much cryptocurrency they have *staked* (locked up as collateral). Think of it as a raffle: if you hold 2% of all staked tokens, you have roughly a 2% chance of being chosen to produce the next block. There is no puzzle to solve, so the energy cost is negligible—a standard laptop can run a Cardano stake pool.

Ouroboros divides time into *epochs* (each lasting five days) and further into *slots* (each one second long). At the beginning of each epoch a lottery determines which stake-pool operator is eligible to produce a block in each slot. If the designated leader is online, they produce the block and earn rewards; if not, the slot is skipped and the next eligible leader takes over. In practice, a new block appears roughly every 20 seconds.

The security guarantee of Ouroboros has been formally proven in peer-reviewed academic papers—a distinguishing feature of Cardano. As long as at least 51% of the staked ada is controlled by honest participants, the protocol guarantees that the chain will grow correctly and that double-spending is impossible.

1.3 Public Key Cryptography

Before we can understand how transactions are authorised on a blockchain, we need to grasp **public key cryptography** (also called *asymmetric cryptography*). This is the mathematical machinery that lets you prove you own something without revealing a secret.

1.3.1 The Mailbox Analogy

Imagine you install a mailbox on your front lawn. The mailbox has a slot (the **public key**) through which anyone can drop a letter, but it also has a lock (the **private key**) that only you can open. You freely share your address (the slot location) with the world, but you never share the lock’s key. Anyone can send you mail; only you can read it.

In blockchain terms, your *public key* (or a shortened version called your *address*) is what you share so that others can send you cryptocurrency or verify your signatures. Your *private key* is the secret that lets you spend your funds or sign documents. If someone obtains your private key, they can impersonate you—guard it as carefully as you would guard the key to a safety-deposit box.

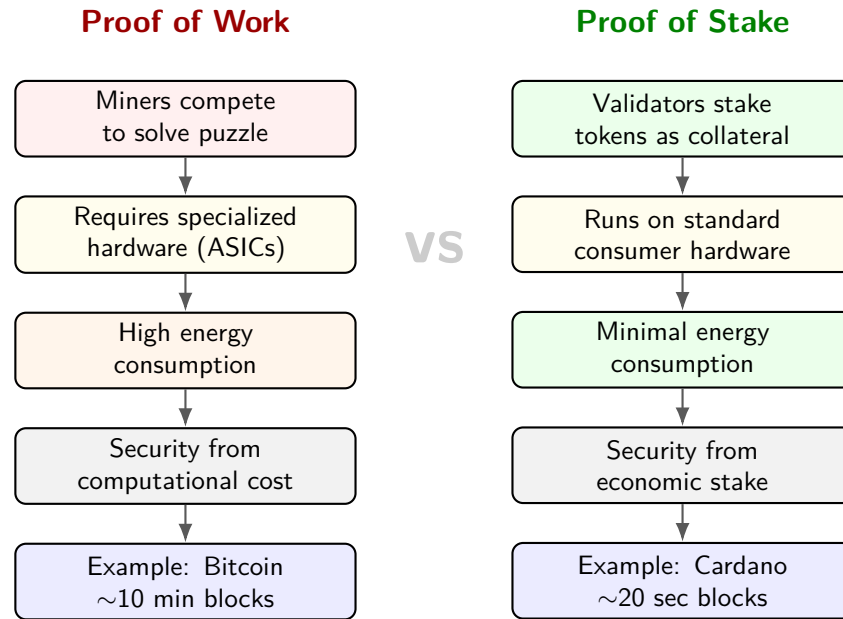


Figure 1.2: Proof of Work versus Proof of Stake. PoS eliminates the enormous energy expenditure of PoW by replacing computational puzzles with economic incentives.

1.3.2 Digital Signatures

A **digital signature** is the blockchain equivalent of a handwritten signature on a cheque, but far more secure. When you want to send a transaction, your wallet software takes the transaction data and combines it with your private key using a mathematical algorithm (Cardano uses *Ed25519*). The output is a short string of bytes—the signature.

Anyone who has your public key can then run a verification algorithm on the (transaction, signature) pair. If the signature is valid, they know two things with mathematical certainty:

1. **Authenticity:** The transaction was created by whoever holds the corresponding private key.
2. **Integrity:** The transaction has not been modified since it was signed—not even by a single bit.

This is what makes blockchain transactions trustworthy. There is no bank or notary in the middle; the mathematics itself provides the proof. For our credential-verification system, this same mechanism will let a licensing authority sign a credential, and any third party can verify that signature without contacting the authority.

1.3.3 Keys, Addresses, and Wallets

In practice, a *wallet* is a piece of software that manages one or more private keys on your behalf. When you “create a wallet,” the software generates a random private key, derives the corresponding public key, and further derives an *address* (a shorter, human-readable encoding of the public key, often with a checksum to catch typos). On Cardano, addresses start with `addr1...` on mainnet and `addr_test1...` on the test network.

A single wallet can contain many key pairs, all derived from a single master seed phrase (typically 24 words). This hierarchical key derivation means you only need to back up those 24 words to recover all of your keys and addresses.

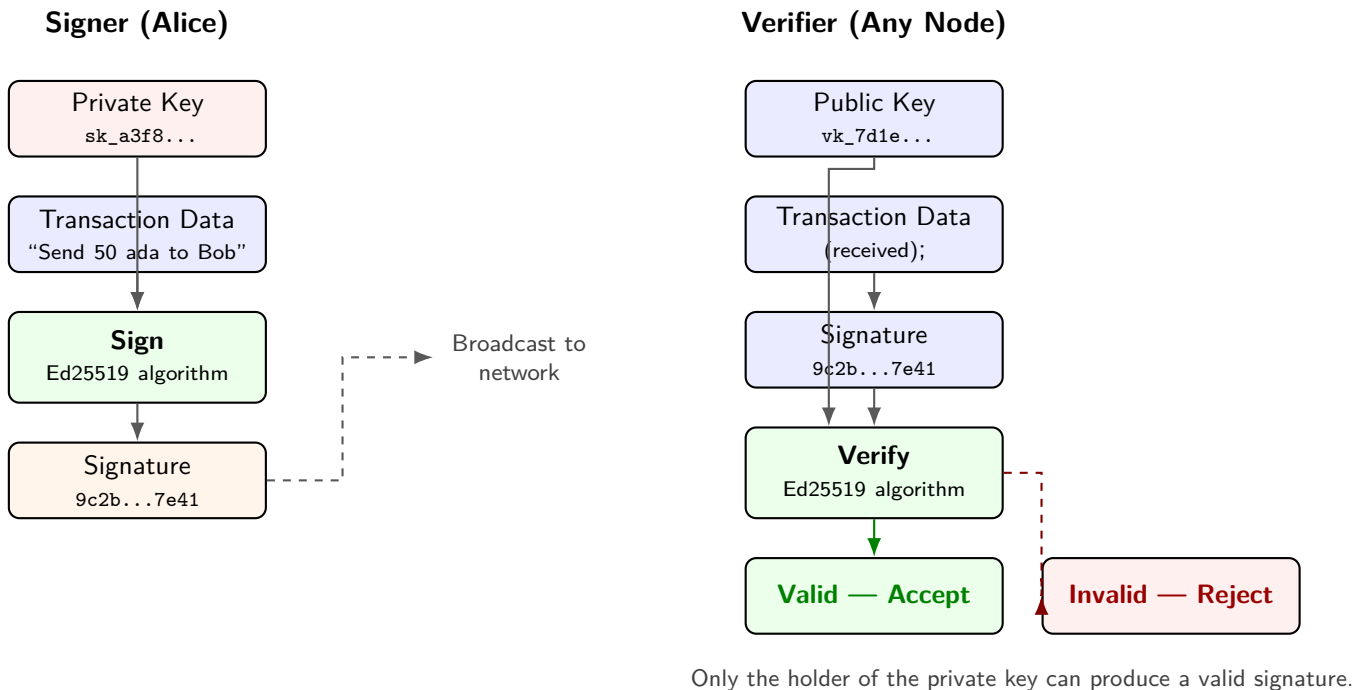


Figure 1.3: Digital signature flow. Alice signs with her private key; any node verifies with her public key. If the data or signature is altered, verification fails.

1.4 Hashing

Hashing is one of the most important primitives in all of blockchain technology. A **hash function** takes an input of any size—a single character, a novel, an entire database—and produces an output (called a *digest* or simply a *hash*) of a fixed size. SHA-256, for example, always produces a 256-bit (32-byte) output regardless of whether the input is one byte or one terabyte.

1.4.1 One-Way Functions

The critical property of a cryptographic hash function is that it is **one-way**: given the output, there is no feasible way to reconstruct the input. This is analogous to a meat grinder: you can push a steak in and get ground beef out, but you cannot reassemble the steak from the ground beef. Even a tiny change to the input—flipping a single bit—produces a completely different hash, a property known as the *avalanche effect*.

This one-way property is what makes the blockchain's hash-chain tamper-proof. An attacker cannot start with a desired hash and work backwards to craft a block that matches. They would have to try inputs at random, which for a 256-bit hash means searching through 2^{256} possibilities—a number larger than the estimated count of atoms in the observable universe.

1.4.2 SHA-256 and Blake2b

SHA-256 (Secure Hash Algorithm, 256 bits) is the hash function used by Bitcoin. It was designed by the U.S. National Security Agency and has been extensively studied for over two decades. It produces a 256-bit hash (commonly displayed as a 64-character hexadecimal string).

Blake2b is a newer hash function that is faster than SHA-256 on modern hardware while maintaining equivalent security. Cardano uses Blake2b-256 (256-bit output) for most on-chain hashing, including the computation of transaction IDs and policy IDs. It also uses Blake2b-224 (224-bit output, i.e. 28 bytes) for deriving the key-hash portion of addresses.

For our purposes the choice between SHA-256 and Blake2b is an implementation detail—both behave like ideal one-way functions. What matters is understanding the *concept*: any input produces a fixed-size fingerprint, and that fingerprint cannot be reversed.

1.4.3 Practical Examples

To build intuition, here are some example hashes (SHA-256, shown as hexadecimal):

Input	SHA-256 Hash (first 16 hex chars...)
"hello"	2cf24dba5fb0a30e...
"Hello"	185f8db32271fe25...
"hello!"	ce06092fb948d936...
"hello" (repeated 1000×)	f1d3ff8443297732...

Notice that capitalising the “h” or adding an exclamation mark produces a completely unrelated hash. There is no pattern, no partial similarity—this is the avalanche effect in action. Also note that the hash length is the same regardless of whether the input is 5 characters or 5 000.

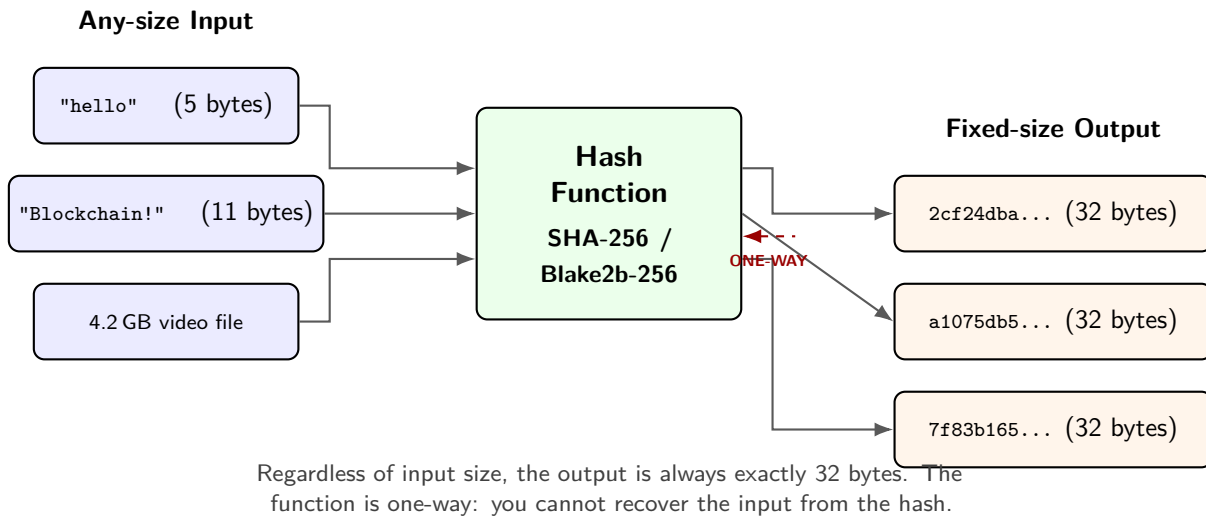


Figure 1.4: A hash function maps any-size input to a fixed-size output. The process is deterministic (same input always yields the same hash) but irreversible.

1.5 Transaction Models: Account vs UTxO

How does a blockchain keep track of who owns what? There are two dominant models, and understanding the difference is *critical* for working with Cardano.

1.5.1 The Account Model (Ethereum)

The **account model** works exactly like a bank account. Each address has a single balance, and a transaction simply debits one account and credits another. If Alice has 100 tokens and sends 30 to Bob, the ledger updates two numbers: Alice’s balance becomes 70 and Bob’s balance becomes 30 (or increases by 30 if he already had some).

This is intuitive and easy to reason about, which is why Ethereum chose it. However, it introduces complications around ordering (what if two transactions try to spend from Alice’s account simultaneously?) and requires global state tracking—every node must maintain and agree on the current balance of every account.

1.5.2 The UTxO Model (Bitcoin and Cardano)

Cardano and Bitcoin use the **Unspent Transaction Output (UTxO)** model. This is best understood by analogy to physical cash.

Imagine Alice has a \$50 bill and a \$20 bill in her wallet (two UTxOs). She wants to pay Bob \$60. She cannot tear the bills in half; she must hand over both bills (\$70 total) and receive \$10 in change. After the transaction, Alice has one new UTxO worth \$10 and Bob has one new UTxO worth \$60. The original \$50 and \$20 UTxOs are now “spent” and can never be used again.

More precisely, every transaction *consumes* one or more existing UTxOs (the **inputs**) and *produces* one or more new UTxOs (the **outputs**). The sum of the inputs must equal the sum of the outputs plus any transaction fee. A UTxO can only be spent once—this is how double-spending is prevented.

The UTxO model has several advantages for our credential system:

- **Determinism:** A transaction’s validity depends only on its inputs, not on any global state. You can validate a transaction offline.
- **Parallelism:** Transactions that consume different UTxOs are independent and can be processed in parallel.
- **Auditability:** You can trace the complete history of every token by following the UTxO chain backwards, which is ideal for credential provenance.

1.5.3 Cardano’s Extended UTxO (EUTxO)

Cardano extends the basic UTxO model with two powerful additions:

1. **Datum:** Each UTxO can carry an arbitrary piece of data (the *datum*). This is where we will store credential metadata—the licence holder’s name, expiry date, credential type, and so on.

2. **Validator scripts:** Instead of being locked by a simple public-key hash, a UTxO can be locked by a *Plutus script* (a program that runs on-chain). The script decides whether a transaction is allowed to consume the UTxO, based on the datum, the *redeemer* (an argument supplied by the spender), and the full *script context* (information about the transaction).

This “Extended UTxO” (EUTxO) model is what makes Cardano a programmable blockchain while retaining the determinism and parallelism benefits of the original UTxO design. We will explore Plutus scripts in detail in a later chapter.

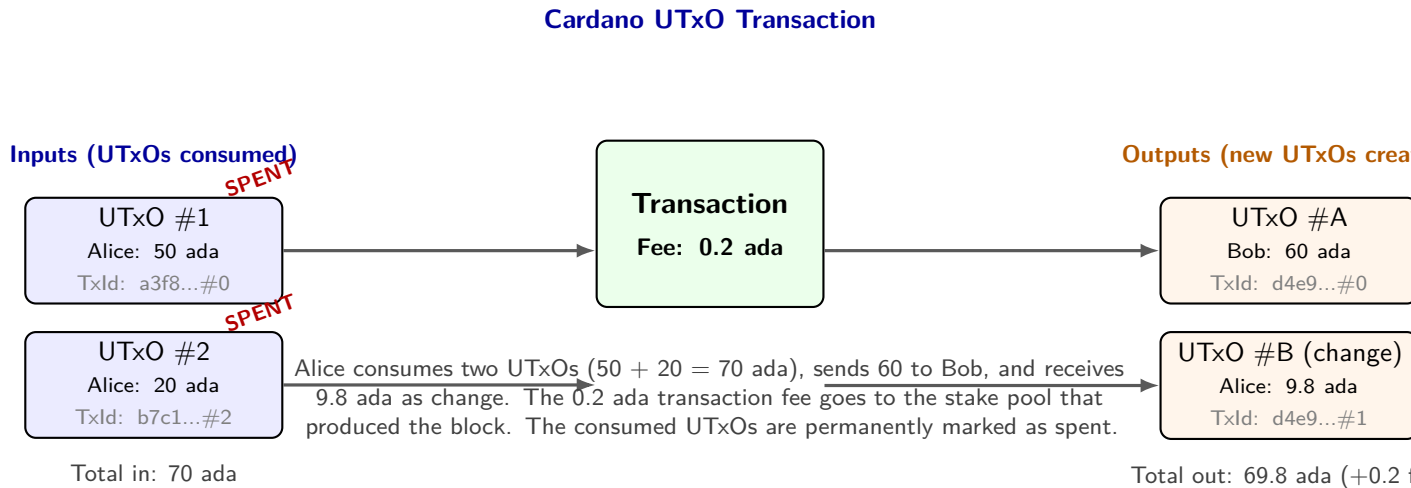


Figure 1.5: Anatomy of a UTxO transaction. Inputs are consumed (destroyed) and outputs are created. The sum of inputs equals the sum of outputs plus the fee.

1.6 What Is Cardano?

Cardano is a third-generation, open-source, proof-of-stake blockchain platform. It was founded in 2015 by Charles Hoskinson (a co-founder of Ethereum) and is developed primarily by Input Output Global (IOG), with support from the Cardano Foundation and Emurgo.

1.6.1 Generational Context

Blockchains are sometimes categorised by “generation”:

First generation (Bitcoin, 2009): Introduced decentralised digital money and the basic blockchain data structure. Limited to simple value-transfer transactions.

Second generation (Ethereum, 2015): Added smart contracts—programs that run on the blockchain—enabling decentralised applications (dApps), token standards, and DeFi. However, Ethereum faces scalability challenges and uses an account-based model that can lead to non-deterministic transaction outcomes.

Third generation (Cardano, 2017): Aims to solve the remaining challenges of scalability, interoperability, and sustainability through peer-reviewed research, formal methods, and a layered architecture.

Cardano is distinctive in its *research-first* approach. Every major design decision—the Ouroboros consensus protocol, the EUTxO ledger model, the Plutus smart-contract platform—was first published as a peer-reviewed academic paper and subjected to formal verification before being implemented. This makes Cardano one of the most rigorously designed blockchain platforms in existence.

1.6.2 Architecture and Timing

Cardano’s architecture is divided into two layers:

- **Cardano Settlement Layer (CSL):** Handles the transfer of ada (Cardano’s native cryptocurrency) and native tokens. This is the accounting layer.
- **Cardano Computation Layer (CCL):** Handles smart contracts (Plutus and Aiken scripts). Separating computation from settlement allows each layer to be optimised independently.

Key timing parameters:

- **Slot duration:** 1 second.
- **Block production:** A block is produced approximately every **20 seconds** on average (not every slot has a block; the slot leader lottery means roughly 1 in 20 slots produce a block).
- **Epoch length:** 432 000 slots = **5 days**.
- **Transaction finality:** After approximately **5–10 minutes** (about 15–30 blocks), a transaction is considered practically irreversible. For high-value applications, waiting for one full epoch provides overwhelming assurance.

1.6.3 Native Multi-Asset Support

One of Cardano’s most powerful features—and one we rely on heavily for credential verification—is **native multi-asset support**. On Ethereum, creating a new type of token requires deploying a smart contract (e.g., an ERC-20 contract). On Cardano, custom tokens are *native*: they live directly on the ledger, are transferred using the same mechanisms as ada, and incur no additional smart-contract execution cost for simple transfers.

Every native token is identified by a **policy ID** (the hash of the minting policy script that controls its creation) and a **token name** (an arbitrary byte string up to 32 bytes). The minting policy script defines the rules under which tokens can be minted (created) or burned (destroyed). For credential tokens, we will write policies that allow only the authorised licensing body to mint, and that make the tokens non-fungible (each one is unique to a specific licence holder).

1.6.4 Formal Verification Heritage

Cardano’s smart-contract language, Plutus, is based on Haskell—a purely functional programming language with a strong type system. This choice was deliberate: pure functions and strong types make it dramatically easier to formally verify that a program does what it claims. Additionally, the newer **Aiken** language provides a more developer-friendly syntax while compiling to the same underlying Plutus Core, offering the same on-chain guarantees with a gentler learning curve.

For a credential system, formal verification is not an academic luxury—it is a practical necessity. A bug in a minting policy could allow counterfeit licences; a bug in a validator could lock legitimate credentials forever. Cardano’s tooling helps us avoid these catastrophic outcomes.

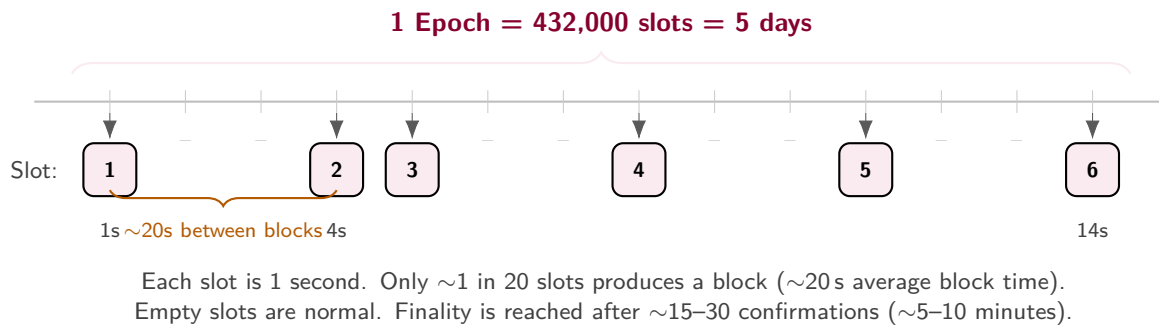


Figure 1.6: Cardano timing: slots, blocks, and epochs. Not every slot produces a block; the average inter-block time is approximately 20 seconds.

Chapter Summary

Blockchain	Append-only ledger of hash-linked blocks; immutable once confirmed.
Consensus	Ouroboros PoS: validators stake ada; no mining hardware needed.
Cryptography	Public/private key pairs enable digital signatures and ownership.
Hashing	One-way function producing fixed-size fingerprints (Blake2b on Cardano).
UTxO model	Transactions consume UTxOs and create new ones (like spending cash).
Cardano	Third-gen, peer-reviewed, EUTxO, native tokens, ~20s blocks, ~5 min finality.

Chapter 2

Cardano’s Extended UTxO Model and Architecture

In Chapter 1 we established the foundational concepts of blockchain technology: cryptographic hashing, public-key cryptography, consensus mechanisms, and the distinction between the account model (Ethereum) and the UTxO model (Bitcoin). We saw that Bitcoin’s Unspent Transaction Output model treats value like physical coins—each output is a discrete, indivisible unit that must be consumed entirely when spent, with any excess returned as “change.” This chapter takes that foundation and builds upon it, exploring how Cardano extends the UTxO model into something far more powerful: a programmable, data-rich ledger capable of supporting complex credential verification workflows.

Understanding Cardano’s architecture is not merely academic. Every design decision in the credential verification system described in later chapters—from how license tokens are minted, to how revocation propagates, to how transaction costs are predicted—flows directly from the architectural properties covered here. By the end of this chapter you will understand *why* Cardano was chosen for this system, not just *that* it was chosen.

2.1 The Extended UTxO (eUTxO) Model

2.1.1 From Bitcoin’s UTxO to Cardano’s eUTxO

Recall from Chapter 1 that Bitcoin’s UTxO model works like a collection of sealed envelopes, each containing a specific amount of currency and a lock that only the recipient’s private key can open. When Alice sends Bob 3 BTC, she does not modify a balance in an account database. Instead, she *consumes* one or more existing UTxOs she controls and *produces* new UTxOs: one paying Bob 3 BTC, and possibly another returning change to herself. The consumed UTxOs cease to exist; the new UTxOs become available for future spending. This is fundamentally different from the account model, where a transfer simply decrements one balance and increments another.

Bitcoin’s UTxOs, however, are limited. Each UTxO carries only two pieces of information: a value (how much BTC it holds) and a locking script (who can spend it, typically requiring a specific public key signature). There is no way to attach arbitrary data to a UTxO, no way for

a spending transaction to provide arguments to the locking script, and the locking script itself has only minimal awareness of the transaction that attempts to spend it.

Cardano’s Extended UTxO model preserves the core advantages of the UTxO approach—determinism, parallelism, and simple local validation—while adding three critical extensions that transform UTxOs from simple value carriers into programmable data containers. These three extensions are **Datums**, **Redeemers**, and **Script Context**.

2.1.2 Datums: Data Attached to UTxOs

A **Datum** is an arbitrary piece of structured data that can be attached to any UTxO. Think of it as a label glued to the outside of the sealed envelope. In Bitcoin, the envelope only tells you how much money is inside and whose key opens it. In Cardano, the envelope can also carry a data payload—a license number, an expiration date, a list of authorized signers, a document hash, or any other structured information that a validator script might need to examine.

Datums are encoded using Cardano’s Plutus data format (CBOR-serialized algebraic data types), and they can range from simple integers to deeply nested records with multiple fields. For credential verification, datums are the mechanism that turns a native token from a bare proof of existence into a rich credential record. A license NFT’s reference token, for example, carries an inline datum containing the license type, jurisdiction, issue date, licensee identity, and a mutable status field—all readable by any party querying the blockchain.

Before the Vasil hard fork (September 2022), datums had to be provided separately from the UTxO and matched by hash, which required the spending party to know the datum’s content in advance. The introduction of **inline datums** (CIP-32) changed this: the datum is stored directly within the UTxO on the ledger, making it readable by anyone without requiring off-chain coordination. This is what makes instant credential verification possible—a verifier simply reads the inline datum from the reference token UTxO.

2.1.3 Redeemers: Data Provided When Spending

A **Redeemer** is a piece of data provided by the party attempting to spend a script-locked UTXO. If the datum is the label on the envelope, the redeemer is the message you hand to the guard who decides whether to let you open it. The redeemer does not exist on the blockchain until the moment a spending transaction is submitted—it is transaction-specific, ephemeral data that the validator script receives as an argument.

Redeemers enable a single validator script to support multiple operations. For example, the Signature Collection Validator in our credential system accepts three different redeemer values: “Collect” (a signer deposits tokens), “Finalize” (the authority completes the process), and “Reclaim” (a signer cancels and retrieves tokens). The script examines the redeemer to determine which set of validation rules to apply, then checks the transaction against those rules. This is analogous to a single contract with multiple clauses—the redeemer selects which clause is being invoked.

The redeemer pattern is powerful because it separates the *what* (the action being performed) from the *how* (the validation logic). A single script address can manage complex multi-step workflows without requiring separate contracts for each step.

2.1.4 Script Context: The Transaction in Full

The **Script Context** is the complete set of information about the transaction that is made available to the validator script during execution. This includes all inputs being consumed, all outputs being produced, all minting and burning operations, all signatures present, the valid time range (slot interval), and the transaction fee. The script context is what gives Cardano validators their power: they can inspect the *entire* transaction, not just the single UTxO being spent.

This is a crucial distinction from Bitcoin's Script, which can only see the specific input being validated. A Cardano validator can enforce rules like "this UTxO may only be spent if the transaction also produces an output paying at least 50 ADA to address X" or "this UTxO may only be consumed if the transaction mints exactly one token under policy Y." For credential systems, script context enables validators to verify that a signer's deposit transaction includes the correct tokens, references the correct document hash, and falls within the validity window—all in a single atomic check.

2.1.5 Determinism: The Key Advantage

The combination of datums, redeemers, and script context produces a property that is perhaps the eUTxO model's most important advantage over account-based models: **deterministic execution**. Given the same inputs, datum, redeemer, and script context, a Cardano validator always produces the same result, regardless of what other transactions are happening on the network at the same time.

In Ethereum's account model, the outcome of a smart contract call depends on the global state at the moment of execution, which can change between the time a transaction is submitted and when it is included in a block. This leads to failed transactions that still consume gas, front-running attacks where miners reorder transactions for profit, and unpredictable costs. In Cardano's eUTxO model, you can validate a transaction completely off-chain before submitting it. If it validates locally, it *will* validate on-chain. The fee is known exactly. The outcome is guaranteed. For a credential verification system handling professional licenses and legal signatures, this predictability is not merely convenient—it is essential.

2.2 Native Multi-Asset Framework

2.2.1 Tokens Without Smart Contracts

One of Cardano's most distinctive architectural features—and a primary reason it was chosen for the credential verification system—is its **native multi-asset framework**. On Ethereum, creating a custom token requires deploying a smart contract (ERC-20 for fungible tokens, ERC-721 for non-fungible tokens). Every transfer of that token requires executing the smart contract, consuming gas, and relying on the contract code being correct. A bug in an ERC-20 contract can freeze or drain every token it manages, as numerous exploits have demonstrated.

On Cardano, custom tokens—both fungible and non-fungible—exist at the *ledger level*, the same level as ADA itself. They are tracked by the same UTxO accounting rules, validated by the same consensus mechanism, and transferred with the same transaction logic. No smart contract execution is required to transfer a native token from one address to another. The token simply moves from one UTxO to another, just like ADA. This means that native tokens

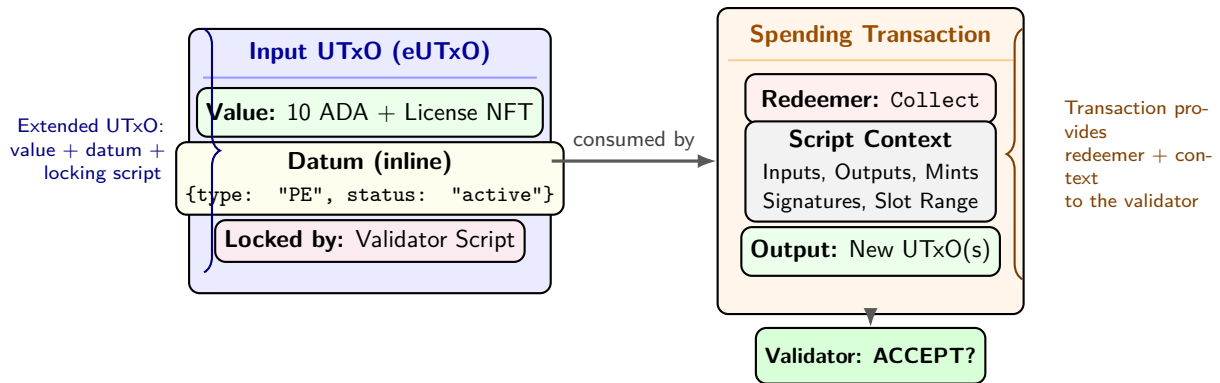


Figure 2.1: Anatomy of an Extended UTxO and a spending transaction. The input UTxO carries a value, an inline datum, and is locked by a validator script. The spending transaction provides a redeemer (the action to perform) and the script context (complete transaction data). The validator examines all three—datum, redeemer, context—to decide whether to accept or reject the spend.

inherit all of the security guarantees of the base currency: they cannot be double-spent, they cannot be created out of thin air (except by an authorized minting policy), and they cannot be frozen by a buggy contract.

For a credential verification system, this architecture is transformative. A professional license NFT on Cardano is not a record in a smart contract’s internal storage—it is a first-class asset on the ledger, as “real” as ADA itself. Transferring it costs only a standard transaction fee (~0.17 ADA), not a smart contract execution fee. Verifying its existence requires only a UTxO query, not a contract call.

2.2.2 Policy ID and Asset Name

Every native token on Cardano is identified by a two-part identifier:

Policy ID A 28-byte (56 hex character) Blake2b-224 hash of the **minting policy script**. The minting policy is the script that defines who can create (mint) or destroy (burn) tokens under this policy. Because the Policy ID is a cryptographic hash of the script, it is deterministic and unforgeable—anyone can independently verify which script produced a given Policy ID. If the California Board of Professional Engineers publishes their Policy ID, any verifier can confirm that a license token was minted under that exact policy.

Asset Name A human-readable (or arbitrary byte string) name within a policy, up to 32 bytes. For credential tokens, this typically encodes the credential type or identifier, such as `LIC_PE_2026_00042` or `SIG_TOKEN`. The combination of Policy ID and Asset Name uniquely identifies every token on the Cardano ledger.

This two-part identification scheme is analogous to a domain name system: the Policy ID is like the domain (controlled by the authority), and the Asset Name is like a path within that domain (chosen by the authority for each token type). A forger cannot create a token with the California Board’s Policy ID because they do not possess the authority key that the minting policy requires.

2.2.3 Multi-Asset Bundles in UTxOs

A single Cardano UTxO can carry *multiple* tokens alongside ADA. This is called a **multi-asset bundle** (or simply a “value bundle”). Unlike Bitcoin, where each UTxO carries only the native currency, a Cardano UTxO’s value field is a nested map:

```
Value = Map<PolicyId, Map<AssetName, Quantity>>
```

A single UTxO might contain 5 ADA, plus 100 signature tokens under Policy A, plus 1 license NFT under Policy B, plus 1 validity token under Policy C. This bundling capability is essential for the credential system’s signature workflow, where a signer must deposit multiple token types in a single atomic transaction.

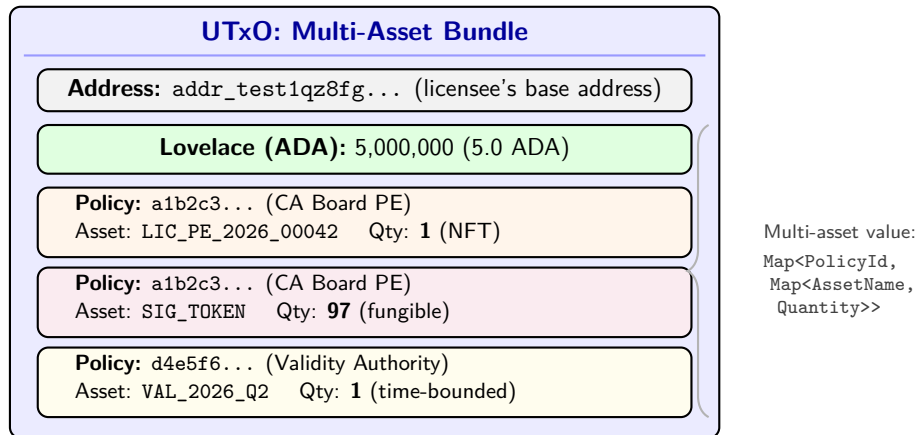


Figure 2.2: A single Cardano UTxO carrying a multi-asset bundle: ADA (required for the minimum UTxO deposit), a license NFT, 97 fungible signature tokens, and a time-bounded validity token. All reside in a single UTxO under the licensee’s address. The tokens span two different Policy IDs, demonstrating that a UTxO can hold tokens from multiple authorities.

2.3 Minimum UTxO (minUTxO) Deposit

2.3.1 The Problem: Ledger Bloat

Every UTxO on the Cardano blockchain consumes storage space on every full node in the network. Unlike account-based systems where an empty account can be cheaply represented (or garbage-collected), every UTxO persists until explicitly consumed by a transaction. If creating UTxOs were free, an attacker could create billions of tiny, worthless UTxOs to bloat the ledger, degrading performance for all participants. This is known as the **dust attack** problem.

Cardano addresses this with the **minimum UTxO deposit** requirement: every UTxO must contain a minimum amount of ADA, proportional to the size of the UTxO (including any native tokens and datums it carries). The formula is based on a protocol parameter called `coinsPerUTxOByte` (currently 4,310 lovelace per byte). In practice, this works out to approximately:

- **~1.0 ADA** for a simple ADA-only UTxO
- **~1.5 ADA** for a UTxO carrying a single native token
- **~2.0 ADA** for a UTxO carrying multiple tokens or a datum

- **~2.5–5.0 ADA** for a UTxO with a large inline datum (e.g., CIP-68 reference tokens)

2.3.2 The Apartment Deposit Analogy

The minUTxO deposit works exactly like a security deposit on an apartment. When you move in (create a UTxO carrying native tokens), the landlord (the Cardano protocol) requires you to put down a deposit (the minimum ADA). This deposit is not a fee—it is not spent or consumed. It sits in the UTxO, locked alongside your tokens. When you move out (consume the UTxO by spending its contents), you get the deposit back. The ADA returns to you as part of the transaction's change output.

This has a direct impact on the economics of the credential system. Every license NFT, every batch of signature tokens, and every validity token requires an accompanying ADA deposit of approximately 1.5–2.0 ADA. For a board issuing 10,000 licenses, this means roughly 20,000 ADA (~\$6,000 at \$0.30/ADA) must be available as working capital for deposits, even though the ADA is fully recoverable when tokens are burned or consolidated.

2.3.3 Why This Matters for Credential Systems

Understanding the minUTxO deposit is crucial for three reasons:

1. **Cost Planning.** While the deposit is recoverable, it represents locked capital. A licensing board must budget for both transaction fees (consumed, non-recoverable) and minUTxO deposits (locked, recoverable).
2. **Token Consolidation.** If a licensee accumulates many small UTxOs (e.g., receiving signature tokens in separate batches), the total locked ADA can grow substantially. Periodic consolidation of UTxOs reduces the total deposit overhead.
3. **Burn Recovery.** When a credential is revoked and the token burned, the minUTxO deposit is returned. This creates a natural economic incentive to clean up expired or revoked credentials, keeping the ledger tidy.

2.4 Slots, Epochs, and Time

2.4.1 Cardano's Time Model

Cardano does not use wall-clock time directly. Instead, time on the blockchain is measured in **slots**—discrete, sequential time intervals. Understanding this time model is essential because the credential system uses slot-based expiry for time-bounded validity tokens, and every transaction specifies a valid slot range.

Slot The fundamental unit of time on Cardano. Each slot is exactly **1 second** long. Slots are numbered sequentially from the genesis block. At the time of writing, the Cardano mainnet is at slot ~140,000,000+. A slot leader (the stake pool selected to produce the next block) has exactly one slot in which to create and propagate their block.

Block Not every slot produces a block. On average, a block is produced every **~20 seconds** (the active slot coefficient is 0.05, meaning 1 in 20 slots produces a block). This 20-second average block time is a protocol parameter that balances throughput against propagation delay across the global network.

Epoch An epoch is a period of **432,000 slots (5 days)**. Epochs are the administrative unit of Cardano: stake delegation changes, reward distribution, and protocol parameter updates all take effect at epoch boundaries. For credential systems, epochs provide a natural granularity for periodic operations like dues payment deadlines.

2.4.2 Slot-Based Transaction Validity

Every Cardano transaction specifies a **validity interval**: a range of slots during which the transaction is valid. A transaction with `invalid_before = 100000` and `invalid_hereafter = 100500` can only be included in a block produced during slots 100,000 through 100,499. If a block producer encounters the transaction outside this window, it is rejected.

This mechanism is critical for time-bounded credentials. A validity token's expiry is expressed as a slot number in its datum. The validator script checks the transaction's validity interval against the token's expiry slot: if the transaction's valid slot range extends beyond the token's expiry, the validator rejects the spend. This ensures that a validity token cannot be used to sign a document after it has expired, even if the licensee still holds it in their wallet.

2.4.3 Converting Between Slots and Wall-Clock Time

Converting between slots and human-readable timestamps is straightforward but requires knowing the chain's genesis parameters. For mainnet:

```

wall_clock = genesis_time + (slot_number * slot_length)
where genesis_time = 2017-09-23T21:44:51Z and slot_length = 1 second
    
```

In practice, the PyCardano library and the Blockfrost API handle this conversion automatically. When the credential system mints a validity token with a 90-day expiry, it calculates the target slot as `current_slot + (90 * 24 * 60 * 60)`.

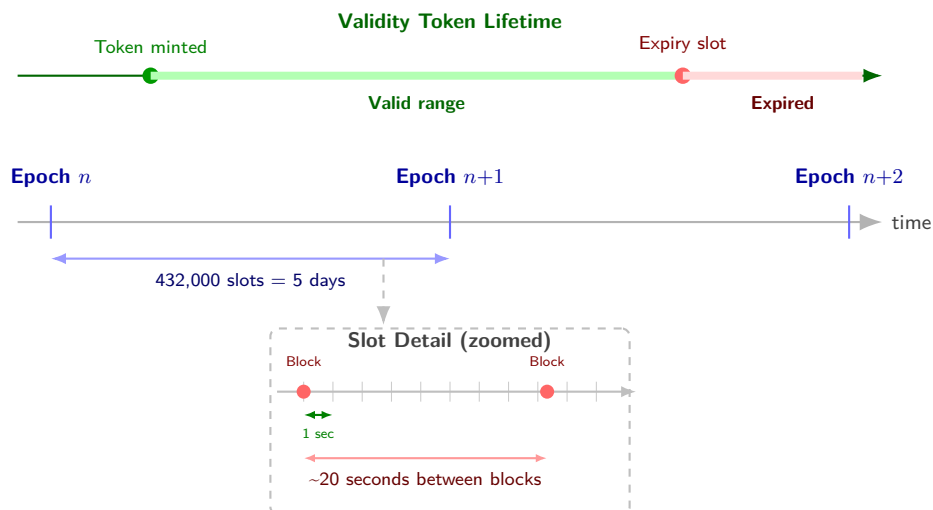


Figure 2.3: Cardano's time model. The upper timeline shows a validity token's lifetime, from minting to slot-based expiry. The middle timeline shows epochs (5 days each). The inset zooms into individual slots (1 second each) and blocks (~20-second intervals). Validators check transaction validity intervals against token expiry slots to enforce time-bounded credentials.

2.5 Transaction Fees

2.5.1 Deterministic Fee Model

Cardano's fee model is one of its most practically important features for production systems. Unlike Ethereum, where the cost of a transaction is determined by a “gas auction” (users bid for block space, and actual costs depend on network congestion at the moment of inclusion), Cardano uses a **deterministic, formula-based fee model**:

$$\text{fee} = A \times \text{tx_size (bytes)} + B$$

where A and B are protocol parameters. At current mainnet values:

- $A = 44$ lovelace per byte (the “per-byte coefficient”)
- $B = 155,381$ lovelace (the “constant fee”; the minimum fee for any transaction)

For a typical native script minting transaction of ~400 bytes:

$$\text{fee} = 44 \times 400 + 155,381 = 173,181 \text{ lovelace} \approx 0.17 \text{ ADA}$$

This means the *exact fee is known before the transaction is submitted*. There is no gas estimation, no fee spike due to network congestion, and no failed transaction that still costs money. You build the transaction, calculate its serialized size, apply the formula, and the result is guaranteed. For a credential system handling thousands of minting and signing operations, this predictability enables precise budgeting and cost forecasting.

2.5.2 Plutus Script Execution Fees

Transactions that execute Plutus validator scripts incur additional fees based on two execution resource metrics: **CPU steps** and **memory units**. The Plutus fee formula adds:

$$\text{script_fee} = \text{cpu_price} \times \text{cpu_steps} + \text{mem_price} \times \text{mem_units}$$

The total transaction fee is the maximum of the size-based fee and the execution-based fee. For credential system operations:

- **Native script operations** (minting, simple transfers): ~0.17 ADA. No Plutus execution, so only the size-based fee applies.
- **Plutus V2 operations** (signature collection, dues enforcement, CIP-68 datum updates): ~0.3–0.5 ADA. The script execution adds CPU and memory costs.
- **Complex Plutus operations** (multi-input finalization consuming 5+ UTxOs): ~0.5–1.5 ADA. More inputs mean more script context for the validator to process.

2.5.3 Comparison with Ethereum Gas

The practical difference is dramatic. During high network activity on Ethereum (e.g., a popular NFT mint), gas prices can spike from a base of ~20 gwei to 500+ gwei, making a \$2 mint cost \$50 or more. A transaction that fails validation *still consumes gas* on Ethereum—the user pays for the failed attempt. On Cardano, fee spikes do not occur because fees depend only on

PKHs must deposit tokens”), and the licensee’s PKH is recorded in license metadata for identity binding.

2.6.3 Bech32 Encoding

Cardano addresses are displayed in **Bech32** encoding, a human-readable format that includes an error-detecting checksum. Mainnet addresses start with **addr1...** and testnet addresses with **addr_test1...** The Bech32 encoding ensures that a mistyped character in an address is detected before a transaction is submitted, preventing accidental loss of funds. The prefix also prevents accidentally sending mainnet transactions on the testnet or vice versa.

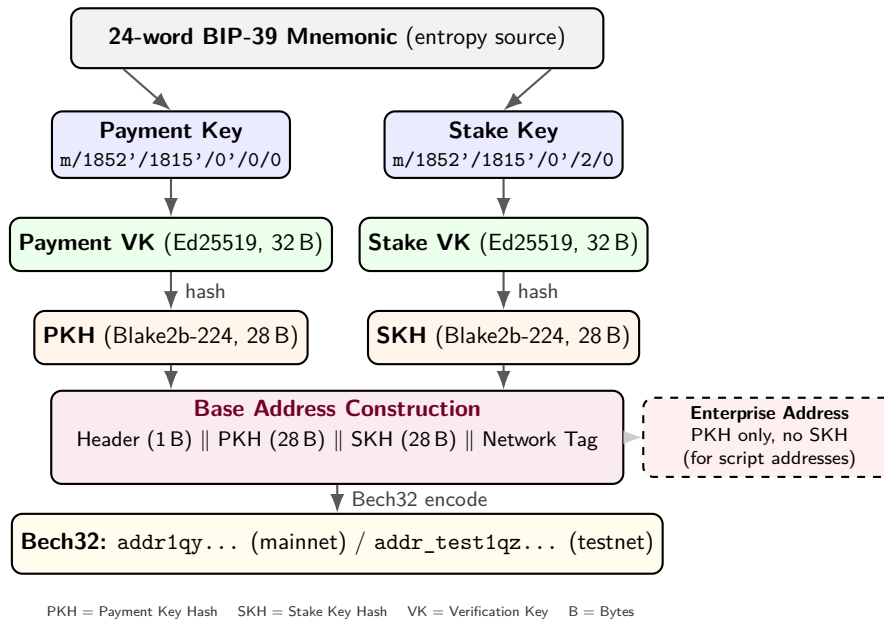


Figure 2.4: Cardano address construction from mnemonic to Bech32-encoded address. A 24-word mnemonic seeds HD key derivation (CIP-1852), producing payment and stake verification keys. Each is hashed with Blake2b-224 to produce 28-byte key hashes. A base address combines both hashes with a network tag; an enterprise address uses only the payment key hash.

2.7 UTxO Selection Strategies

2.7.1 The UTxO Selection Problem

When building a transaction, you must select which UTxOs to consume as inputs. This is a non-trivial optimization problem. Consider a licensee who has received signature tokens in 20 separate transactions, each creating a separate UTxO. To sign a document (requiring 1 signature token and 1 validity token), the transaction builder must choose which UTxO(s) to consume. The choice affects:

1. **Transaction size.** Each input adds ~40–60 bytes to the transaction. More inputs mean a larger transaction and a higher fee.
2. **Change outputs.** If a UTxO contains more value than needed, the excess must be returned as a change output, which itself requires a minUTxO deposit.

3. **UTxO set health.** Poor selection strategies can fragment the UTxO set into many tiny outputs, each locking a minUTxO deposit, gradually eroding the wallet's effective balance.
4. **Transaction size limit.** Cardano enforces a maximum transaction size of **16,384 bytes (16 KB)**. A transaction that consumes too many inputs may exceed this limit and fail to build.

2.7.2 LargestFirstSelector

The **LargestFirstSelector** strategy sorts all available UTxOs by value (largest first) and greedily selects UTxOs until the required value is covered. This is the default strategy in PyCardano and the one used by the credential system.

Advantages:

- Minimizes the number of inputs (and therefore transaction size) by preferring large UTxOs.
- Produces fewer, larger change outputs, reducing UTxO fragmentation over time.
- Simple and predictable—the same inputs produce the same selection.

Disadvantages:

- Large UTxOs are consumed first, potentially leaving a “long tail” of small UTxOs that are individually too small to cover transaction fees.
- Deterministic selection can lead to repeated use of the same UTxOs, causing temporary contention in high-throughput scenarios.

2.7.3 RandomImproveSelector

The **RandomImproveSelector** takes a different approach. It first makes a random initial selection that covers the required value, then iteratively attempts to improve the selection by swapping UTxOs to minimize the change output. The goal is to produce a change output as close to zero as possible, reducing the number of dust UTxOs over time.

Advantages:

- Distributes UTxO consumption more evenly across the wallet's UTxO set, reducing fragmentation over time.
- The randomness reduces contention in scenarios where multiple transactions are built concurrently (relevant for high-volume minting operations).
- Tends to produce a healthier UTxO distribution in the long run.

Disadvantages:

- Non-deterministic: building the same transaction twice may select different inputs, complicating testing and debugging.
- May select more inputs than **LargestFirstSelector** for a given transaction, increasing fees slightly.

2.7.4 Practical Implications

For the credential system, UTxO selection strategy matters most during two operations:

1. **Batch minting.** When minting 100 signature tokens for a new licensee, the authority wallet must have a UTxO with enough ADA to cover fees and the minUTxO deposit. If the authority's ADA is fragmented across dozens of small UTxOs, the transaction builder may need to consume many inputs, potentially exceeding the 16 KB limit.
2. **Token consolidation.** A licensee who has signed many documents will have accumulated change UTxOs. Periodic consolidation (sending all UTxOs back to oneself in a single transaction) reduces the UTxO count and recovers locked minUTxO deposits.

The credential system uses `LargestFirstSelector` by default with a 200,000 lovelace fee buffer, switching to `RandomImproveSelector` for high-volume operations where concurrent transaction building might cause contention on the same large UTxOs.

2.8 Chapter Summary

This chapter has covered the seven architectural pillars of Cardano that underpin the credential verification system:

1. **The eUTxO Model** extends Bitcoin's UTxO with datums (data on UTxOs), redeemers (spending arguments), and script context (full transaction visibility), enabling deterministic smart contract execution.
2. **Native Multi-Assets** allow tokens to exist at the ledger level without smart contracts, providing the same security guarantees as ADA at lower cost.
3. **The minUTxO Deposit** prevents ledger bloat by requiring a recoverable ADA deposit proportional to UTxO size—a cost that must be budgeted but is not consumed.
4. **Slots, Epochs, and Time** provide the discrete time model that enables slot-based token expiry and time-bounded transaction validity.
5. **Deterministic Fees** ensure that transaction costs are known exactly before submission, enabling precise cost forecasting for credential operations.
6. **Address Construction** from HD-derived key hashes provides the identity primitives (PKH, SKH) used throughout the system for access control and credential binding.
7. **UTxO Selection Strategies** balance transaction efficiency against UTxO set health, with implications for high-volume minting and long-term wallet management.

With this architectural foundation in place, Chapter 3 will introduce the specific token types, minting policies, and smart contracts that implement the credential verification system on top of these primitives.

Chapter 3

Cardano Improvement Proposals (CIPs)

Cardano’s technical evolution is governed by a structured, community-driven process called the *Cardano Improvement Proposal* (CIP) system. Every significant feature—from how wallets derive keys to how smart contracts reference on-chain data—begins life as a numbered CIP, undergoes public review, and is either accepted, modified, or rejected before implementation. Understanding the CIP landscape is essential for anyone building on Cardano because CIPs define the *interface contracts* that all tools, wallets, and dApps must honour.

This chapter examines the eight CIPs most relevant to the credential verification system described in this guide. We begin with the governance process itself, then walk through the token metadata standards (CIP-25 and CIP-68), the wallet connectivity bridge (CIP-30), the Plutus V2 trio that made advanced datum handling possible (CIPs 31, 32, and 33), and the hierarchical deterministic wallet derivation standard (CIP-1852). For each CIP we explain *what problem it solves, how it works technically, and how the credential system uses it*.

3.1 What Are CIPs?

A Cardano Improvement Proposal is a formal design document that describes a new feature, process, or informational guideline for the Cardano ecosystem. The CIP process is modelled directly on the Internet Engineering Task Force’s *Request for Comments* (RFC) system and Python’s PEP process: anyone may author a proposal, but acceptance requires community review, editor approval, and—for protocol-level changes—implementation by the core development teams at Input Output Global (IOG), the Cardano Foundation, or Emurgo.

CIPs are classified into three categories:

Standards Track

Proposals that change the protocol, transaction format, validation rules, or networking layer. Examples include CIP-31 (Reference Inputs) and CIP-32 (Inline Datums).

Process

Proposals that change governance procedures, the CIP process itself, or tooling requirements.

Informational

Proposals that document conventions, best practices, or design rationale without mandating

changes. CIP-25 (NFT Metadata) began as an informational standard that the community adopted voluntarily.

The lifecycle of a CIP proceeds through several stages: **Draft** (initial submission to the CIP repository on GitHub), **Proposed** (editors confirm the proposal is well-formed), **Active** (the community has reached rough consensus and implementations exist), and optionally **Inactive** or **Superseded** if a newer CIP replaces it.

For the credential system, CIPs are not merely background reading—they are the *specifications* that the code implements. The minting policy encodes CIP-25 metadata structures. The wallet module follows CIP-1852 derivation paths. The revocation mechanism depends on CIP-68 datum semantics. Understanding each CIP at a technical level is prerequisite to understanding why the system works the way it does.

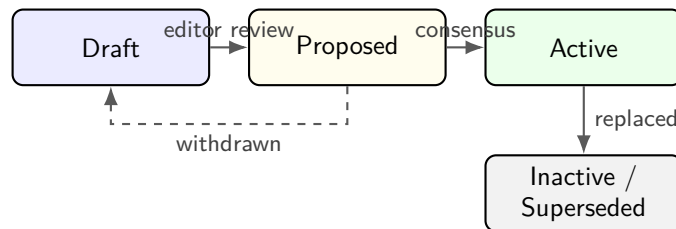


Figure 3.1: CIP lifecycle stages. A proposal moves from Draft through Proposed to Active once community consensus is reached and implementations exist. Proposals may be withdrawn or eventually superseded by newer standards.

3.2 CIP-25: NFT Metadata Standard

3.2.1 The Problem

Cardano’s native multi-asset framework allows anyone to mint custom tokens, but the ledger itself stores only three pieces of information about each token: the *policy ID* (a Blake2b-224 hash of the minting policy script), the *asset name* (an arbitrary byte string up to 32 bytes), and the *quantity*. There is no built-in mechanism for attaching human-readable names, images, descriptions, or any other metadata to a token. Without a standard, every marketplace, wallet, and explorer would need to negotiate metadata formats bilaterally—an untenable situation for an open ecosystem.

3.2.2 How It Works

CIP-25 solves this by defining a JSON schema that is embedded in the **transaction auxiliary data** (formerly called “transaction metadata”) at the time the token is minted. The auxiliary data is a map keyed by integer labels; CIP-25 reserves label 721 for NFT metadata.

The schema is hierarchical: under label 721, the top-level key is the **policy ID** (hex-encoded). Under each policy ID, keys are **asset names** (hex or UTF-8). Under each asset name, the metadata object contains at minimum:

- **name** — a human-readable display name
- **image** — a URI (typically IPFS) pointing to the token’s image
- **description** — free-text description

Additional fields are permitted and commonly used: `mediaType`, `files` (for multi-asset bundles), and arbitrary key-value pairs. Wallets and marketplaces parse the minting transaction’s auxiliary data to display this information to users.

A critical property of CIP-25 metadata is that it is **immutable once written**. The metadata exists in the auxiliary data of a specific transaction. Transactions, once confirmed on-chain, cannot be altered. This means that whatever metadata is attached at mint time is the metadata forever—there is no mechanism within CIP-25 to update, correct, or revoke the metadata after minting.

For simple NFTs—artwork, collectibles, event tickets—immutability is a feature. For credentials that may need to reflect status changes (revocation, suspension, renewal), immutability is a fundamental limitation. This limitation is precisely what motivated CIP-68.

3.2.3 How the Credential System Uses CIP-25

In the credential system, CIP-25 metadata is attached to license NFTs at mint time. The metadata records the license type, jurisdiction, issue date, license number, and the licensee’s public key hash. Because this information is immutable, it serves as a permanent attestation: “At this point in time, this authority issued this credential to this individual.”

Verification of the attestation is straightforward: a verifier checks that (1) the token exists in the holder’s wallet, (2) the policy ID matches the known authority, and (3) the CIP-25 metadata in the minting transaction matches the claimed credential details.

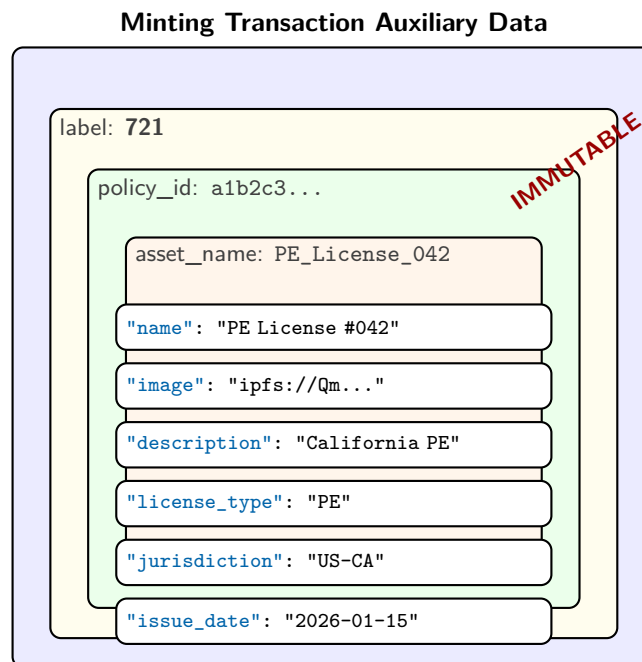


Figure 3.2: CIP-25 metadata structure. The JSON schema is nested under label 721, then by policy ID and asset name. Once written to the transaction auxiliary data, the metadata cannot be changed.

3.3 CIP-68: Datum Metadata Standard

CIP-68 is the single most important CIP for the credential verification system. It solves the fundamental limitation of CIP-25—immutable metadata—by introducing a **dual-token pattern** that separates ownership from metadata, enabling authority-controlled updates without requiring access to the token holder’s private key. This section is intentionally the longest in the chapter because understanding CIP-68 is prerequisite to understanding how credential revocation works.

3.3.1 The Problem CIP-68 Solves

Consider a professional engineering license issued as a CIP-25 NFT. The metadata at mint time includes `status: active`. Two years later, the licensing board discovers misconduct and needs to revoke the license. Under CIP-25, this is *impossible on-chain*: the metadata is baked into the minting transaction and cannot be altered. The board’s only options are:

1. **Burn and re-mint.** The authority could burn the original token and mint a new one with `status: revoked`. But burning requires consuming the UTxO that holds the token—which is in the *licensee’s* wallet. Under the eUTxO model, spending a UTxO requires the holder’s private key. A revoked licensee has no incentive to cooperate, and the authority *cannot* spend the licensee’s UTxO without that key.
2. **Off-chain revocation list.** The authority maintains a database of revoked credentials, and verifiers must check both the on-chain token and the off-chain list. This reintroduces centralized infrastructure, single points of failure, and availability dependencies —exactly the problems blockchain was supposed to solve.
3. **Accept that on-chain revocation is impossible.** Many early Cardano credential projects reached this conclusion and treated the blockchain as a timestamping service only.

CIP-68 provides a fourth option: **mutable on-chain metadata** through a clever separation of concerns.

3.3.2 The Dual-Token Pattern

CIP-68 defines a naming convention using **asset name labels**—a 4-byte prefix prepended to the asset name that identifies the token’s role. The two most important labels are:

Label 100 — Reference Token

Held by the **issuing authority**. This token sits at a script address (not a personal wallet) and carries an **inline datum** containing all mutable metadata: license type, jurisdiction, issue date, status, and any other fields the authority may need to update. Because the Reference Token is at a script address controlled by the authority, the authority can spend and re-create this UTxO with an updated datum at any time, without any involvement from the token holder.

Label 222 — User Token

Held by the **credential holder** (the licensee). This token is a standard native asset that lives in the holder’s personal wallet. It is *immutable* in the CIP-25 sense—the holder possesses it as proof of credential issuance. The User Token contains no mutable metadata; its sole purpose is to attest that the holder was issued a credential under a specific policy.

The two tokens share the same policy ID and base asset name, differing only in their 4-byte label prefix. This creates a cryptographic link: given a User Token with asset name `000de140 + PE_License_042`, anyone can deterministically compute the corresponding Reference Token's asset name as `000643b0 + PE_License_042` and look up its inline datum to read the current metadata.

3.3.3 How Revocation Works

The breakthrough insight of CIP-68 for credential systems is this: the authority can **revoke a credential by updating the Reference Token's datum** from `status: active` to `status: revoked`. This operation:

- Requires **only the authority's signature** (the Reference Token is at the authority's script address).
- Does **not touch the User Token** in the licensee's wallet—the authority never needs the licensee's private key.
- Propagates **globally within minutes** (Cardano's finality window).
- Is **verifiable by anyone**: read the Reference Token's inline datum, check the status field.
- Is **auditable**: the chain of datum updates (`active` → `suspended` → `revoked`) is visible in the transaction history.

This is the mechanism that makes eUTxO-based credential revocation possible without off-chain infrastructure, without the holder's cooperation, and without burning the original token.

3.3.4 Technical Details

The Reference Token's inline datum follows a standard structure defined by CIP-68:

```
{
  "constructor": 0,
  "fields": [
    {
      // metadata map
      "license_type": "PE",
      "jurisdiction": "US-CA",
      "license_number": "PE-2026-00042",
      "issue_date": "2026-01-15",
      "holder_pkh": "abc123...",
      "status": "active", // <-- MUTABLE
      "last_updated": 12345678 // slot number
    },
    1 // version number
  ]
}
```

Listing 3.1: CIP-68 datum structure (simplified)

The datum is encoded as CBOR (Concise Binary Object Representation) and stored directly in the UTxO as an inline datum (CIP-32). The `status` field can hold any of: `active`, `suspended`, `revoked`, or `expired`. Each status change creates a new transaction on-chain, providing a complete audit trail.

The script address holding the Reference Token enforces that only the authority's key can spend

(and re-create) the UTxO. This is typically implemented as a simple `ScriptPubkey` validator or a Plutus V2 spending validator that checks for the authority’s signature in the transaction signatories.

3.3.5 Verification Flow

When a verifier wants to check a credential, the process is:

1. Observe the User Token (label 222) in the holder’s wallet.
2. Extract the policy ID and base asset name.
3. Compute the Reference Token (label 100) asset name.
4. Query the blockchain for the UTxO containing the Reference Token.
5. Read the inline datum from that UTxO.
6. Check the `status` field.

This entire process requires only standard blockchain queries—no off-chain service, no authority server, no API key. The data is public, the datum is inline (CIP-32), and the verification is purely mathematical.

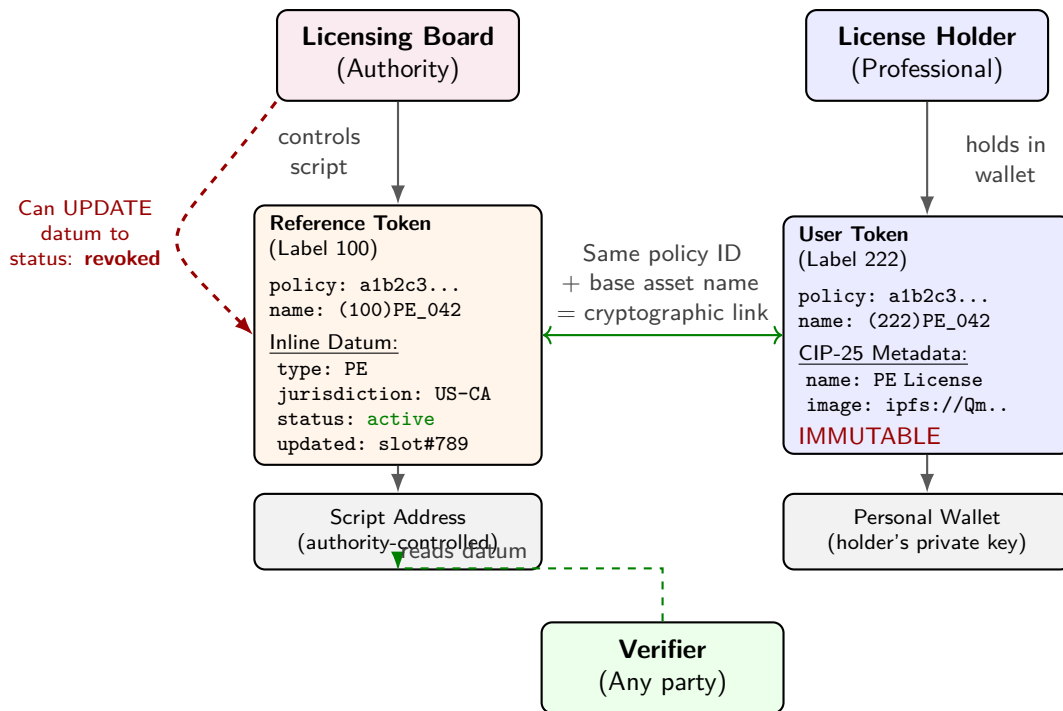


Figure 3.3: CIP-68 dual-token pattern for credential management. The authority controls the Reference Token (label 100) at a script address and can update its inline datum—including the status field—at any time. The holder possesses the immutable User Token (label 222) in their personal wallet. A verifier reads the Reference Token’s datum to determine current credential status. The authority never needs the holder’s private key to revoke.

3.3.6 CIP-25 vs. CIP-68: Comparison

Table 3.1 summarizes the key differences between the two metadata standards. For the credential system, CIP-68’s mutable datum is the enabling feature that makes on-chain revocation practical.

Table 3.1: Comparison of CIP-25 and CIP-68 metadata standards

Property	CIP-25	CIP-68
Metadata location	Transaction auxiliary data (label 721)	Inline datum on Reference Token UTxO
Mutability	Immutable — written once at mint	Mutable — authority can update datum
Token count	1 token per credential	2 tokens: Reference (100) + User (222)
Who holds metadata	Embedded in minting tx (no custodian)	Reference Token at authority script address
Revocation	Not possible on-chain	Datum update: <code>status</code> → revoked
Authority key needed	Only at mint time	At mint <i>and</i> for every datum update
Holder key needed	Only to transfer/burn the token	Only to transfer/burn the User Token
On-chain cost	~0.2 ADA (single token mint)	~0.4 ADA (two tokens + datum)
Reading metadata	Parse minting tx auxiliary data	Read inline datum from Reference UTxO
Audit trail	Single minting event	Full history of datum updates on-chain
Requires CIP-31	No	Yes (reference inputs for reading datum)
Requires CIP-32	No	Yes (inline datums for storing datum)

3.3.7 Why CIP-68 Matters for Credentials

The importance of CIP-68 for the credential system cannot be overstated. Before CIP-68, any Cardano-based credential project faced an impossible trilemma: (1) you could store metadata on-chain but it was immutable; (2) you could store metadata off-chain but that reintroduced centralization; or (3) you could accept that revocation was not truly decentralized.

CIP-68 breaks this trilemma by enabling **mutable on-chain metadata under authority control**. The result is a system where:

- Credential issuance is on-chain and cryptographically verifiable.
- Credential status is on-chain and always current.
- Revocation is unilateral by the authority—no holder cooperation required.
- Verification is fully decentralized—any node can read the datum.
- The audit trail is immutable—every status change is a new transaction.

The dual-token pattern also provides a natural separation between *proof of issuance* (the User Token in the holder’s wallet says “this authority issued me a credential”) and *current status* (the Reference Token’s datum says “the credential is currently active/revoked/suspended”). This separation mirrors real-world credential systems where a paper certificate proves issuance but a database query confirms current status— except that in the CIP-68 model, both proofs are on-chain.

3.4 CIP-30: dApp–Wallet Web Bridge

3.4.1 The Problem

A web-based credential management application needs to interact with a user’s Cardano wallet: reading their address, checking token balances, and—most critically—asking the wallet to sign transactions. Without a standard API, every wallet (Eternl, Nami, Lace, Flint, Typhon, etc.) would expose its own proprietary JavaScript interface, forcing dApp developers to write wallet-specific code and creating fragmentation across the ecosystem.

More importantly, the security model must be **non-custodial**: the dApp must *never* see the user’s private keys. If the application server were compromised, an attacker with access to private keys could drain wallets, forge signatures, or transfer credentials. The wallet must sign transactions locally and return only the signed transaction to the dApp.

3.4.2 How It Works

CIP-30 defines a JavaScript API that Cardano wallets inject into the browser’s `window.cardano` namespace. The API follows a two-phase pattern:

1. **Discovery and Connection.** The dApp checks `window.cardano` for available wallets, then calls `cardano.<wallet>.enable()` to request user authorization. The user sees a popup in their wallet extension asking whether to connect. If approved, the wallet returns an API handle.
2. **Interaction.** The dApp uses the API handle to call methods such as `getUsedAddresses()`, `getUtxos()`, `getBalance()`, and—most importantly—`signTx(tx, partialSign)`. The `signTx` method takes an unsigned transaction (CBOR-encoded), displays it to the user for approval, signs it with the wallet’s private key, and returns the signature (or signed transaction).

The critical security property is that the private key *never leaves the wallet extension*. The dApp constructs the transaction, but the wallet signs it. This is analogous to how a hardware wallet works: the application builds the transaction, the device signs it, and the application never sees the signing key.

3.4.3 How the Credential System Uses CIP-30

In the credential system, licensees and signers connect their wallets via CIP-30 to the web dashboard. When a professional needs to sign a document (depositing signature and validity tokens into the work product validator), the application:

1. Builds the transaction off-chain using PyCardano (or a JavaScript library in the browser).
2. Sends the unsigned transaction CBOR to the wallet via `signTx()`.
3. The wallet displays the transaction details (tokens being sent, destination address, fee) and asks the user to confirm.
4. The user approves; the wallet signs and returns the witness.
5. The application attaches the witness and submits the transaction to the network.

At no point does the application server have access to the user’s signing key. Even if the server is compromised, the attacker cannot sign transactions on behalf of users. This non-custodial

model is essential for a credential system where users must maintain sovereign control over their professional identities.

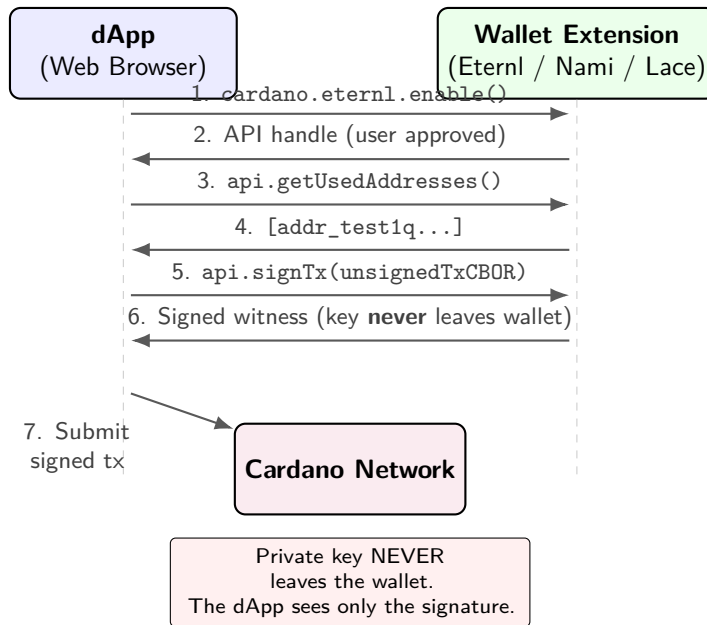


Figure 3.4: CIP-30 dApp–Wallet interaction flow. The web application requests connection, reads addresses and UTxOs, and sends unsigned transactions to the wallet for signing. The wallet extension holds the private key locally and returns only the cryptographic witness. The dApp never has access to the signing key.

3.5 CIP-31: Reference Inputs

3.5.1 The Problem

In the original eUTxO model, every UTxO referenced by a transaction is **consumed** (spent). If a transaction needs to read data from a UTxO—for example, checking a CIP-68 Reference Token’s datum to verify a credential’s status—it must spend that UTxO and recreate it in the same transaction. This creates three problems:

1. **Concurrency.** If two transactions both need to read the same Reference Token datum, they both try to spend the same UTxO. Only one can succeed; the other fails with a UTxO conflict. For a credential that many parties may verify simultaneously, this is unacceptable.
2. **Complexity.** The spending transaction must include the script that guards the UTxO, a redeemer, and must recreate the UTxO with exactly the same value and datum—adding transaction size, fees, and potential for bugs.
3. **Authorization.** Spending a UTxO requires satisfying its spending conditions (e.g., the authority’s signature). A verifier who merely wants to *read* the datum should not need the authority’s key.

3.5.2 How It Works

CIP-31 introduces **reference inputs**: a new transaction field that allows a transaction to *read* a UTxO without consuming it. The UTxO remains untouched on the ledger after the transaction is processed. The transaction’s scripts can access the reference input’s value, datum, and address, but the UTxO is not spent.

This is a Plutus V2 feature introduced in the Vasil hard fork. In the transaction body, reference inputs are listed in a separate field (**reference_inputs**) distinct from regular inputs (**inputs**). The ledger validates that each referenced UTxO exists at the time the transaction is processed but does not remove it from the UTxO set.

3.5.3 How the Credential System Uses CIP-31

CIP-31 is critical for the credential system’s verification flow. When a verifier or a smart contract needs to check a license’s current status, the transaction includes the CIP-68 Reference Token UTxO as a **reference input**. The script reads the inline datum, checks the **status** field, and proceeds—all without spending the Reference Token.

This means:

- Multiple verifications can happen simultaneously (no UTxO contention).
- The verifier does not need the authority’s key.
- The Reference Token UTxO remains stable on-chain.
- Transaction size is smaller (no need to recreate the UTxO).

Without CIP-31, the CIP-68 pattern would be impractical for any credential that is verified frequently, because each verification would compete to spend the same Reference Token UTxO.

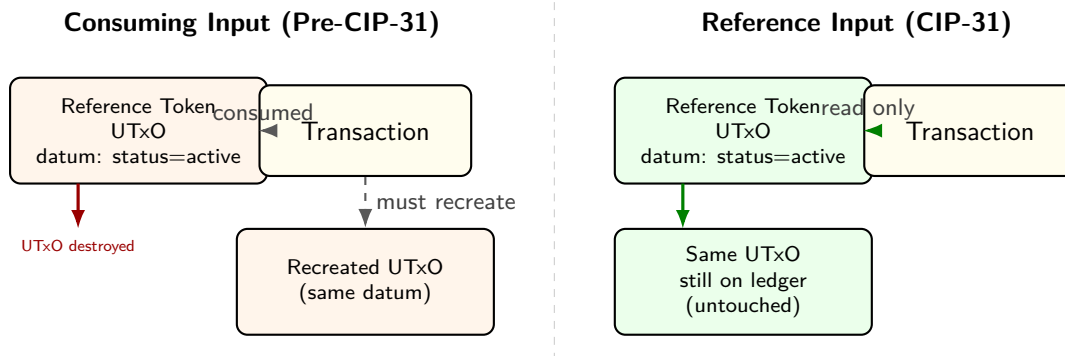


Figure 3.5: CIP-31 reference inputs compared to consuming inputs. On the left, the pre-CIP-31 approach: reading a datum requires consuming the UTxO and recreating it (expensive, causes contention). On the right, CIP-31: the transaction reads the UTxO without consuming it—the UTxO survives untouched, enabling concurrent reads by multiple transactions.

3.6 CIP-32: Inline Datums

3.6.1 The Problem

Before CIP-32, the eUTxO model supported attaching data to script-locked UTxOs through **datum hashes**. When creating a UTxO at a script address, the transaction included only the

hash of the datum in the UTxO itself. The actual datum content had to be provided separately: either embedded elsewhere in the transaction’s witness set, or stored entirely off-chain and supplied by whoever eventually spent the UTxO.

This design had two major consequences:

1. **Off-chain burden.** The party spending the UTxO had to know the original datum and include it in their spending transaction. This required off-chain coordination: the datum had to be stored somewhere (a database, IPFS, a centralized service) and communicated to the spender. If the datum was lost, the UTxO became unspendable.
2. **No public readability.** A third party observing the chain could see that a UTxO had a datum hash, but could not determine the datum’s contents without access to the original data. This made on-chain data effectively opaque to verifiers who had not participated in the original transaction.

For a credential system where *any party* must be able to read a credential’s current status, datum hashes are fundamentally inadequate. A verifier cannot check `status: active` if they can only see `datum_hash: 0x3f7a...`

3.6.2 How It Works

CIP-32 introduces **inline datums**: the full datum content is stored *directly in the UTxO* on the ledger. Any node, wallet, or explorer that can read the UTxO can also read the datum—no off-chain lookup required, no special access, no coordination with the datum creator.

Technically, a UTxO’s datum field now supports three states:

None No datum attached (standard ADA-only UTxO).

Datum Hash Only the hash is stored on-chain (pre-CIP-32 behavior).

Inline Datum The full datum is stored on-chain (CIP-32).

The trade-off is that inline datums increase the UTxO’s size on the ledger and therefore increase the minimum ADA required (`minUTxO`) to hold the UTxO. For a credential datum containing license type, jurisdiction, status, and a few additional fields, the size increase is modest (a few hundred bytes), and the `minUTxO` impact is approximately 0.2–0.5 ADA additional.

3.6.3 How the Credential System Uses CIP-32

CIP-32 is what makes CIP-68 *possible*. The CIP-68 Reference Token stores its metadata as an inline datum so that:

- Any verifier can read the credential status by querying the UTxO.
- No off-chain service is needed to resolve datum hashes.
- Smart contracts executing on-chain (e.g., the Signature Collection Validator) can read the Reference Token’s datum via reference inputs (CIP-31) and make validation decisions based on the credential’s current status.

Without inline datums, the dual-token pattern would require an off-chain datum registry—reintroducing exactly the centralized infrastructure that the blockchain architecture is designed to eliminate. CIP-32, together with CIP-31, forms the technical foundation upon which CIP-68’s mutable on-chain metadata is built.

3.7 CIP-33: Reference Scripts

3.7.1 The Problem

Plutus scripts (smart contracts) can be large. A compiled Plutus V2 script may be 5–15 KB of serialized CBOR. Before CIP-33, every transaction that invoked a Plutus script had to include the *entire compiled script* in its witness set. For a credential system where every signing operation invokes the Signature Collection Validator, this means attaching the same multi-kilobyte script to every transaction—wasting block space, increasing fees, and potentially pushing transactions over the 16,384-byte maximum transaction size.

3.7.2 How It Works

CIP-33 allows a Plutus script to be **deployed once to a UTxO** and then **referenced** by subsequent transactions. The deployment transaction creates a UTxO containing the script in a special `reference_script` field. Future transactions that need to execute the script include only a reference to this UTxO (its transaction hash and output index), not the script itself.

The ledger resolves the reference at validation time: when processing a transaction that references a script UTxO, the node fetches the script from the referenced UTxO and uses it for validation. The referenced UTxO is *not consumed*—like CIP-31 reference inputs, the script UTxO persists on-chain and can be referenced by unlimited future transactions.

3.7.3 How the Credential System Uses CIP-33

The credential system deploys the Signature Collection Validator and the Dues Enforcement Contract as reference scripts. This provides three benefits:

1. **Smaller transactions.** Signing transactions include only a 32-byte reference (transaction hash) plus an index, instead of a multi-kilobyte script. This can reduce transaction size by 80–90%.
2. **Lower fees.** Cardano transaction fees are proportional to transaction size. Smaller transactions mean lower fees for every signing operation.
3. **Auditability.** The script is deployed once at a known UTxO. Anyone can inspect the exact validator code that governs the credential system by reading this single UTxO.

3.8 CIP-1852 / BIP-44: HD Wallet Derivation

3.8.1 The Problem

A naive approach to key management would generate a single random private key and derive a single address from it. This creates several problems:

1. **No backup recoverability.** If the key file is lost, all funds and credentials are permanently inaccessible. There is no way to regenerate the key.
2. **Privacy.** Using a single address for all transactions creates a complete, publicly visible transaction graph. Every credential, every payment, every interaction is linked.
3. **Key management complexity.** If a user needs separate keys for different purposes (payment, staking, multiple credentials), they must manage multiple independent keys with independent backups.

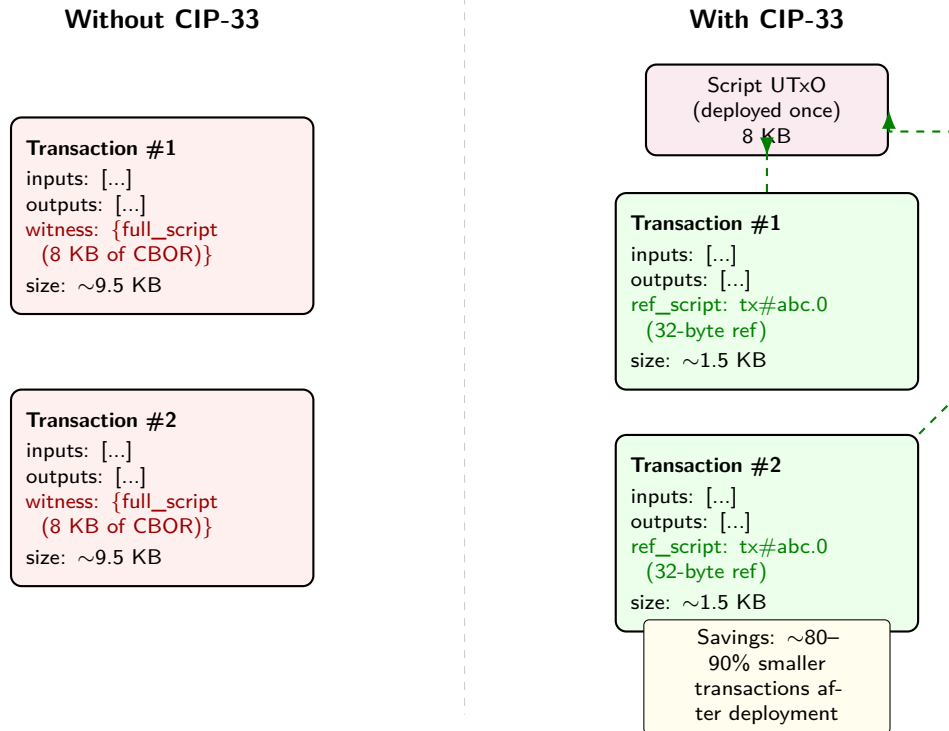


Figure 3.6: CIP-33 reference scripts. Without CIP-33 (left), every transaction must include the full compiled Plutus script in its witness set, consuming kilobytes of space per transaction. With CIP-33 (right), the script is deployed once to a UTxO, and subsequent transactions reference it with a compact 32-byte pointer. Transaction size drops by 80–90%.

4. **No standard.** Without a derivation convention, every wallet would generate keys differently, making wallet recovery across different software impossible.

3.8.2 How It Works

CIP-1852 defines Cardano’s **hierarchical deterministic (HD) wallet** derivation scheme, building on the BIP-32 (key derivation), BIP-39 (mnemonic encoding), and BIP-44 (multi-account structure) standards from the Bitcoin ecosystem.

The core idea is that a single **24-word mnemonic phrase** (BIP-39) encodes a master seed from which an unlimited number of keys can be derived deterministically. Given the same mnemonic, any compliant wallet software will derive exactly the same keys in exactly the same order. This means:

- **Backup is trivial.** Write down 24 words. That is the complete backup for all current and future keys.
- **Recovery works across wallets.** Import the mnemonic into Eternl, Nami, Lace, or any CIP-1852-compliant wallet, and all keys are regenerated identically.
- **Unlimited addresses.** Increment the index in the derivation path to generate fresh addresses for privacy.

The Derivation Path

CIP-1852 defines the following derivation path structure:

`m / purpose' / coin_type' / account' / role / index`

m The master key derived from the BIP-39 mnemonic.

purpose' = 1852' Identifies this as a Cardano Shelley-era derivation (named after Ada Lovelace's birth year). The apostrophe denotes *hardened derivation*.

coin_type' = 1815' Identifies the Cardano blockchain (named after Ada Lovelace's death year). Registered in the SLIP-44 coin type registry.

account' = 0' Account index. Most users use account 0; multiple accounts allow logical separation (e.g., personal vs. professional credentials).

role Distinguishes key purpose:

- 0 = External payment keys (receiving addresses)
- 1 = Internal payment keys (change addresses)
- 2 = Staking keys

index Sequential index within the role. Incrementing this generates additional keys.

The two most important paths for the credential system are:

`m/1852'/1815'/0'/0/0` First payment key (for receiving tokens)
`m/1852'/1815'/0'/2/0` First staking key (for delegation)

Hardened vs. Unhardened Derivation

The apostrophe (') in the derivation path denotes **hardened derivation**. This is a critical security distinction:

Unhardened (normal) derivation uses the parent's *public key* to derive child keys. This means anyone with the parent public key can compute all child public keys. Useful for generating receiving addresses without exposing the private key (e.g., a watch-only wallet or a payment server).

Hardened derivation uses the parent's *private key* to derive child keys. Knowing the parent public key reveals *nothing* about child keys. This provides a security firewall: if a child private key is compromised, the attacker cannot compute sibling or parent keys.

CIP-1852 requires hardened derivation for purpose, coin type, and account levels. This ensures that compromise of a single payment key does not allow an attacker to derive the staking key, other account keys, or the master key. The role and index levels use unhardened derivation, which enables watch-only wallets to generate addresses for receiving payments without access to private keys.

3.8.3 How the Credential System Uses CIP-1852

Every wallet in the credential system—authority, licensee, signer, and observer—is generated from a 24-word BIP-39 mnemonic using CIP-1852 derivation. The `generate_wallet()` function:

1. Generates or accepts a BIP-39 mnemonic (24 words, 256 bits of entropy).
2. Derives the master key using PBKDF2-HMAC-SHA512.
3. Derives the payment key at `m/1852'/1815'/0'/0/0`.
4. Derives the staking key at `m/1852'/1815'/0'/2/0`.

5. Constructs the base address by combining the payment key hash (PKH, 28 bytes) and staking key hash (SKH, 28 bytes) with the network identifier.
6. Encodes the address in bech32 format (`addr_test1...` for testnet, `addr1...` for mainnet).

The mnemonic is the **single point of backup**: losing it means losing all keys and all credentials. In the production deployment, authority wallet mnemonics are encrypted with AES-256-GCM and stored with strict access controls. Licensee and signer mnemonics are managed by their personal wallets (Eternal, Nami, etc.) via CIP-30—the credential system never sees or stores these mnemonics.

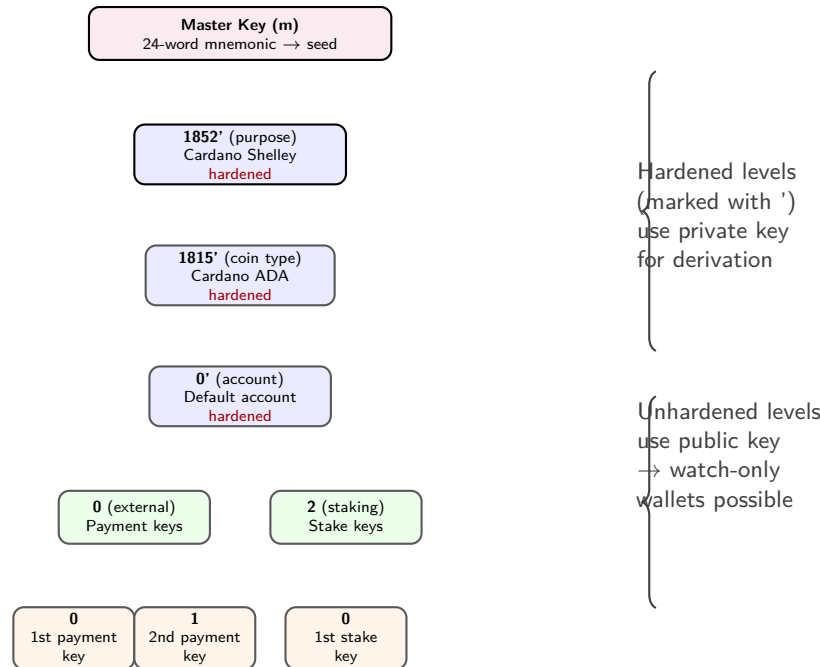


Figure 3.7: CIP-1852 / BIP-44 hierarchical deterministic wallet derivation tree. The master key is derived from a 24-word mnemonic. Purpose (1852'), coin type (1815'), and account (0') levels use hardened derivation (private key required), providing a security firewall. Role and index levels use unhardened derivation, enabling watch-only wallets. The credential system uses `m/1852'/1815'/0'/0/0` for payment and `m/1852'/1815'/0'/2/0` for staking.

3.9 How the CIPs Work Together

The CIPs described in this chapter are not isolated features—they form an interlocking system where each standard enables or enhances the others. Understanding their interactions is essential for understanding the credential architecture as a whole.

CIP-32 enables CIP-68. Without inline datums, the Reference Token’s metadata would be stored as a hash, requiring off-chain resolution. Inline datums make the metadata directly readable on-chain.

CIP-31 makes CIP-68 practical. Without reference inputs, every credential verification would consume (and have to recreate) the Reference Token UTxO, causing contention. Reference inputs allow unlimited concurrent reads.

CIP-33 makes CIP-68 affordable. Without reference scripts, every transaction interacting with the credential validator would include the full Plutus script, inflating transaction size and fees. Reference scripts reduce per-transaction overhead to a 32-byte pointer.

CIP-68 makes CIP-25 revocable. CIP-25 provides immutable attestation (“this credential was issued”). CIP-68 adds mutable status (“the credential is currently active/revoked”). Together, they provide both historical provenance and current validity.

CIP-30 connects users to the system. Without the dApp–wallet bridge, every credential operation would require server-side key management, creating custodial risk. CIP-30 keeps private keys under user control.

CIP-1852 makes wallets recoverable and interoperable. Without HD derivation, losing a key file means losing all credentials. With CIP-1852, 24 words are sufficient to recover everything.

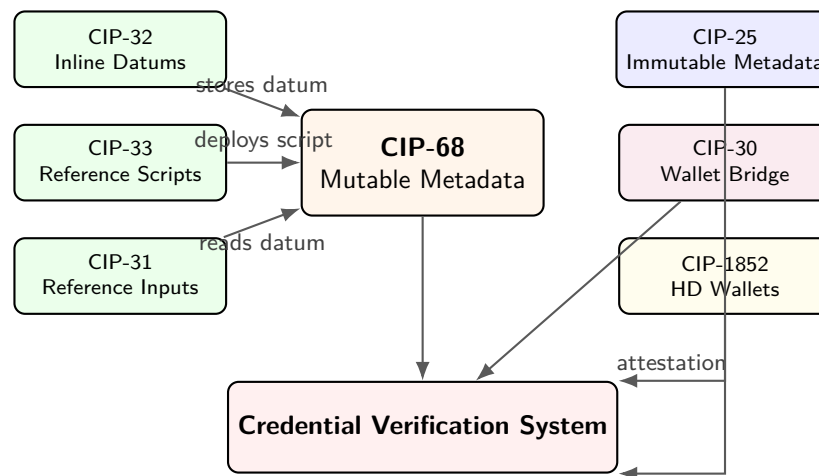


Figure 3.8: CIP dependency graph for the credential verification system. CIPs 31, 32, and 33 (the Plutus V2 trio) form the technical foundation that makes CIP-68’s mutable datum pattern practical and affordable. CIP-25 provides immutable attestation. CIP-30 and CIP-1852 handle wallet connectivity and key management. All six converge in the credential verification system.

3.10 Chapter Summary

This chapter examined the eight CIPs that form the technical foundation of the credential verification system:

1. **CIP-25** provides immutable NFT metadata, establishing permanent on-chain attestation of credential issuance.
2. **CIP-68** introduces the dual-token pattern with mutable inline datums, enabling authority-controlled revocation without requiring the holder’s private key—the key innovation for on-chain credential management.
3. **CIP-30** defines the non-custodial dApp–wallet bridge, keeping private keys under user control while enabling web-based credential operations.
4. **CIP-31** introduces reference inputs, allowing transactions to read UTxO data without consuming it—essential for concurrent credential verification.

5. **CIP-32** introduces inline datums, storing full data directly in UTxOs—the prerequisite for CIP-68’s on-chain metadata.
6. **CIP-33** introduces reference scripts, reducing transaction size and fees by deploying Plutus scripts once and referencing them.
7. **CIP-1852** defines HD wallet derivation, providing recoverable, interoperable key management from a single mnemonic.

The next chapter will examine how these CIPs are combined in the system architecture: the smart contract layer, the token taxonomy, and the transaction workflows that tie everything together.

Chapter 4

Smart Contracts on Cardano

Smart contracts are the enforcement layer of any blockchain-based system. They are the reason a credential issued on Cardano is not merely a database entry with a fancy name—they are what makes the credential *trustlessly verifiable*. In this chapter we build a complete mental model of how Cardano smart contracts work, from the simplest native scripts through full Plutus V2 validators, and then show exactly how the credential system uses each type. By the end you will understand why the system can guarantee that *only* a licensed authority can mint credential tokens, how multiple professionals can sign a work product concurrently without conflicting, and how license validity can be automatically linked to periodic dues payments.

4.1 What is a Smart Contract?

4.1.1 The Core Idea

A smart contract is a piece of code that lives on a blockchain and enforces rules automatically. Think of a vending machine: you insert the correct amount of money and press a button, and the machine dispenses the product. No cashier, no negotiation, no trust required. The rules are mechanical. A smart contract is the digital equivalent—it encodes rules in software that the blockchain itself enforces, removing the need for any human intermediary to verify compliance.

In everyday life, a traditional contract says “Party A agrees to pay Party B \$100 on January 1st in exchange for Service X.” Enforcement depends on good faith, or failing that, a court. A smart contract encodes the same logic in executable code: when the conditions are met, the action happens automatically. When the conditions are *not* met, the action is rejected. There is no appeals process, no bureaucratic delay, and no possibility of selective enforcement. The code runs the same way for everyone.

4.1.2 Cardano Validators Are Not Ethereum Smart Contracts

This distinction is critical and trips up nearly everyone coming from Ethereum. On Ethereum, a smart contract is a long-lived program with its own persistent storage. You *call functions* on it, it updates its internal state, and it can call other contracts. It is like a microservice that lives on the blockchain.

Cardano does not work this way.

On Cardano, a smart contract is a **validator**—a function that receives some data and returns **True** or **False**. That is it. The validator does not store state. It does not call other validators. It does not have methods you invoke. It simply answers one question: *“Is this transaction allowed to spend this UTxO?”*

The best analogy is a bouncer at a nightclub. The bouncer does not own the club. The bouncer does not manage the bar inventory. The bouncer has exactly one job: check IDs at the door and decide “yes, you may enter” or “no, you may not.” A Cardano validator is the bouncer for a UTxO. When someone builds a transaction that tries to spend a UTxO locked at a script address, the validator examines the transaction and says “yes” or “no.”

This design has profound consequences. Because validators are stateless and pure—they examine inputs and produce a boolean—they are *deterministic*. The same inputs always produce the same output. You can run the validator locally before submitting the transaction and know with certainty whether it will succeed. There are no surprises from concurrent state changes, no front-running attacks that alter the execution path, and no gas estimation failures from state that changed between simulation and execution.

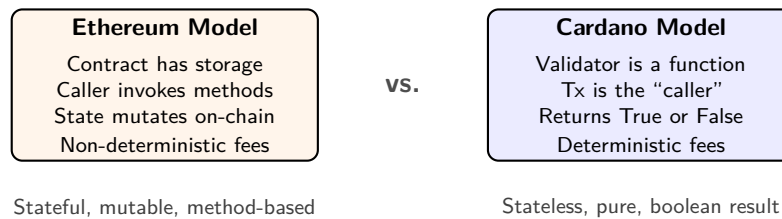


Figure 4.1: Ethereum’s stateful contract model versus Cardano’s stateless validator model. Cardano validators are pure functions that approve or reject UTxO spending.

4.1.3 Why This Matters for Credentials

In the credential system, this design means that the minting policy for license tokens is a validator that answers: “Does this transaction include a signature from the authorized licensing board?” If yes, minting proceeds. If no, the transaction is rejected by every node on the network. There is no admin backdoor, no override function, and no way to “call” the policy and ask it to mint tokens on your behalf. The only path to minting is constructing a valid transaction that satisfies the validator’s conditions.

4.2 Native Scripts (Phase 1)

4.2.1 Overview

Native scripts are Cardano’s simplest form of smart contract. They were introduced in the Shelley era and require no Plutus execution engine—the ledger evaluates them directly during Phase 1 validation. This means they incur **zero execution fees**. The only cost is the standard transaction fee (~0.17 ADA), making them ideal for operations that need to happen frequently and cheaply.

Native scripts are built from five primitive building blocks. Each block performs a simple check, and they can be combined to express sophisticated conditions. Think of them as logic gates:

each gate checks one thing, and you wire them together to build the overall access policy.

4.2.2 The Five Script Types

ScriptPubkey: Single Key Requirement

The simplest native script. It requires that the transaction includes a signature from a specific verification key hash (public key hash). If Alice's key hash is embedded in the script, then only a transaction signed by Alice satisfies the script. This is the script type used for the credential system's minting policies.

In the credential system, every authority wallet has a unique Ed25519 public key hash (PKH). The minting policy is a **ScriptPubkey** containing that PKH. Any transaction that mints or burns tokens under this policy must be signed by the authority's key. No one else on the planet can produce a valid signature for that key, so no one else can create or destroy credential tokens.

ScriptAll: AND Logic

Requires that *all* listed conditions are satisfied. If the script contains three sub-scripts, all three must pass. This is logical AND.

A practical example: a corporate account that requires signatures from both the CEO and CFO. Neither can act alone. In the credential system, **ScriptAll** is used when time-lock constraints are combined with a key requirement—the authority's key *and* the time constraint must both be satisfied.

ScriptAny: OR Logic

Requires that *at least one* listed condition is satisfied. If the script contains three sub-scripts, any single one passing is sufficient. This is logical OR.

A practical example: a recovery mechanism where either the primary key or a backup key can authorize a transaction. If you lose your primary key, the backup still works.

ScriptNofK: Threshold (N-of-K)

Requires that at least N out of K listed conditions are satisfied. This generalizes both **ScriptAll** ($N = K$) and **ScriptAny** ($N = 1$). A 3-of-5 multi-signature governance scheme uses **ScriptNofK** with $N = 3$ and $K = 5$.

This is particularly relevant for the credential system's future governance upgrade path. Instead of a single authority key controlling minting, a **ScriptNofK** policy could require signatures from 3 of 5 board members, distributing trust and eliminating single-key centralization risk.

TimeLock: Temporal Constraints

Two time-lock primitives restrict *when* a script is valid:

- **InvalidBefore** (also called **TimelockStart**): the transaction is invalid before a specified slot number. The script cannot be satisfied until that slot has passed.
- **InvalidHereAfter** (also called **TimelockExpiry**): the transaction is invalid at or after a specified slot number. The script can only be satisfied before that slot.

Time locks are used in the credential system to create minting policies that expire. For example, an authority might create a policy that can only mint tokens during a specific fiscal year. After the expiry slot, no new tokens can be minted under that policy, even by the authority. This provides a cryptographic guarantee that the token supply is frozen after a certain point.

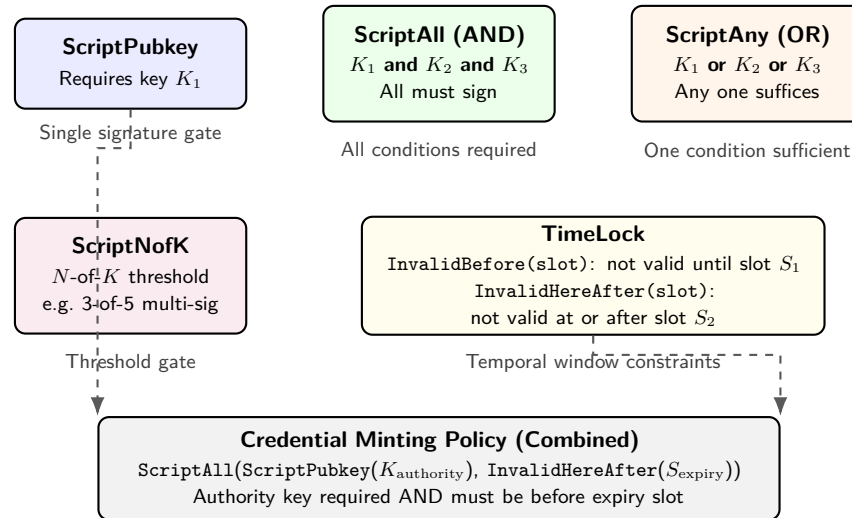


Figure 4.2: The five native script types and how they compose. The credential system’s minting policy typically combines a **ScriptPubkey** (authority key) with an optional **InvalidHereAfter** (time lock) inside a **ScriptAll** wrapper.

4.2.3 Why Native Scripts for Minting Policies?

The credential system deliberately uses native scripts—not Plutus validators—for its minting policies. The reason is economics. A native script minting transaction costs approximately 0.17 ADA with zero execution fees. A Plutus V2 minting transaction costs 0.3–0.5 ADA because the Plutus execution engine must evaluate the script, consuming CPU steps and memory units that are priced into the fee.

For a system that might mint thousands of credential tokens (one license NFT per professional, plus signature tokens and validity tokens), the difference compounds. At 10,000 licenses with associated tokens, native scripts save approximately 1,300–3,300 ADA compared to Plutus-based minting policies. The trade-off is expressiveness: native scripts can only check keys and time constraints, while Plutus scripts can examine the full transaction. For minting policies where the rule is simply “the authority must sign,” native scripts provide the same security guarantee at a fraction of the cost.

4.3 Plutus V2 Validators (Phase 2)

4.3.1 Beyond Simple Scripts

Native scripts answer simple questions: “Is this key present?” “Is it before slot X?” But many operations require examining the *content* of a transaction—which tokens are being moved, what data is attached, how much ADA is being paid to whom. For these operations, Cardano provides Plutus validators: full-powered smart contracts written in Haskell (compiled to Plutus

Core, which runs on the Untyped Plutus Lambda Calculus—UPLC) and evaluated during Phase 2 validation.

Plutus V2, introduced in the Vasil hard fork (September 2022), added three features critical for the credential system:

- **Reference Inputs (CIP-31)**: read a UTxO without consuming it. This lets verifiers inspect credential metadata without spending the token.
- **Inline Datums (CIP-32)**: attach data directly to a UTxO output instead of storing only a datum hash. This eliminates the need to look up datums separately.
- **Reference Scripts (CIP-33)**: reference a script stored in a UTxO rather than including the script in every transaction. This reduces transaction sizes significantly.

4.3.2 The Three Inputs to a Validator

Every Plutus V2 validator receives exactly three pieces of information when evaluating whether a UTxO can be spent. Understanding these three inputs is the key to understanding all Cardano smart contracts.

Datum: The State

The **datum** is data attached to the UTxO being spent. Think of it as a note taped to a locked box. When someone created the UTxO and sent tokens to the script address, they included this datum to record the state or context of that particular UTxO.

In the credential system, the **SignerDatum** is attached to each signer’s deposit UTxO. It records the signer’s public key hash, the SHA-256 hash of the document being signed, the slot at which the deposit was made, the policy IDs of the signature and validity tokens, and the validity token’s expiry slot. This datum is the “evidence packet” that proves who signed, what they signed, and when.

Redeemer: The Action

The **redeemer** is data provided by the transaction that wants to spend the UTxO. Think of it as the instruction or request. The datum says “here is what was stored.” The redeemer says “here is what I want to do with it.”

In the credential system, the redeemer is an integer discriminator that tells the validator which action is being performed: **COLLECT** (redeemer 0), **FINALIZE** (redeemer 1), or **RECLAIM** (redeemer 2). Each action triggers different validation logic inside the validator.

Script Context: The Transaction

The **script context** is the full transaction being validated—all inputs, all outputs, all signatures, all minted tokens, all time constraints, everything. The validator can inspect any part of the transaction to make its decision.

This is what gives Plutus validators their power compared to native scripts. A native script can only check “is this key present?” A Plutus validator can check “does this transaction include an output paying exactly 50 ADA to the authority’s address?” or “does this transaction mint exactly one token under policy XYZ?”

The validator examines all three inputs—datum, redeemer, and script context—and returns **True** (the UTxO may be spent, the transaction is valid) or **False** (the spending is rejected, the transaction fails).

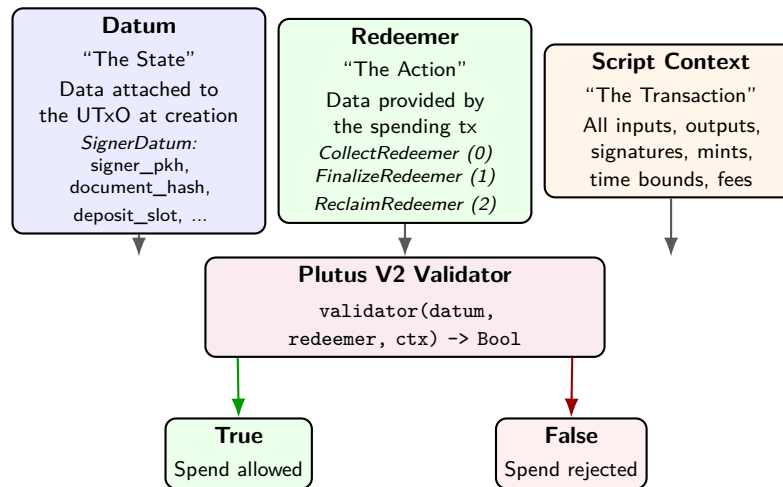


Figure 4.3: The three inputs to a Plutus V2 validator. The validator function receives the datum (state stored with the UTxO), the redeemer (action requested by the spending transaction), and the script context (the full transaction), and returns a boolean.

4.3.3 Deterministic Execution

One of Cardano’s most important properties is that Plutus execution is **deterministic**. Because the validator is a pure function of its three inputs, and because the eUTxO model means the inputs are known before the transaction is submitted, you can run the validator locally and know *exactly* what will happen on-chain. The fee is calculable in advance. The result is predictable. There are no surprises.

This stands in stark contrast to Ethereum, where a transaction can fail at execution time because another transaction altered the contract’s state between when you simulated it and when it was included in a block. On Cardano, if the validator succeeds locally, it will succeed on-chain (assuming the UTxOs have not been spent by another transaction in the interim—but that is a concurrency issue the eUTxO model makes explicit, not a hidden state-mutation surprise).

4.4 Execution Units (exUnits)

4.4.1 CPU Steps and Memory Units

Every Plutus script execution consumes two budgeted resources: **CPU steps** and **memory units**. CPU steps measure computational complexity—how many operations the validator performs. Memory units measure how much memory the validator allocates during execution. Both are metered precisely by the UPLC evaluator.

Each transaction has a maximum execution budget, currently set at 10 billion CPU steps and 10 million memory units per transaction (with per-script limits as well). If a validator exceeds

either limit, the transaction fails. The transaction builder calculates the exact execution cost before submission, so you know whether you will exceed the budget before spending any ADA.

4.4.2 How exUnits Affect Fees

Transaction fees on Cardano follow a linear formula:

$$\text{fee} = a \times \text{size_bytes} + b + c_{\text{cpu}} \times \text{cpu_steps} + c_{\text{mem}} \times \text{mem_units}$$

where a and b are the base fee parameters (currently $a = 44$ lovelace per byte, $b = 155,381$ lovelace), and c_{cpu} and c_{mem} are the execution fee coefficients.

For native scripts, the execution terms (c_{cpu} and c_{mem}) are zero—native scripts are evaluated by the ledger rules themselves, not by the Plutus execution engine. This is why native script transactions cost ~ 0.17 ADA: only the size-based fee applies.

For Plutus V2 transactions, the execution terms add $0.1\text{--}0.3$ ADA depending on script complexity, bringing the total to $\sim 0.3\text{--}0.5$ ADA. This is still dramatically cheaper than Ethereum (where gas costs can reach $\$1\text{--}\$15+$ for contract interactions), but it is a measurable difference when multiplied across thousands of transactions.

4.4.3 Deterministic Cost Estimation

Because Plutus execution is deterministic, the cost is known *before* the transaction is submitted to the network. The transaction builder runs the validator against the exact inputs, records the CPU and memory consumption, embeds the exUnits in the transaction, and calculates the precise fee. If the fee changes between estimation and submission, it is because a UTxO was spent (a concurrency event), not because the validator behaved differently.

Phase 1 Only	Phase 1 + Phase 2
Native Script Tx Size fee: ~ 0.17 ADA Execution fee: 0 ADA Total: ~ 0.17 ADA Used for: Minting policies	Plutus V2 Tx Size fee: ~ 0.17 ADA Execution fee: $\sim 0.15\text{--}0.35$ ADA Total: $\sim 0.3\text{--}0.5$ ADA Used for: Validators, dues
10,000 mints: Native = $\sim 1,700$ ADA ADA	Plutus = $\sim 3,000\text{--}5,000$ Savings: 1,300–3,300 ADA

Figure 4.4: Fee comparison between native script and Plutus V2 transactions. Native scripts avoid Phase 2 execution costs entirely, making them the economical choice for simple minting policies.

4.5 Minting Policies

4.5.1 What is a Minting Policy?

A minting policy is a special type of validator that controls the creation (minting) and destruction (burning) of native tokens. On Cardano, every native token belongs to a **policy ID**, which is the Blake2b-224 hash of the minting policy script. The policy ID is the token’s identity—it tells you *which rules govern this token’s existence*.

This is fundamentally different from Ethereum, where token contracts are long-lived programs that maintain a balance mapping. On Cardano, tokens are ledger-level assets. The minting policy only runs when tokens are created or destroyed. Transfers between wallets require no script execution at all—they are plain UTxO transactions, just like sending ADA.

4.5.2 Policy ID = Script Hash

The policy ID is computed deterministically:

$$\text{PolicyID} = \text{Blake2b}_{224}(\text{CBOR}(\text{script}))$$

Because the hash is deterministic, anyone can independently verify that a given policy ID corresponds to a specific script. If the script says “only Alice’s key can mint,” and you verify the script hash matches the policy ID, then you know with mathematical certainty that only Alice can have minted those tokens. No trust required. No authority to query. Just hash verification.

4.5.3 The Credential System’s Minting Policy

In the credential system, the minting policy is a `ScriptPubkey` native script containing the authority’s public key hash. The `PlutusV2MintingPolicy` class (despite its name, it uses native scripts for economic efficiency) constructs this policy:

1. The authority’s 28-byte Ed25519 verification key hash is embedded in a `ScriptPubkey` native script.
2. Optionally, time-lock constraints are added via `ScriptAll` (combining the key requirement with `InvalidBefore/InvalidHereAfter`).
3. The policy ID is the Blake2b-224 hash of the CBOR-serialized script.
4. The policy is saved as both a JSON metadata file and a CBOR binary for transaction building.

This means **only the authority can mint or burn credential tokens**. A transaction attempting to mint tokens under this policy must include a valid Ed25519 signature from the authority’s private key. Without that key, no entity—not even Cardano node operators or stake pool operators—can create tokens under that policy.

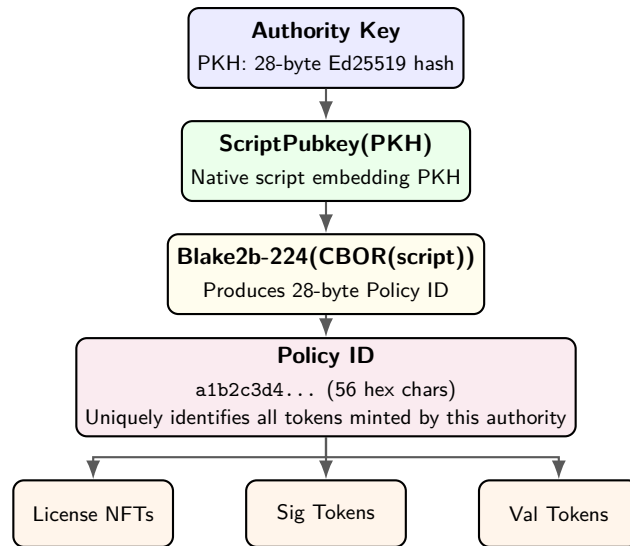


Figure 4.5: Minting policy derivation: the authority’s key hash is embedded in a native script, which is hashed to produce the deterministic Policy ID that governs all credential tokens.

4.5.4 Verification Without Trust

A verifier who wants to confirm that a credential token is legitimate performs these steps:

1. Look up the token’s policy ID on-chain.
2. Check that the policy ID is in the registry of known authorities (e.g., “Policy ID abc123... belongs to the California Board of Professional Engineers”).
3. The mathematical relationship between the policy ID and the authority’s key guarantees that only the authority could have minted that token.

No API call to the licensing board. No database query. No waiting for business hours. The verification is a deterministic mathematical operation that takes milliseconds.

4.6 The SignatureCollectionValidator

4.6.1 The Problem: Multi-Party Signing on eUTxO

The Signature Collection Validator is the most complex and most important smart contract in the credential system. It solves a problem that is conceptually simple—collect signatures from multiple professionals on a single document—but technically challenging on Cardano’s eUTxO model.

The challenge is **concurrency**. On Ethereum, multiple users can call the same contract simultaneously because the contract maintains internal state (a mapping of who has signed). The EVM serializes these calls and updates the state sequentially. On Cardano’s eUTxO model, there is no shared mutable state. A UTxO can only be spent once. If two signers try to spend the same UTxO at the same time, one transaction succeeds and the other fails.

Consider a naive design: a single UTxO at the script address holds the “work product state” (which signers have signed so far). Signer A builds a transaction that spends this UTxO and creates a new one with their signature added. Signer B, working simultaneously, builds a

transaction that also spends the *same original UTxO*. Both transactions cannot succeed—the first one to be included in a block wins, and the second fails because the UTxO it references no longer exists. This is the eUTxO concurrency bottleneck.

4.6.2 The Solution: Independent Per-Signer UTxOs

The credential system's `SignatureCollectionValidator` solves this elegantly: instead of maintaining a single shared-state UTxO, **each signer creates their own independent UTxO** at the script address. There is no shared state to contend over. Signer A deposits their tokens into one UTxO. Signer B deposits their tokens into a different UTxO. Both UTxOs sit at the same script address but are completely independent. They can be created in the same block, in different blocks, or even simultaneously—there is zero contention.

Think of it like a suggestion box with individual envelopes. Instead of everyone writing on the same sheet of paper (which creates a queue), each person drops their own sealed envelope into the box. The envelopes do not interfere with each other. When the authority is ready to finalize, they open all the envelopes at once.

4.6.3 The Three Redeemer Actions

The validator supports three actions, identified by their `CONSTR_ID` discriminator:

COLLECT (Redeemer 0)

The `CollectRedeemer` (constructor ID 0) is used when a signer deposits their tokens. Any required signer can submit a transaction that:

1. Creates a new UTxO at the validator's script address.
2. Attaches a `SignerDatum` containing their PKH, the document hash, the deposit slot, the policy IDs of their signature and validity tokens, and the validity expiry slot.
3. Includes at least one signature token and one validity token in the UTxO's value.
4. Is signed by the signer's private key (proving ownership of the PKH in the datum).

The validator checks that:

- The signer's PKH is in the list of required signers.
- The signer has not already deposited (no duplicate deposits).
- The validity token has not expired (expiry slot is in the future relative to the current slot).
- If a validity deadline is set, the expiry slot does not exceed it.
- The required tokens (signature token + validity token) are present in the UTxO value.

This is the critical concurrency-safe operation. Because each `COLLECT` creates a *new, independent* UTxO rather than modifying an existing one, multiple signers can deposit simultaneously without any risk of transaction collision. Signer 1 in New York and Signer 2 in Los Angeles can submit their deposit transactions at the exact same second, and both will succeed because they are creating different UTxOs, not competing to spend the same one.

FINALIZE (Redeemer 1)

The `FinalizeRedeemer` (constructor ID 1) is used by the authority to complete the signing process. When all required signers have deposited their tokens, the authority builds a single

transaction that:

1. Consumes *all* per-signer UTxOs at the script address as inputs (one input per signer).
2. Creates a single output containing all collected tokens.
3. Must be signed by the authority’s key.
4. Includes the `FinalizeRedeemer` for each script input.

This is an **atomic sweep**: all per-signer deposits are consumed in a single transaction. Either the finalization succeeds entirely (all UTxOs are spent and the work product is complete) or it fails entirely (no UTxOs are spent). There is no partial finalization.

The validator checks that:

- The transaction is signed by the authority’s PKH.
- All required signers have deposits present among the inputs.
- The document hash in each `SignerDatum` matches the work product’s document hash.

RECLAIM (Redeemer 2)

The `ReclaimRedeemer` (constructor ID 2) is the escape hatch. If a work product is cancelled before finalization, signers need a way to recover their deposited tokens. A signer can submit a reclaim transaction that:

1. Consumes their specific UTxO at the script address.
2. Returns their signature token and validity token to their wallet.
3. Must be signed by the same key that made the original deposit (the signer’s PKH in the `SignerDatum`).

The validator checks that the transaction signer matches the `signer_pkh` in the datum. Only the original depositor can reclaim their tokens—no one else, not even the authority, can spend a signer’s deposit UTxO via the `RECLAIM` action.

4.6.4 The SignerDatum

Each deposit UTxO carries a `SignerDatum`, which is a Plutus data type (`PlutusData` subclass with `CONSTR_ID = 0`) containing six fields:

Field	Type	Description
<code>signer_pkh</code>	bytes	28-byte public key hash of the signer
<code>document_hash</code>	bytes	SHA-256 hash of the document being signed
<code>deposit_slot</code>	int	Cardano slot number at deposit time
<code>sig_token_policy</code>	bytes	Policy ID of the signature token
<code>val_token_policy</code>	bytes	Policy ID of the validity token
<code>validity_expiry_slot</code>	int	Slot at which the validity token expires

Table 4.1: `SignerDatum` fields. Each deposit UTxO at the validator address carries this datum, creating a complete evidence record for each signer.

This datum serves as the on-chain evidence record. After finalization, anyone can read the datum from the consumed UTxOs (via transaction history) and independently verify: *who* signed (`signer_pkh`), *what* they signed (`document_hash`), *when* they signed (`deposit_slot`),

under what authority (token policy IDs), and *whether their credential was valid* at signing time ($\text{validity_expiry_slot} > \text{deposit_slot}$).

4.6.5 Concurrency in Detail: The COLLECT $\rightarrow$ FINALIZE Flow

Let us trace through a concrete example with three signers—Alice (a structural engineer), Bob (a geotechnical engineer), and Carol (the project manager)—signing off on a seismic retrofit design.

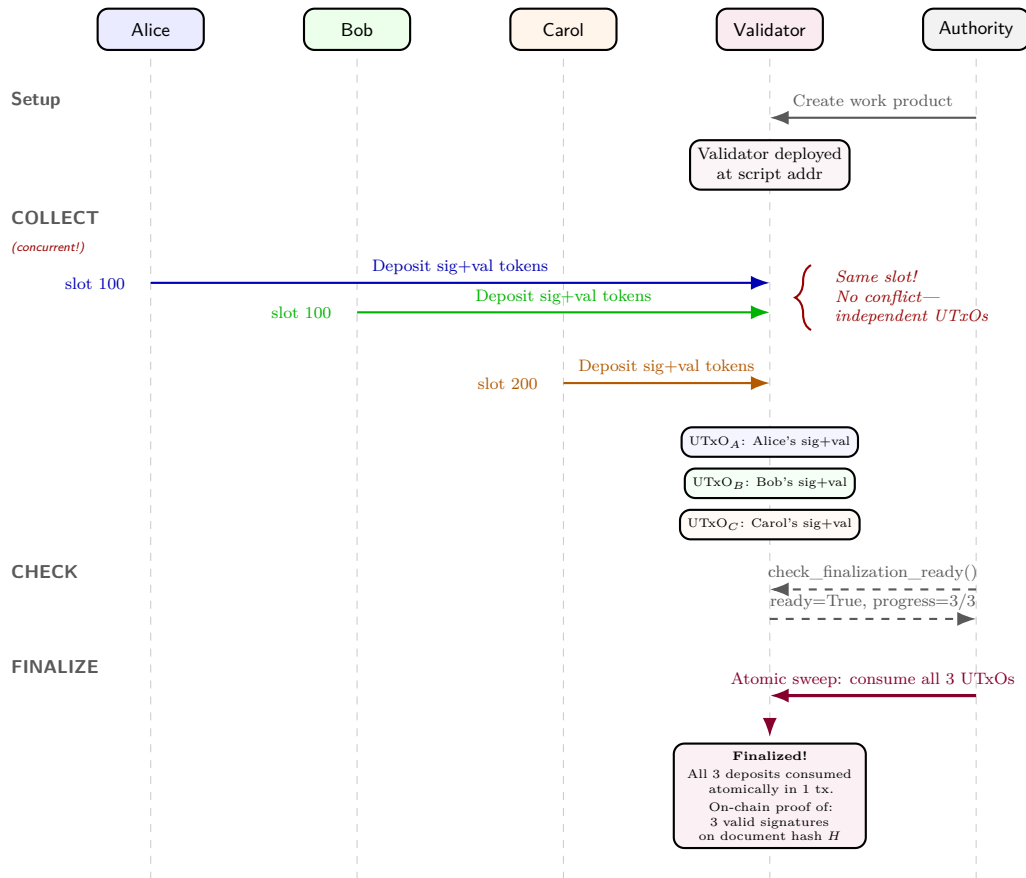


Figure 4.6: Sequence diagram showing the COLLECT $\rightarrow$ FINALIZE flow with three signers. Alice and Bob deposit in the same slot without conflict because each creates an independent UTxO. Carol deposits later. The authority then atomically sweeps all three UTxOs in a single finalization transaction.

Step 1: Setup. The authority creates a `SignatureCollectionValidator` parameterized with the three signers' PKHs, the document hash, and optionally a validity deadline slot. The validator is deployed (its script hash determines a unique script address on-chain).

Step 2: Alice deposits (slot 100). Alice builds a transaction that creates a new UTxO at the validator address containing: 1 signature token, 1 validity token, a `SignerDatum` with her PKH and the document hash, and the minimum ADA (~ 2 ADA). The transaction includes the `CollectRedeemer`. She signs it with her private key. The validator approves because her PKH is in `required_signers`, her validity token has not expired, and she has not already deposited.

A new $UTxO_A$ now sits at the script address.

Step 3: Bob deposits (slot 100—same slot!). Bob does exactly the same thing at the same time. His transaction creates a *different* $UTxO_B$ at the same script address. Because $UTxO_A$ and $UTxO_B$ are separate $UTxOs$, there is no contention. Both transactions can be included in the same block. This is the concurrency solution in action—if this were a shared-state design, one of them would fail.

Step 4: Carol deposits (slot 200). Carol deposits a day later. She creates $UTxO_C$. The validator now holds three independent $UTxOs$, one per signer.

Step 5: Authority checks readiness. The authority calls `check_finalization_ready()`, which compares `_collected_signers` against `required_signers` and returns `ready=True, progress="3/3"`.

Step 6: Authority finalizes. The authority builds a single transaction that consumes $UTxO_A$, $UTxO_B$, and $UTxO_C$ as inputs (each with a `FinalizeRedeemer`), and creates an output containing all collected tokens. The authority signs the transaction with their key. The validator runs once per input, checking that the authority's signature is present and that all required signers have deposits. The transaction is submitted atomically—all three $UTxOs$ are consumed in one block, creating an immutable on-chain record that all three professionals signed the seismic retrofit design with valid credentials.

4.6.6 What if Something Goes Wrong?

If the work product is cancelled before all signers have deposited, each signer who has already deposited can reclaim their tokens using the `RECLAIM` redeemer. The validator verifies that the reclaim transaction is signed by the same key recorded in the `SignerDatum`, ensuring that only the original depositor can recover their tokens. The authority cannot steal deposited tokens, and neither can other signers.

If a signer's validity token expires between deposit and finalization, the deposit remains valid—the validator checked validity at deposit time and recorded the expiry slot in the datum. However, the authority may choose to reject the finalization and require the signer to renew their validity token and re-deposit. This policy decision is made off-chain; the on-chain validator records the facts and lets the authority act accordingly.

4.7 The DuesEnforcementContract

4.7.1 Linking License Validity to Payments

Many professional licenses require periodic dues or renewal fees. A medical license might require annual renewal with continuing education verification. A PE license might require periodic dues to the state board. The `DuesEnforcementContract` automates this link between payment and validity, ensuring that a professional's license validity token can only be renewed when dues are paid.

Think of it like a subscription service with a grace period. As long as you pay, your subscription (validity token) remains active. If you miss a payment, there is a grace period during which you can still renew by paying. If the grace period expires, the authority can revoke your validity entirely, and you must go through a reinstatement process.

4.7.2 Contract Parameters

The contract is parameterized at deployment time with four values:

Parameter	Type	Description
<code>annual_dues_lovelace</code>	<code>int</code>	Annual dues in lovelace (1 ADA = 1,000,000 lovelace). Must be between 1 ADA and 10,000 ADA.
<code>grace_period_slots</code>	<code>int</code>	Number of slots after expiry during which renewal is still allowed. Default: 86,400 (~1 day).
<code>license_ref</code>	<code>int</code>	Database reference to the license this contract governs.
<code>authority_pkh</code>	<code>bytes</code>	28-byte PKH of the authority who can revoke.

Table 4.2: DuesEnforcementContract parameters.

4.7.3 Two Redeemer Actions

PayDues (Redeemer 0)

The `PayDuesRedeemer` (constructor ID 0) is used when a licensee pays their annual dues. The validator checks:

- The payment amount is at least `annual_dues_lovelace`. Overpayment is allowed; underpayment is rejected.
- The payment is directed to the authority’s address.
- If the current validity has expired, the current slot must be within the grace period ($\text{current slot} \leq \text{expiry slot} + \text{grace period}$).

Upon successful validation, the off-chain system mints a new validity token with an updated expiry slot ($\text{current slot} + \text{one year's worth of slots}$) and delivers it to the licensee’s wallet.

RevokeValidity (Redeemer 1)

The `RevokeValidityRedeemer` (constructor ID 1) is used by the authority to revoke a license’s validity when dues remain unpaid past the grace period. The validator checks that the transaction is signed by the authority’s key. Upon successful revocation, the licensee can no longer sign documents—the `check_validity_for_signing()` function will return `False` for any expired validity token, and even the grace period does not extend signing rights (it only extends the *renewal* window).

4.7.4 Grace Period Mechanics

The grace period is a carefully designed buffer between “your validity expired” and “your license is revoked.” During the grace period:

- **Signing is blocked.** An expired validity token prevents document signing, even during the grace period. This protects the public—an engineer with expired credentials should not be stamping designs.

- **Renewal is allowed.** The licensee can pay dues and receive a new validity token. This prevents harsh penalties for minor payment delays.
- **Revocation is deferred.** The authority cannot revoke until the grace period has passed, giving the licensee fair notice and opportunity to cure.

After the grace period expires without payment, the authority can submit a revocation transaction. At that point, the licensee must go through a full reinstatement process (which may involve re-application, additional fees, and board review).

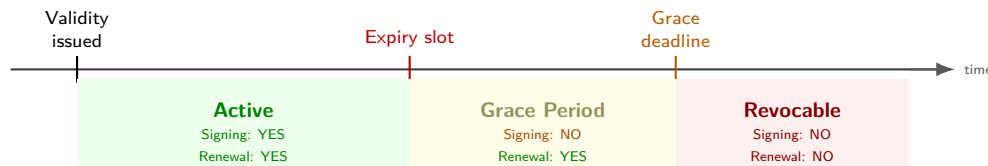


Figure 4.7: Dues enforcement timeline. The active period allows both signing and renewal. The grace period blocks signing but allows dues payment. After the grace deadline, the authority can revoke.

4.7.5 The DuesContractDatum

The on-chain datum for the dues contract is a `PlutusData` subclass with `CONSTR_ID = 0`, containing four fields: `authority_pkh` (bytes), `annual_dues` (int), `license_ref` (int), and `grace_period_slots` (int). This datum is attached to the contract's UTxO and can be read by any verifier to understand the dues terms for a given license.

The contract's native script is a `ScriptPubkey` requiring the authority's key, ensuring that only the authority can deploy and manage dues enforcement terms. The contract address is derived from the script hash, just as with the signature collection validator.

4.8 Aiken: The Future

4.8.1 The Problem with PlutusTx

PlutusTx, the current standard for writing Plutus validators, compiles Haskell code to Plutus Core (UPLC). While Haskell is a powerful language well-suited to formal verification, it has practical drawbacks for smart contract development:

- **Large compiled output.** PlutusTx scripts compile to relatively large UPLC bytecode. Larger scripts mean larger transactions, higher fees, and a tighter budget against the 16 KB maximum transaction size.
- **High exUnits.** The compiled UPLC tends to be verbose, consuming more CPU steps and memory units than hand-optimized code.
- **Steep learning curve.** Haskell is notoriously difficult for developers accustomed to Python, JavaScript, or Rust. This limits the pool of potential contributors and auditors.
- **Complex toolchain.** Setting up a PlutusTx development environment requires GHC, Cabal, and numerous Haskell dependencies that can be challenging to configure.

4.8.2 Enter Aiken

Aiken is a modern smart contract language designed specifically for Cardano. It compiles directly to UPLC without going through Haskell, producing smaller, more efficient bytecode. Key advantages:

- **Smaller compiled scripts.** Aiken-compiled validators are typically 40–60% smaller than equivalent PlutusTx output. Smaller scripts mean lower fees and more headroom within the 16 KB transaction limit.
- **Lower exUnits.** The tighter UPLC output consumes fewer CPU steps and memory units, directly reducing execution fees.
- **Rust-like syntax.** Aiken’s syntax is inspired by Rust and Gleam, making it accessible to a much broader developer community than Haskell.
- **Fast compilation.** Aiken compiles in seconds, compared to minutes for a full PlutusTx build. This dramatically improves the development feedback loop.
- **Built-in testing.** Aiken includes a native test framework with property-based testing support, making it easier to write comprehensive validator tests.

4.8.3 Migration Roadmap (v4.0)

The credential system’s v4.0 roadmap includes migrating the `SignatureCollectionValidator` and `DuesEnforcementContract` from their current PyCardano-based Plutus V2 implementation to Aiken-compiled validators. The expected benefits are:

1. **Fee reduction:** 20–40% lower execution fees due to smaller, more efficient UPLC bytecode.
2. **Audit accessibility:** Aiken’s readable syntax makes third-party security audits more practical and less expensive.
3. **Plutus V3 features:** Aiken supports Plutus V3 natively, enabling the use of new built-in functions and cryptographic primitives (including BLS12-381 for future zero-knowledge proof integration).
4. **Community contribution:** A lower barrier to entry means more developers can review, test, and contribute to the validator logic.

The migration does not change the system’s architecture or token model. The same `SignerDatum` structure, the same three-redeemer validator pattern, and the same native-script minting policies will be preserved. Only the validator implementation language changes—from Haskell/PlutusTx to Aiken—producing the same logical behavior with better performance characteristics.

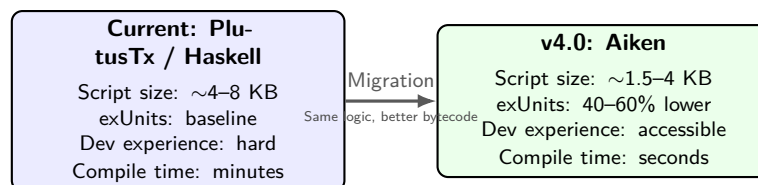


Figure 4.8: Planned migration from PlutusTx/Haskell to Aiken. The validator logic and system architecture remain identical; only the implementation language changes, yielding smaller scripts and lower fees.

Chapter Summary. Cardano smart contracts are validators—pure functions that return `True` or `False` when a transaction attempts to spend a UTxO. Native scripts provide simple, zero-fee key and time checks ideal for minting policies. Plutus V2 validators provide full transaction inspection for complex logic like signature collection and dues enforcement. The credential system uses native scripts for minting (keeping costs at ~ 0.17 ADA) and Plutus V2 validators for multi-party signing and payment enforcement (~ 0.3 – 0.5 ADA). The `SignatureCollectionValidator` solves eUTxO concurrency through independent per-signer UTxOs, and the `DuesEnforcementContract` links license validity to periodic payments with a configurable grace period. Future migration to Aiken will reduce script sizes and fees while preserving identical logic.

Chapter 5

The Credential Verification System

“The preceding chapters established what a blockchain is, how Cardano works, what eUTxO means, and how native tokens and smart contracts operate. This chapter puts all of those pieces together into a single, working application: a credential verification system built on Cardano.”

This chapter describes the actual system—every layer, every token, every role, every lifecycle step, and every cost. By the end, you should be able to trace a professional license from its creation on-chain through signing, renewal, revocation, and third-party verification, and explain exactly why each architectural decision was made.

5.1 System Overview

The credential verification system is organized into three distinct layers, each with a clearly defined responsibility. No single layer can operate in isolation; together they form a complete pipeline from user interaction to immutable on-chain settlement. Understanding this layered separation is essential because it explains why certain operations are fast (local database queries), why others are slow but final (blockchain transactions), and why the middle layer exists at all (to bridge the gap between the two).

5.1.1 Layer 1: Cardano Blockchain

The bottom layer is the Cardano blockchain itself. It provides three critical services that no other layer can replicate: **token custody** (who holds which tokens is determined by the UTxO set on-chain, not by any off-chain record), **policy enforcement** (minting policies are evaluated by every node in the network—no single party can bypass them), and **transaction finality** (once a transaction is confirmed in a block and buried under subsequent blocks, it is irreversible for all practical purposes). The blockchain does not store personal information, does not render dashboards, and does not manage workflow state. It is a settlement layer: slow, expensive relative to a database query, but absolutely authoritative.

Every credential in the system ultimately exists as a Cardano native token. A license is an NFT under a specific minting policy. Signature capacity is a fungible token under the same policy. Validity is a time-bounded token. The blockchain knows nothing about “licenses” or

“signatures” in a semantic sense—it knows only that certain tokens exist at certain addresses under certain policies, and that those policies were satisfied at minting time. All semantic meaning is encoded in metadata and interpreted by the layers above.

5.1.2 Layer 2: Off-Chain Application Layer

The middle layer is a Python application (built on PyCardano v0.19.1) that handles everything the blockchain cannot do efficiently: **wallet management** (generating HD wallets from mnemonics, deriving keys, encrypting private keys at rest), **metadata validation** (ensuring CIP-68 metadata conforms to the expected schema before a transaction is built), and **workflow orchestration** (coordinating multi-step processes like “mint a license, then mint signature tokens, then mint a validity token, then transfer all three to the licensee”). This layer speaks to the blockchain exclusively through the Blockfrost API, an indexed read/write interface that eliminates the need to run a full Cardano node.

The application layer is where business logic lives. It knows what a “license” means, what fields a “credential” should have, which wallets are “authorities” versus “licensees,” and what sequence of transactions constitutes a “signing ceremony.” It translates human intent into Cardano transactions and translates on-chain state back into human-readable status.

5.1.3 Layer 3: Local SQLite Database

The top layer is a local SQLite database with nine tables that cache and extend the on-chain state. It exists for three reasons: **state tracking** (the blockchain is append-only and has no concept of “current status”—the database maintains derived state like “this license is active” by interpreting the chain of transactions), **caching** (querying the blockchain for every dashboard refresh would be slow and rate-limited; the database provides sub-5ms responses for common queries), and **dashboard queries** (the web dashboard and CLI tools query the database, not the blockchain, for all read operations). The database is never the source of truth for token ownership—that is always the blockchain. If the database and the chain disagree, the chain wins, and periodic reconciliation corrects the database.

5.1.4 Communication: The Blockfrost API

All communication between the application layer and the Cardano blockchain flows through Blockfrost, a hosted API service that indexes the entire Cardano ledger and exposes it through RESTful endpoints. Blockfrost provides UTxO queries (“what tokens does this address hold?”), transaction submission (“submit this signed transaction to the network”), and chain tip queries (“what is the current slot number?”). The system does not run a full Cardano node. This is a deliberate trade-off: reduced infrastructure cost and complexity in exchange for a dependency on an external service. The risk is mitigated by the existence of multiple Blockfrost alternatives (Koios, Ogmios, self-hosted Blockfrost) and the fact that Blockfrost is a read/write *interface* to the chain, not the chain itself—switching providers does not affect any on-chain state.

5.2 Token Taxonomy

The system defines four distinct token types, all minted under authority-controlled native script policies. Each token serves a specific purpose in the credential lifecycle, and understanding what each token represents—and who holds it—is fundamental to understanding how the

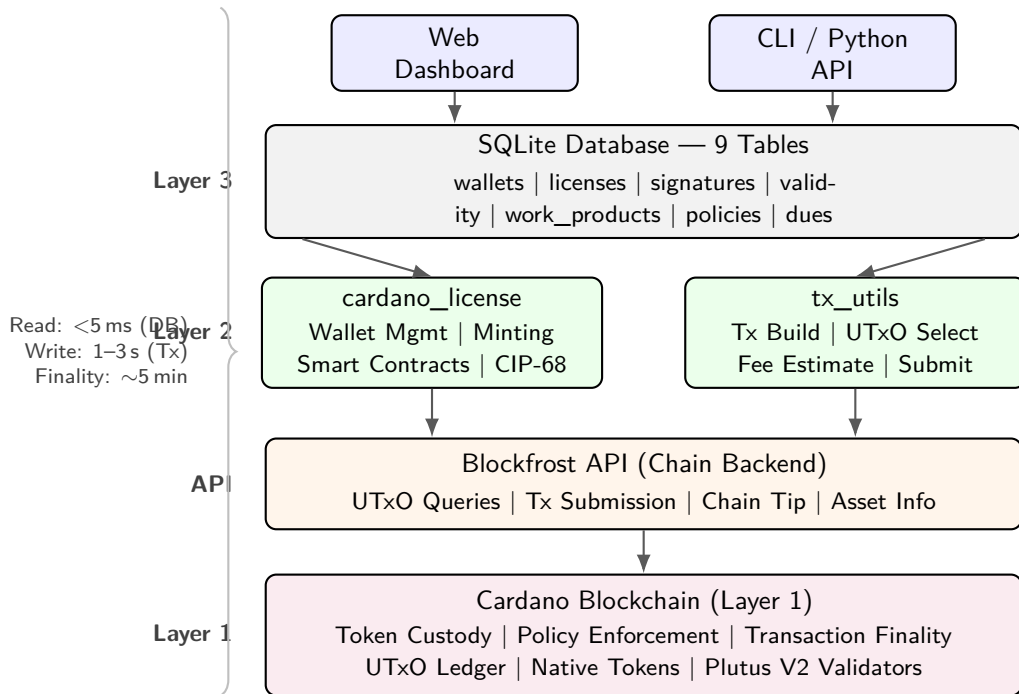


Figure 5.1: Three-layer architecture of the credential verification system. Layer 1 (Cardano blockchain) provides settlement finality. Layer 2 (off-chain application) handles business logic and transaction construction. Layer 3 (SQLite) provides fast reads and state caching. All chain communication flows through the Blockfrost API.

system works. These are not abstract concepts; each token is a real Cardano native asset that occupies a UTxO on-chain, carries a minimum ADA deposit, and can be independently verified by anyone with access to the blockchain.

5.2.1 Token 1: License NFT (CIP-68)

The License NFT is the core credential token. It is **non-fungible**—each one is unique, representing a specific license held by a specific individual. Unlike simple CIP-25 NFTs (which store metadata in the minting transaction’s auxiliary data and are immutable thereafter), the License NFT uses the **CIP-68 datum metadata standard**, which splits the token into a dual-token pair:

- **User Token (label 222):** Held in the licensee’s personal wallet. This token is *immutable*—its asset name is fixed at minting time and cannot be changed. It serves as a “badge” that the licensee can present to prove they were issued a credential. However, the User Token alone does not prove the credential is *currently valid*—it only proves it was once issued.
- **Reference Token (label 100):** Held at the authority’s UTxO (or at a known script address). This token carries an **inline datum** containing the credential’s mutable metadata: license type, jurisdiction, issue date, licensee identity, and—critically—a **status** field. Because the authority controls the UTxO holding the Reference Token, the authority can update the datum at any time (by spending the UTxO and creating a new one with updated datum) without requiring the licensee’s cooperation.

The CIP-68 pattern is the architectural foundation of the entire revocation mechanism (detailed in Section 5.5). The key insight is that verification always checks the Reference Token’s datum, not the User Token. The User Token in the licensee’s wallet is a pointer; the Reference Token at the authority’s address is the source of truth.

5.2.2 Token 2: Signature Token (Fungible)

Signature tokens represent **signing capacity**. They are fungible—one signature token is identical to another under the same policy. When the authority issues a license, it also mints a quantity *N* of signature tokens and transfers them to the licensee’s wallet. Each time the licensee signs a document (or, more precisely, participates in a signing ceremony), one signature token is consumed: it is **deposited into a validator script address** as part of a transaction. The token is not burned; it is locked at the validator, creating a permanent on-chain record that the signing event occurred.

The quantity *N* determines how many documents the licensee can sign before needing additional tokens. This creates a natural metering mechanism: a professional engineering board might issue 100 signature tokens per year, corresponding to the expected number of documents a PE would stamp. Running out of signature tokens requires requesting more from the authority, which can impose whatever verification it deems appropriate before minting additional tokens.

5.2.3 Token 3: Validity Token (Time-Bounded)

The validity token represents **current good standing**. It is a native token with a **slot-based expiry** encoded in its datum or asset name. The validator script that accepts signature deposits also requires a validity token to accompany each deposit. The validator checks that the validity token’s expiry slot is in the future at the time of the transaction. If the validity token has

expired, the transaction fails—the licensee cannot sign documents until they obtain a new validity token.

Validity tokens are linked to the dues enforcement contract. When a licensee pays their annual dues (or satisfies whatever renewal criteria the authority requires), the authority mints a new validity token with an expiry slot set to the appropriate future date. This creates a direct, enforceable link between license renewal and signing capacity: if you do not renew, your validity token expires, and you cannot deposit signatures into any validator.

5.2.4 Token 4: Work Product (Logical Composite)

A work product is not a single token but a **logical composite** tracked in the local SQLite database and coordinated by the `SignatureCollectionValidator` script. A work product represents a document or project that requires signatures from multiple parties—for example, a set of engineering drawings that need stamps from both the designing engineer and the reviewing engineer.

The validator script address accumulates deposits: each required signer creates a separate UTxO at the script address containing their signature token, their validity token, and a datum identifying the work product (by document hash). When all required signers have deposited, the authority submits a **Finalize** transaction that consumes all deposit UTxOs atomically. The finalized work product is recorded on-chain as a set of consumed UTxOs under the validator script, with each UTxO containing the signer’s public key hash, the document hash, and the deposit slot.

5.3 Wallet Roles

The system defines four wallet roles, each with distinct permissions and key storage requirements. This role separation is not merely organizational—it is enforced on-chain by minting policies (which check for the authority’s key) and by validator scripts (which check for specific signer public key hashes). A wallet’s role determines what transactions it can construct and sign.

5.3.1 Authority

The authority wallet belongs to the licensing board, professional association, or credentialing body. It is the most privileged role in the system. The authority can **mint and burn all token types** (license NFTs, signature tokens, validity tokens), **create minting policies** (the native scripts that restrict minting to the authority’s key), and **deploy validator scripts** (the Plutus V2 scripts that manage signature collection and dues enforcement). The authority also controls the Reference Token (label 100) of every CIP-68 license NFT it has issued, giving it unilateral revocation power.

Because the authority key can mint tokens that represent professional credentials, its compromise would be catastrophic: an attacker could issue fraudulent licenses. For this reason, authority keys are stored **server-side with AES-256-GCM encryption**, protected by restrictive file permissions, and ideally backed by hardware security modules in production. The migration path to multi-signature governance (3-of-5 `ScriptNofK` or KERI AID delegation) distributes this trust across multiple key holders, but the single-key model is the current implementation.

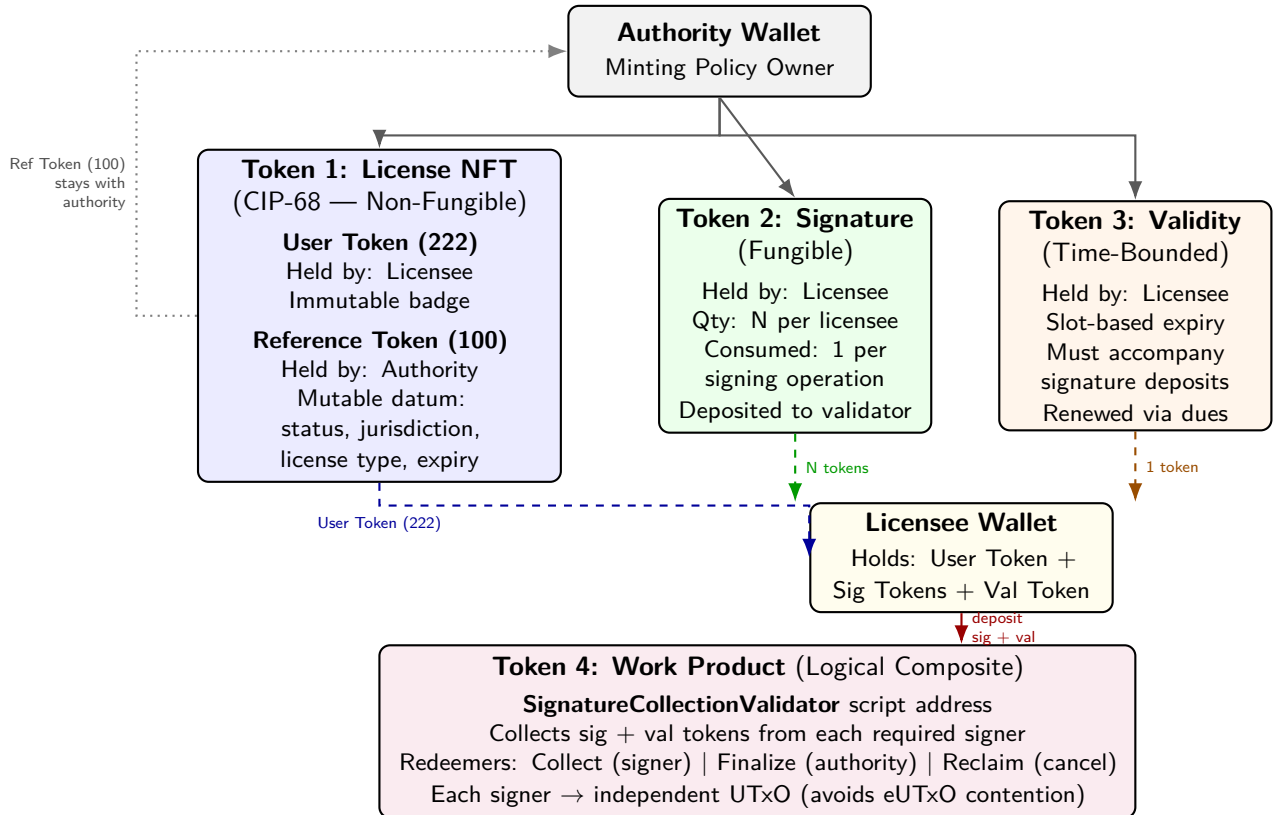


Figure 5.2: Token taxonomy showing all four token types, their holders, and their flow through the system. The authority mints all tokens. The licensee receives the User Token (222), signature tokens, and validity tokens. Signing deposits sig+val tokens into the SignatureCollectionValidator. The Reference Token (100) remains with the authority for status updates.

5.3.2 Licensee

The licensee is the professional—the engineer, physician, attorney, or other credentialed individual. The licensee **holds the license NFT User Token (222)** in their personal wallet, **receives and deposits signature and validity tokens** during signing ceremonies, and **pays dues** to the authority’s dues enforcement contract.

Critically, the licensee wallet uses a **CIP-30 non-custodial model**. The system never has access to the licensee’s private keys. Instead, the licensee connects a standard Cardano browser wallet (Eternl, Nami, or Lace) to the system’s web interface. When a transaction requires the licensee’s signature—for example, depositing tokens into a validator—the application constructs the transaction and sends it to the CIP-30 wallet for signing. The licensee reviews and approves the transaction in their wallet extension. This is the same pattern used by all Cardano DeFi applications and means that the system cannot spend the licensee’s tokens without their explicit consent.

5.3.3 Signer

The signer role is functionally similar to the licensee role but is used specifically in the context of multi-party signing ceremonies. A signer **deposits signature and validity tokens** into the `SignatureCollectionValidator` to attest to a work product. Like the licensee, the signer uses a **CIP-30 non-custodial wallet** and retains full control of their private keys.

In many cases, the licensee and signer are the same entity—a PE signing their own drawings is both licensee (they hold the license) and signer (they deposit tokens into the validator). The role distinction matters primarily in multi-party scenarios where different individuals contribute signatures to the same work product, each with their own license NFT and token balances.

5.3.4 Observer

The observer is any third party that needs to verify a credential but does not need to create, hold, or transfer tokens. Observers have **read-only access**: they can query the blockchain (via Blockfrost or any other chain indexer) to check whether a license NFT exists, whether its Reference Token datum shows **status: active**, and whether the policy ID matches a registered authority. Observers need only a **public key**—no private key, no wallet software, no account.

The observer role is what makes the system publicly verifiable. A hospital verifying a physician’s license, a building department checking an engineer’s credentials, or a client confirming an attorney’s bar membership all act as observers. The verification is permissionless: no registration, no API key, no bilateral agreement with the issuing authority is required.

5.4 Credential Lifecycle

A credential moves through eight distinct steps from creation to verification (and potentially revocation). Each step involves specific participants, specific transactions, and specific on-chain effects. This section traces the full journey in detail.

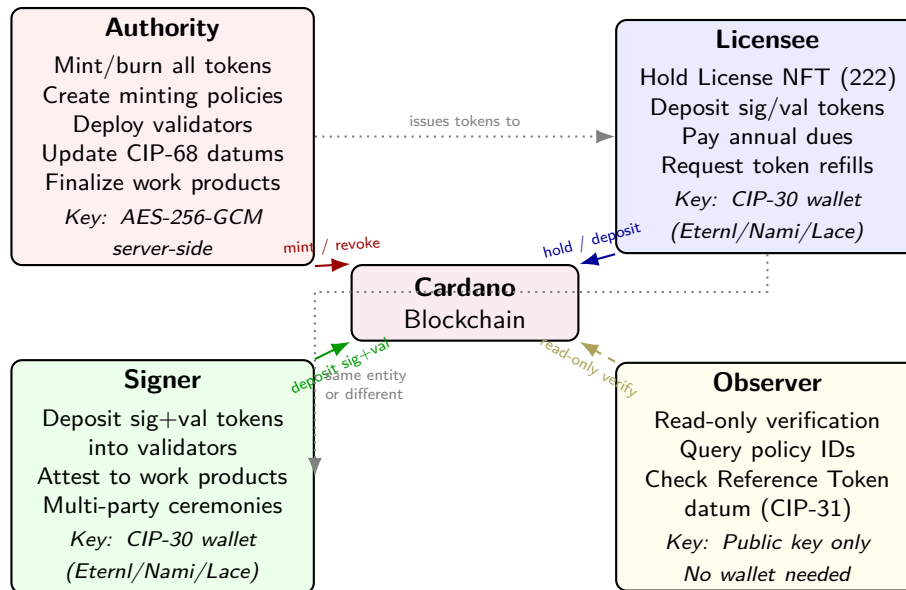


Figure 5.3: Four wallet roles and their interactions with the Cardano blockchain. The authority has full mint/burn privileges with server-side key storage. Licensees and signers use non-custodial CIP-30 wallets. Observers perform read-only verification with no wallet required.

5.4.1 Step (a): Policy Creation

Before any tokens can be minted, the authority creates a **minting policy**. In this system, the minting policy is a **native script**—specifically, a `ScriptPubkey` that requires the authority’s payment verification key hash. The policy script is serialized, and its Blake2b-224 hash becomes the **Policy ID**, a 28-byte identifier that uniquely and deterministically identifies all tokens minted under this policy.

The Policy ID is critical for verification. When a third party sees a license NFT in someone’s wallet, the first thing they check is the Policy ID. If the Policy ID matches one registered to a known licensing board, the token is legitimate (it could only have been minted by someone who controls the authority’s private key). If the Policy ID is unknown, the token is either fraudulent or from an unrecognized authority. This is analogous to checking the seal on a physical certificate—except that forging a Policy ID requires forging an Ed25519 private key, which is computationally infeasible.

The policy creation step is a one-time operation per authority. The resulting Policy ID is published, registered, and used for all subsequent minting operations. No on-chain transaction is required to create the policy itself—the policy exists as a serialized script that is included in minting transactions.

5.4.2 Step (b): License NFT Minting (CIP-68)

The authority constructs a minting transaction that creates two tokens simultaneously under the same Policy ID:

1. The **Reference Token (label 100)** is sent to the authority’s own UTxO. Its inline datum contains the credential metadata:

- `license_type`: “PE”, “MD”, “JD”, etc.
 - `jurisdiction`: “US-CA”, “US-NY”, etc.
 - `issue_date`: ISO 8601 date string
 - `licensee_identity`: public key hash of the licensee
 - `status`: “active” (initial value)
2. The **User Token (label 222)** is sent to the licensee’s wallet address. This token has the same Policy ID and base asset name as the Reference Token, differing only in the CIP-68 label prefix.

The transaction includes the native script as a witness and is signed by the authority’s payment signing key. Upon confirmation, the licensee’s wallet shows the User Token, and the authority’s address holds the Reference Token with its inline datum. The total cost is approximately 2.2 ADA (0.2 ADA transaction fee + 2.0 ADA minimum UTxO deposit for the Reference Token).

5.4.3 Step (c): Signature and Validity Token Minting

In a separate transaction (or batched into the same transaction for efficiency), the authority mints **signature tokens** and a **validity token** for the licensee. Signature tokens are minted in a batch—typically 100 at a time—under the same Policy ID. The validity token is minted with an asset name or datum encoding its expiry slot. Both are sent to the licensee’s wallet address. The cost is approximately 2.2 ADA per batch.

5.4.4 Step (d): Token Holding

The licensee now holds three types of tokens in their CIP-30 wallet: the User Token (222), a balance of signature tokens, and a validity token. These tokens are visible in any Cardano wallet that supports native assets. The licensee can see their token balances, verify the Policy ID, and confirm that the tokens are genuine. No further action is required until the licensee needs to sign a document or renew their validity.

5.4.5 Step (e): Signing (Token Deposit)

When the licensee (acting as a signer) needs to sign a document, they deposit one signature token and one validity token into the `SignatureCollectionValidator` script address. The transaction includes a `SignerDatum` containing:

- The signer’s public key hash (PKH)
- The SHA-256 hash of the document being signed
- The Policy ID of the signer’s license
- The validity token’s expiry slot
- The current slot (deposit timestamp)

The validator script checks that: (1) the signature token has the correct Policy ID, (2) the validity token has the correct Policy ID and its expiry slot is in the future, and (3) the transaction is signed by the signer whose PKH is in the datum. If all checks pass, the transaction is accepted, and the tokens are locked at the script address. Each signer creates an **independent UTxO** at the script address, avoiding the eUTxO contention problem that would arise if all signers tried to spend the same UTxO.

5.4.6 Step (f): Verification

Any observer can verify the signing event by querying the validator script address and checking the UTxOs. The verification process (detailed in Section 5.6) confirms that tokens with the correct Policy ID are present at the validator address, that the Policy ID maps to a registered authority, that the Reference Token datum shows `status: active`, and that the validity token had not expired at the time of deposit. This verification can be performed by anyone, at any time, without permission from any party.

5.4.7 Step (g): Renewal

When the validity token approaches expiry, the licensee pays dues to the authority's **dues enforcement contract**. This Plutus V2 contract verifies that the correct ADA amount was paid and that the payment is within the grace period. Upon successful payment, the authority mints a new validity token with an updated expiry slot and sends it to the licensee's wallet. The old validity token is either burned or simply becomes useless (its expiry slot is in the past, so no validator will accept it).

The renewal process creates a direct economic link between professional standing and signing capacity. If a licensee fails to pay dues, their validity token expires, and they cannot sign documents—even though they still hold the User Token (222) and may have remaining signature tokens. This mirrors real-world licensing: a PE whose license lapses due to unpaid dues cannot legally stamp drawings, even though they once held a valid license.

5.4.8 Step (h): Revocation

If the authority needs to revoke a credential (due to disciplinary action, fraud, or other cause), it updates the Reference Token's inline datum to `status: revoked`. This is a simple transaction: the authority spends the UTxO holding the Reference Token and creates a new UTxO at the same address with the same token but an updated datum. The transaction costs approximately 0.3 ADA and requires only the authority's signature.

The revocation is effective immediately upon confirmation (within approximately 20 seconds for the first block, with practical finality in approximately 5 minutes). Any subsequent verification will read the Reference Token's datum and find `status: revoked`. The User Token (222) remains in the licensee's wallet, but it is now meaningless—a verified credential requires both the User Token *and* an active Reference Token datum. This mechanism is detailed in Section 5.5.

5.5 Revocation Mechanism

This is the most important architectural decision in the entire system. Revocation—the ability for an authority to invalidate a previously issued credential—is the feature that separates a useful credential system from a toy. If a physician loses their license, if an engineer commits fraud, if an attorney is disbarred, the credential must become invalid *immediately and globally*. This section explains why revocation is hard on Cardano, why the naive approach fails, and how the CIP-68 pattern solves the problem.

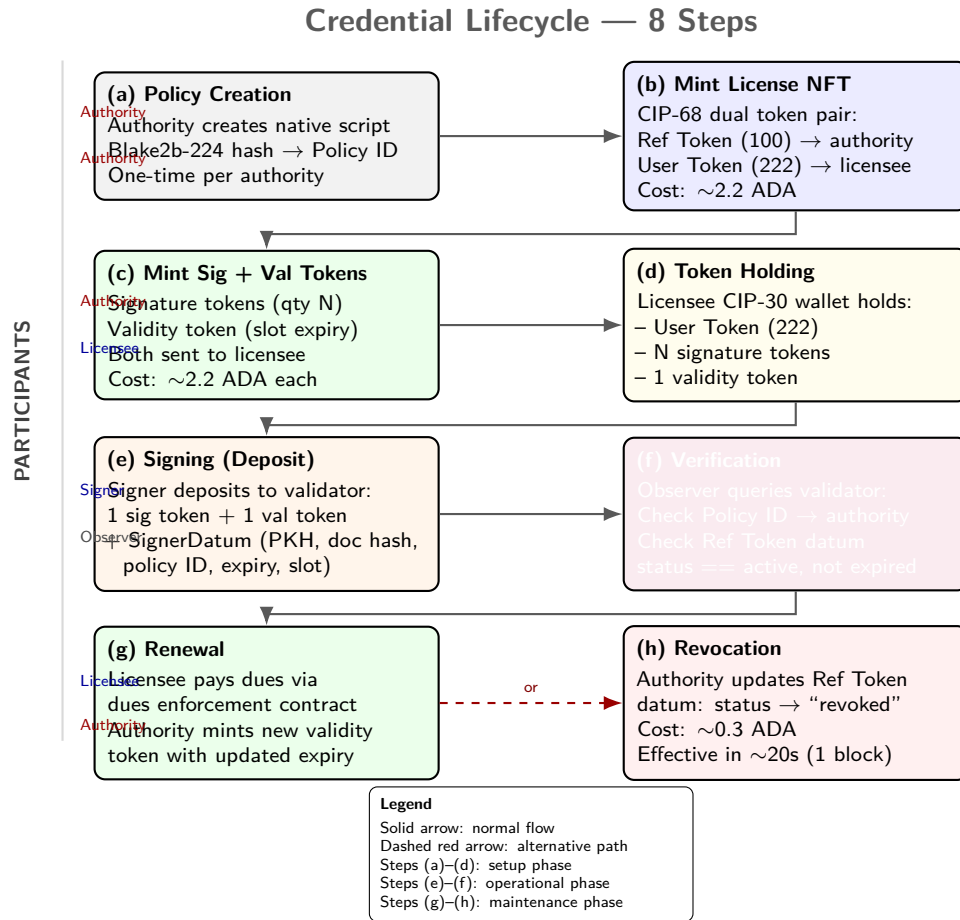


Figure 5.4: Complete credential lifecycle showing all eight steps from policy creation through revocation. The setup phase (a–d) is performed once per credential. The operational phase (e–f) repeats for each signing event. The maintenance phase (g–h) occurs periodically or as needed.

5.5.1 The Problem: eUTxO Ownership

In Cardano’s extended UTxO model, every token sits inside a UTxO, and every UTxO is controlled by either an address (requiring the owner’s private key to spend) or a script (requiring satisfaction of the script’s conditions to spend). This is a feature, not a bug—it is what makes eUTxO secure. But it creates a specific problem for revocation:

You cannot spend (or burn) someone else’s UTxO without their private key.

This is an absolute, non-negotiable property of the ledger. The authority cannot construct a transaction that spends the licensee’s UTxO because the ledger requires the licensee’s signature for that UTxO. The authority does not have the licensee’s private key (and should never have it, by the non-custodial design). This means the authority *cannot forcibly remove a token from the licensee’s wallet*.

5.5.2 The Broken Approach: Direct Token Burn

The naive approach to revocation is: “when the authority wants to revoke a license, burn the licensee’s NFT.” This approach fails completely in eUTxO. To burn a token, you must spend the UTxO that contains it. To spend that UTxO, you need the private key that controls the address where it sits. The licensee’s token sits at the licensee’s address. The authority does not have the licensee’s key. Therefore, the authority cannot burn the licensee’s token. Period.

One might suggest requiring the licensee to send the token to the authority upon revocation, but this requires the licensee’s cooperation—which is precisely what you cannot count on when revoking someone’s license for cause. A physician whose license is being revoked for malpractice is unlikely to voluntarily return their credential token.

Another approach might be to hold the token at a script address controlled by the authority rather than at the licensee’s personal address. But this defeats the purpose of the credential: the licensee should be able to prove they hold the token, which requires the token to be in their wallet. If the token is at a script address, the licensee does not “hold” it in any meaningful sense.

5.5.3 The CIP-68 Solution: Datum-Based Revocation

The CIP-68 standard resolves this tension elegantly by splitting the credential into two tokens with different custody:

1. The **User Token (222)** goes to the licensee’s wallet. The authority *cannot* touch it. The licensee holds it forever (or until they voluntarily send it somewhere). This is fine because **the User Token is not the source of truth**.
2. The **Reference Token (100)** stays at the authority’s UTxO. The authority *can* spend this UTxO at any time because the authority controls the private key for that address. The Reference Token carries an **inline datum** with a **status** field.

Revocation is performed by the authority spending the UTxO that holds the Reference Token and creating a new UTxO at the same address with the same Reference Token but an updated datum: **status: "revoked"**. This transaction:

- Requires **only the authority’s signature**—no interaction with the licensee.

- Costs approximately **0.3 ADA** (a simple transaction fee; no Plutus execution needed for a native script policy).
- Takes effect within approximately **20 seconds** (one block confirmation).
- Is **globally visible**—any observer who reads the Reference Token’s inline datum will see `status: revoked`.

The User Token (222) in the licensee’s wallet is now meaningless. It still exists—the licensee still “holds” it—but any verifier performing a proper verification will check the Reference Token’s datum and find `status: revoked`. The User Token without an active Reference Token is like a physical certificate after the issuing board has published a revocation notice: the paper exists, but the credential it represents has been invalidated.

5.5.4 The minUTxO Economics

Every UTxO on Cardano must carry a minimum ADA deposit (approximately 2.0 ADA for a UTxO carrying a native token with an inline datum). This ADA is not “spent”—it is locked and can be recovered when the UTxO is consumed. When the authority mints the Reference Token, approximately 2.0 ADA is locked at the UTxO. When the authority performs a revocation (spending and re-creating the UTxO with updated datum), the 2.0 ADA remains locked at the new UTxO. If the authority later decides to **burn the Reference Token entirely** (removing it from the ledger), the locked ADA is returned to the authority’s wallet, minus the transaction fee of approximately 0.3 ADA. The net recovery is approximately 1.7 ADA per burned Reference Token.

For a licensing board managing 10,000 licenses, the total locked ADA at Reference Token UTxOs is approximately 20,000 ADA (approximately \$6,000 at \$0.30/ADA). This is the economic cost of maintaining revocable credentials—a modest amount compared to the annual operating budgets of professional licensing boards.

5.6 Verification Flow

Verification is the operation that makes the entire system worthwhile. Everything else—minting, signing, renewal, revocation—exists to produce a state that can be verified. This section describes the exact sequence of steps a third-party observer follows to verify a credential, and explains why each step is necessary.

5.6.1 Step-by-Step Verification

The verification flow consists of seven steps, each building on the previous one. The entire process completes in under 5 seconds, dominated by Blockfrost API latency (200–500ms per query).

Step 1: Identify the License NFT. The verifier obtains the licensee’s wallet address (or the licensee presents their wallet via CIP-30). The verifier queries the address’s UTxO set and identifies the License NFT User Token (label 222). This step confirms that the licensee holds a token that *claims* to be a license. It does not yet prove anything about the token’s legitimacy.

Step 2: Extract the Policy ID. Every Cardano native token has a Policy ID—the Blake2b-224 hash of the minting policy script. The verifier extracts the Policy ID from the User Token. This is a deterministic operation; the Policy ID is part of the token’s on-chain identity.

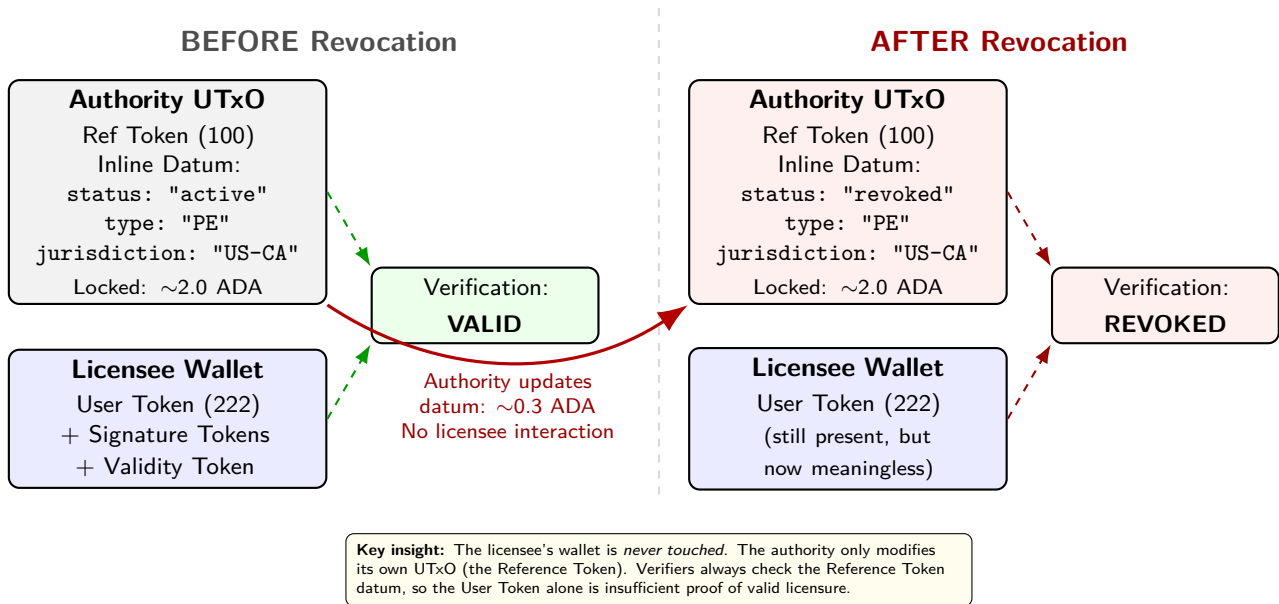


Figure 5.5: Before and after revocation. The authority updates only its own Reference Token datum from “active” to “revoked.” The licensee’s User Token remains in their wallet but is now meaningless for verification purposes. No interaction with the licensee is required.

Step 3: Verify the Policy ID against a registry. The verifier checks whether the extracted Policy ID matches any registered authority. This is the step that distinguishes a legitimate credential from a fraudulent one. The registry could be a published list maintained by a standards body, a smart contract on-chain, or a well-known database. If the Policy ID is not recognized, verification fails—the token was minted by an unknown party and cannot be trusted.

Step 4: Find the CIP-68 Reference Token. The verifier constructs the Reference Token’s asset name by replacing the User Token’s label prefix (222) with the Reference Token’s label prefix (100), keeping the same base asset name. The verifier then searches for this Reference Token on-chain. Under CIP-68, the Reference Token should be at a known address (the authority’s address or a designated script address).

Step 5: Read the inline datum (CIP-31 Reference Input). Using CIP-31 reference inputs, the verifier reads the Reference Token’s inline datum *without spending the UTxO*. This is a read-only operation that does not require any private key and does not create a transaction. The datum contains the credential metadata, including the **status** field, license type, jurisdiction, issue date, and licensee identity.

Step 6: Check status and expiration. The verifier confirms two conditions: (1) **status** == "active" (if **status** == "revoked", verification fails immediately), and (2) **expiration** > **current_slot** (if the credential has expired, verification fails). The current slot is obtained from the Cardano chain tip.

Step 7: Verification complete. If all checks pass, the credential is verified. The verifier now has cryptographic proof that: the credential was issued by a recognized authority (Policy ID match), the credential is currently active (datum status check), the credential has not expired

(slot comparison), and the licensee holds the corresponding User Token (UTxO query).

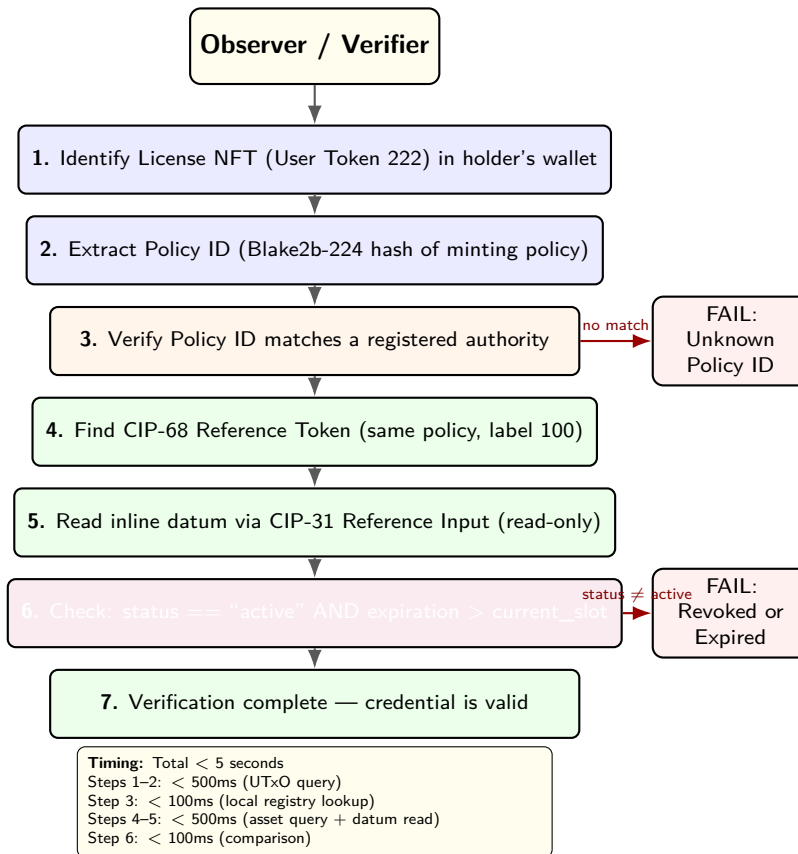


Figure 5.6: Seven-step verification flow. An observer can verify a credential in under 5 seconds with no permission required. Failure at Step 3 (unknown Policy ID) or Step 6 (revoked/expired status) immediately terminates verification with a negative result.

5.6.2 Permissionless Verification

A crucial property of this verification flow is that it is **permissionless**. The observer does not need:

- An account or API key with the issuing authority
- A bilateral trust agreement with any party
- Any special software beyond a Blockfrost API client (or any Cardano chain indexer)
- The licensee’s cooperation (the on-chain state is public)

This is fundamentally different from current verification infrastructure, where a hospital must contact the specific state medical board, provide credentials, wait for a response, and trust that the board’s database is accurate and current. In the blockchain-based system, the verification is mathematical: the Policy ID either matches or it does not, the datum either shows “active” or it does not, and the expiration either exceeds the current slot or it does not.

5.6.3 Verification Without a Full Node

The verification flow described above uses the Blockfrost API, which means the verifier trusts Blockfrost to return accurate chain data. For higher-assurance verification, the observer can run their own Cardano node and query the ledger directly, eliminating all third-party trust. For an intermediate assurance level, Mithril snapshot proofs (planned for 2026) will allow light clients to verify chain state with cryptographic proofs without running a full node.

5.7 Cost Economics

Understanding the on-chain costs is essential for evaluating the system’s economic viability. Every Cardano transaction has two cost components: the **transaction fee** (paid to stake pool operators, approximately 0.17–0.30 ADA depending on transaction size and Plutus execution) and the **minimum UTxO deposit** (ADA locked at the output UTxO, recoverable when the UTxO is consumed, approximately 1.5–2.0 ADA depending on the datum and token payload). The table below summarizes costs for each operation.

Table 5.1: On-chain cost breakdown for credential operations

Operation	Tx Fee (ADA)	minUTxO (ADA)	Total (ADA)
Mint License NFT (CIP-68 pair)	~0.2	~2.0	~2.2
Mint Signature Tokens (batch of 100)	~0.2	~2.0	~2.2
Mint Validity Token	~0.2	~2.0	~2.2
Sign Document (deposit to validator)	~0.2	~2.0	~2.2
Finalize Work Product (Plutus V2)	~0.3	~2.0	~2.3
Revoke License (datum update)	~0.3	0 [†]	~0.3
Renew Validity (new token mint)	~0.2	~2.0	~2.2 + dues
Burn Reference Token (cleanup)	~0.3	−2.0 [‡]	net ~−1.7

[†]Revocation modifies the existing Reference Token UTxO in place; the ~2.0 ADA deposit remains locked.

[‡]Burning the Reference Token releases the locked minUTxO, yielding a net refund of ~1.7 ADA after fees.

5.7.1 Comparison to Ethereum

The same operations on Ethereum would cost significantly more due to the ERC-721 contract execution model. Minting an NFT on Ethereum requires deploying or interacting with a smart contract, with gas costs that vary dramatically based on network congestion. During periods of high activity, gas prices have exceeded 500 gwei, pushing a single NFT mint above \$100. Even at moderate gas prices, a typical ERC-721 mint costs \$1–15, and complex operations like multi-party signature collection (requiring multiple contract calls and state updates) can cost \$5–50 per operation. Cardano’s deterministic fee model eliminates this variability entirely: the cost of a transaction is known before submission and does not change based on network load.

Solana offers lower absolute costs (sub-\$0.01 per transaction) but with a different risk profile:

multiple network outages, a proof-of-history consensus mechanism with less formal verification than Ouroboros, and a programming model (Rust-based programs) less suited to formal verification of credential logic.

5.7.2 Annual Cost Projection

For a deployment managing 10,000 professional licenses with standard operational assumptions (one license mint per licensee, 100 signature tokens per licensee per year, one validity renewal per year, 10 signing operations per licensee per year), the annual on-chain costs at current ADA prices (\$0.30/ADA) are:

Table 5.2: Annual cost projection for 10,000 licenses at \$0.30/ADA

Category	ADA	USD
Initial license minting (one-time, year 1)	22,000	\$6,600
Signature token minting (annual batch)	22,000	\$6,600
Validity token renewal (annual)	22,000	\$6,600
Signing operations (100,000/year)	220,000	\$66,000
Revocations (est. 2% = 200/year)	60	\$18
Year 1 Total	286,060	\$85,818
Recurring Annual	264,060	\$79,218

Note: The bulk of the cost is signing operations. If each licensee performs only 1 signing per year (as in annual license renewal), the recurring cost drops to approximately \$6,300–8,400 annually. Verification queries are free (read-only Blockfrost API calls). Hydra state-channel batching (2026 roadmap) could reduce signing costs by 90%+.

The key economic insight is that **verification is free**. Reading on-chain state through Blockfrost or any chain indexer does not require a transaction and incurs no ADA cost. The current verification infrastructure charges \$5–25 per query (FSMB, Nursys, NCEES verification services). For an organization performing thousands of verifications annually, the savings from free verification alone can offset the entire on-chain transaction cost.

5.7.3 Locked ADA and Capital Efficiency

The minUTxO requirement means that ADA is locked at token-carrying UTxOs. For 10,000 licenses, approximately 20,000 ADA (~\$6,000) is locked at Reference Token UTxOs, plus additional ADA locked at signature and validity token UTxOs held by licensees. This locked ADA is not lost—it can be recovered by consuming the UTxOs—but it represents an opportunity cost. CIP-68 datum patterns and potential future Hydra state channels could reduce locked ADA requirements by over 90% through token bundling and off-chain settlement.

5.8 Chapter Summary

This chapter has presented the complete credential verification system built on Cardano. The key takeaways are:

1. **Three-layer architecture:** The blockchain provides settlement finality, the application layer provides business logic, and the SQLite database provides fast queries. Blockfrost bridges the application to the chain.
2. **Four token types:** License NFTs (CIP-68 dual-token pair), signature tokens (fungible, consumed during signing), validity tokens (time-bounded, linked to dues), and work products (logical composites coordinated by validators).
3. **Four wallet roles:** Authority (full minting privileges, server-side keys), licensee (non-custodial CIP-30 wallet), signer (non-custodial CIP-30 wallet), and observer (read-only, public key only).
4. **Eight lifecycle steps:** Policy creation, license minting, token minting, token holding, signing, verification, renewal, and revocation—each with specific participants, costs, and on-chain effects.
5. **CIP-68 revocation:** The authority controls the Reference Token (100) and can update its datum to `status: revoked` unilaterally, without touching the licensee’s wallet. This is the core architectural innovation that makes on-chain revocation possible in eUTxO.
6. **Permissionless verification:** Any observer can verify a credential in under 5 seconds by checking the Policy ID against a registry and reading the Reference Token’s inline datum. No account, no API key, no bilateral agreement required.
7. **Economically viable:** Annual on-chain costs for 10,000 licenses range from \$6,300 (minimal signing) to \$85,000 (heavy signing), with verification queries free. This compares favorably to current verification service fees of \$5–25 per query.

The next chapter examines the smart contract layer in detail, including the Plutus V2 validator logic, redeemer patterns, and the security properties that make the system resistant to the six attack surfaces identified in the threat model.

Chapter 6

Multi-Party Document Signing, Privacy, and Legal Compliance

The preceding chapters established how a single professional obtains a license NFT, holds signature and validity tokens, and participates in the credential ecosystem. This chapter confronts three harder questions that arise the moment the system moves from individual credentials to real-world workflows: *How do multiple professionals sign a single document while proving their credentials? How does a public blockchain protect private information? Does any of this hold up in court?*

These three concerns—multi-party signing, privacy architecture, and legal compliance—are deeply intertwined. A signing workflow that leaks personal data violates GDPR. A privacy scheme that prevents verification defeats the legal admissibility of the signature. A legally compliant system that cannot handle concurrent signers is useless for the building-design reviews, multi-attorney closings, and medical peer-review panels that constitute the system’s primary use cases. This chapter addresses all three in sequence and then shows how they compose into a coherent whole.

Part A: Multi-Party Document Signing

6.1 The Signing Problem

Consider a concrete scenario. A seismic retrofit design for a hospital in Nevada requires signatures from two licensed Professional Engineers (one structural, one geotechnical) and one licensed notary public. Each professional must *prove* that they hold a valid credential *at the exact moment* they sign. If either engineer’s license was suspended yesterday, the signature is worthless—and potentially fraudulent.

Under current practice, this verification happens through one of three mechanisms, all of which

are inadequate:

- **Wet signatures with manual verification.** Each professional signs a physical document. The reviewing authority then contacts each licensing board—potentially across multiple states—to confirm that each license was active on the date of signing. This process takes days to weeks and produces no machine-verifiable audit trail.
- **Disconnected e-signature tools.** Services like DocuSign or Adobe Sign capture digital signatures, but they verify *identity* (“this email address clicked ‘sign’”), not *credentials* (“this person held an active PE license in California at 14:32 UTC on February 19, 2026”). The credential verification step is still a separate, manual process.
- **PDF digital signatures with PKI.** X.509 certificate-based PDF signatures provide cryptographic proof of identity, but the certificate’s validity period is coarse (typically one year), the revocation check depends on the issuing CA’s CRL or OCSP responder being online, and there is no standard mechanism for binding the signature to a *professional license* rather than a *personal identity*.

The fundamental gap is that no existing commercial signing tool answers the question: “Was this signer a licensed professional in good standing at the moment of signing?” The credential verification system described in this guide closes that gap by making credential status an *on-chain, machine-verifiable precondition* of the signing act itself.

The multi-party dimension compounds the difficulty. It is not enough for one signer to prove credentials; every required signer must do so, and the final work product must atomically record that *all* required signatures were collected with *all* credentials verified. Partial completion—two of three signatures—must not be treated as final. Yet the system must also handle the practical reality that three professionals in different time zones will not sign simultaneously.

6.2 The Six-Phase Signing Workflow

The multi-party signing workflow proceeds through six distinct phases, each with specific on-chain and off-chain operations. Understanding this workflow in detail is essential because it is the mechanism by which the credential system transforms from a *credential registry* into a *document signing infrastructure* with built-in credential verification.

6.2.1 Phase 1: Setup

The process begins when an authority (or an authorized coordinator) creates a **work product entry** in the system. This entry records three pieces of information:

1. **Document hash.** The SHA-256 hash of the document to be signed. This 32-byte value uniquely identifies the document and ensures that any modification—even changing a single bit—produces a completely different hash. The document itself is never stored on-chain; only its fingerprint is.
2. **Required signers.** A list of public key hashes (PKHs) identifying the professionals who must sign. Each PKH corresponds to a Cardano wallet address derived from the signer’s CIP-1852 HD wallet.
3. **Work product metadata.** A human-readable description, project identifier, and creation timestamp. This metadata is stored in the off-chain database and referenced by the on-chain records.

At this stage, nothing is written to the blockchain. The work product entry exists only in the local SQLite database. The purpose of Phase 1 is to establish the *requirements*—what document, which signers—before any on-chain resources are committed.

6.2.2 Phase 2: Credential Verification

Before any signer can deposit tokens, the system verifies that each required signer holds the necessary credentials. For each signer, three checks are performed:

1. **License NFT (CIP-68).** The system queries the Cardano blockchain to confirm that the signer’s wallet contains a license NFT minted under a recognized authority’s policy ID. Critically, it also reads the CIP-68 *Reference Token* (asset label 100) held by the authority to confirm that the inline datum field **status** is set to **active**—not **revoked** or **suspended**. This is the mechanism by which real-time credential status is verified: even if the signer possesses the User Token (asset label 222), the authority can unilaterally revoke the credential by updating the Reference Token’s datum.
2. **Signature tokens.** The signer must hold at least one signature token (a fungible native token) minted under the same authority’s policy. Each signing act consumes one signature token, so the system checks the signer’s wallet balance.
3. **Validity token.** The signer must hold a validity token whose expiry slot is in the *future*. The validity token represents current good standing—it may be linked to dues payments, continuing education, or periodic reauthorization. If the validity token has expired, the signer must renew it (typically by paying dues through the Dues Enforcement Contract) before proceeding.

If any signer fails any check, the system halts and reports which signer failed which requirement. This prevents wasted on-chain transactions and provides clear feedback for remediation.

6.2.3 Phase 3: Signing (Token Deposit)

This is the critical on-chain phase. Each signer constructs and submits a transaction that deposits **one signature token** and **one validity token** to the `SignatureCollectionValidator` script address. The transaction also attaches a `SignerDatum` (detailed in Section 6.4) recording the signer’s identity, the document hash, and temporal information.

The crucial design decision is that **each signer creates an independent UTxO** at the script address. This means that if three signers deposit, there are three separate UTxOs sitting at the validator address, each containing one signer’s tokens and datum. This is not an incidental implementation choice—it is the solution to the eUTxO concurrency problem (Section 6.3).

Each signer’s deposit transaction uses the `COLLECT` redeemer (redeemer index 0), which tells the validator: “I am a required signer, and I am depositing my tokens for this work product.” The validator script checks that the deposited tokens come from a recognized policy ID and that the datum fields are well-formed.

6.2.4 Phase 4: Collection

As deposits arrive, the system tracks the state of the work product: which signers have deposited and which have not. This tracking occurs both off-chain (in the SQLite database, for dashboard display and status queries) and on-chain (by querying the validator address for UTxOs whose datums reference the work product’s document hash).

The collection phase is inherently asynchronous. Signer A might deposit at 9:00 AM EST, Signer B at 2:00 PM EST, and Signer C at 11:00 PM EST the following day. The system accommodates this by not requiring any ordering or timing constraints on deposits—only that all required deposits are present before finalization.

During this phase, any signer who has already deposited may invoke the RECLAIM redeemer (redeemer index 2) to cancel their participation and recover their tokens. This is important for practical workflow management: if a project is cancelled or a signer needs to be replaced, deposited tokens are not permanently locked.

6.2.5 Phase 5: Finalization

Once all required signers have deposited, the authority submits a FINALIZE transaction (redeemer index 1). This is the most consequential on-chain operation in the entire workflow because it **atomically consumes ALL per-signer UTxOs** in a single transaction.

The atomicity guarantee is critical. Either *all* deposits are consumed and the work product is finalized, or *none* are—there is no intermediate state where some signatures are finalized and others are not. This is a natural property of Cardano’s transaction model: a transaction either succeeds completely or fails completely.

The finalize transaction:

- Consumes each signer’s deposit UTxO (one input per signer)
- Verifies that the authority’s signature is present on the transaction
- Confirms that the set of consumed UTxOs covers all required signers
- Records the finalization on-chain as a completed transaction with all signer datums preserved in the transaction metadata

After finalization, the signature tokens and validity tokens are consumed (effectively burned from the signers’ perspective, as they now reside in the spent transaction history rather than in any wallet).

6.2.6 Phase 6: Verification

After finalization, *any party*—a building inspector, a court, a regulatory body, a counterparty—can verify the signed work product by querying the blockchain. The verification process confirms:

- **Who signed:** Each signer’s PKH is recorded in the **SignerDatum** attached to their deposit UTxO.
- **When they signed:** The deposit slot timestamp records when each signer’s deposit transaction was included in a block.
- **What they signed:** The SHA-256 document hash in each datum binds the signature to a specific document version.
- **Credential status at signing time:** The validity token’s expiry slot, combined with the deposit slot, proves that the signer’s credentials were valid at the moment of signing.
- **Finalization:** The finalize transaction’s existence proves that the authority confirmed all signatures were collected and valid.

This verification requires only a Cardano blockchain query—no contact with any licensing board, no reliance on any centralized server, no trust in any single party’s attestation.

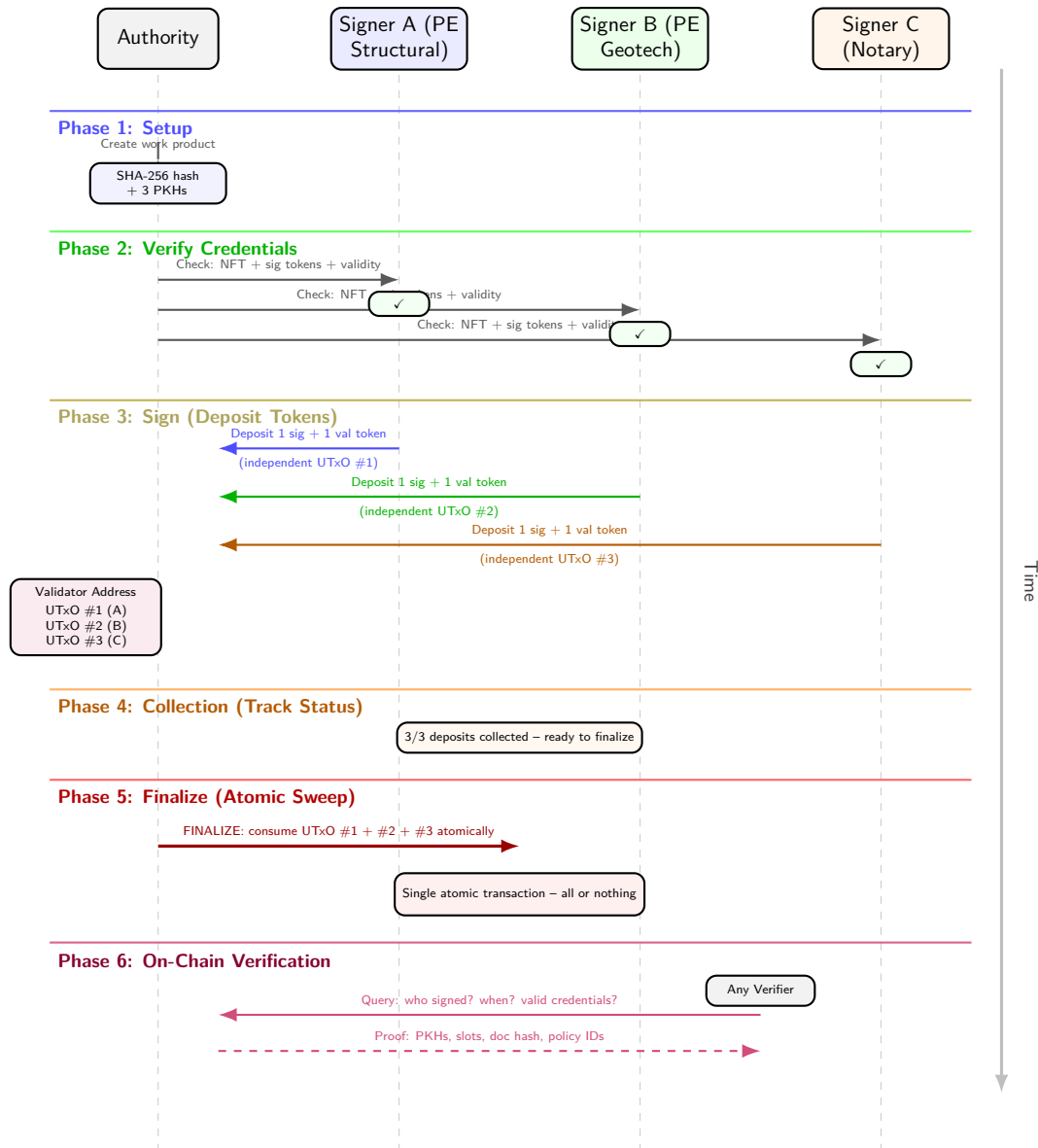


Figure 6.1: Complete six-phase multi-party signing sequence with three signers. Each signer creates an independent UTxO at the validator address (Phase 3), avoiding eUTxO concurrency conflicts. The authority’s finalize transaction (Phase 5) atomically consumes all deposits.

6.3 The eUTxO Concurrency Solution

6.3.1 Understanding the Problem

Cardano uses the **extended UTxO (eUTxO) model**, which is fundamentally different from Ethereum’s account model. In the account model, a smart contract has a single “account” whose state is updated by transactions. Multiple transactions can modify the same account in the same block (the EVM processes them sequentially within the block). In the eUTxO model, each UTxO is a discrete, one-time-use object: a transaction *consumes* one or more UTxOs and *produces* new ones. A UTxO can be consumed exactly once—it is destroyed in the process.

This creates a concurrency challenge. Suppose the system used a naive design where all signers deposit into the *same* UTxO at the validator address. Signer A sees UTxO X (containing zero deposits) and builds a transaction that consumes UTxO X and produces UTxO X' (containing A's deposit). Meanwhile, Signer B also sees UTxO X and builds a transaction consuming it to produce UTxO X'' (containing B's deposit). Both transactions reference UTxO X as an input. But UTxO X can only be consumed once.

When both transactions are submitted to the network, only one will succeed. The other will fail with an error indicating that its input UTxO has already been consumed. This is the **eUTxO contention problem**: concurrent transactions competing to spend the same UTxO cause all but one to fail.

In high-throughput DeFi applications on Cardano, this problem is well-known and has motivated various batching and order-book designs. For a credential signing system, however, the problem is especially acute because signers are *expected* to act concurrently—telling three professionals “please sign one at a time and wait for on-chain confirmation between each” is operationally unacceptable.

6.3.2 The Independent UTxO Solution

The solution is elegant in its simplicity: **each signer creates their own independent UTxO at the script address**. There is no shared UTxO to contend over. Signer A creates UTxO_A, Signer B creates UTxO_B, and Signer C creates UTxO_C—all at the same validator address, but as separate, non-conflicting UTxOs. Since no two transactions reference the same input, they can all succeed in the same block if submitted simultaneously.

The authority then “sweeps” all per-signer UTxOs in a single finalization transaction. This finalize transaction has multiple inputs (one per signer UTxO) and the authority's signature, and it is the only transaction that needs to consume multiple UTxOs atomically. Since only the authority submits the finalize transaction, there is no concurrency conflict—only one party is building this transaction.

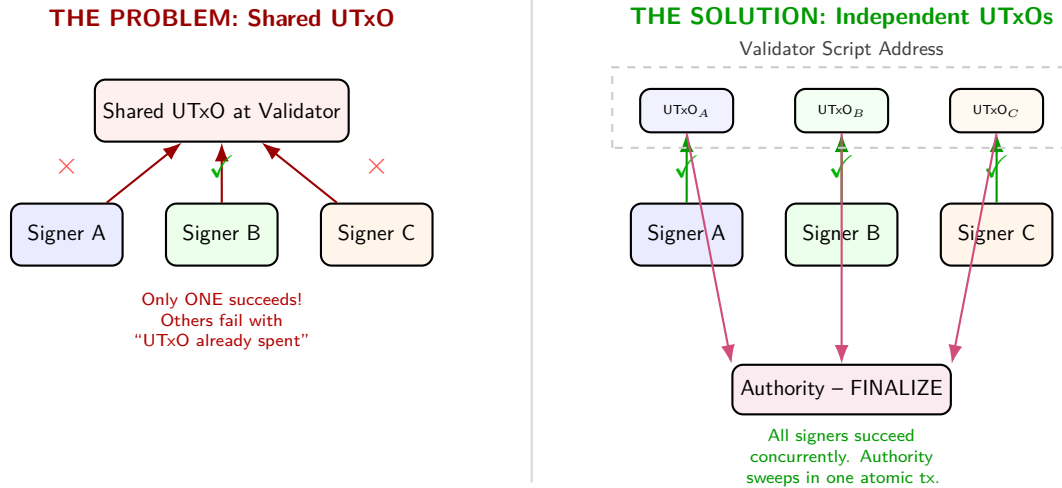


Figure 6.2: eUTxO concurrency: the shared-UTxO problem (left) versus the independent-UTxO solution (right). When multiple signers target the same UTxO, only one transaction succeeds. With independent UTxOs, all signers can deposit concurrently, and the authority sweeps all deposits in a single finalize transaction.

6.3.3 Why This Works in Cardano’s Model

The independent-UTxO pattern works because Cardano’s eUTxO model treats each UTxO as a separate object. A validator script is identified by its script hash, which determines the script address. *Any number of UTxOs* can sit at the same script address, each with its own datum, its own tokens, and its own spending conditions. The validator script is invoked separately for each UTxO being consumed, receiving the datum attached to *that specific UTxO* and the redeemer provided by the transaction.

This means the validator can enforce per-signer rules (“the datum’s PKH matches a required signer”) without needing to manage a shared state. The “global” view—“have all required signers deposited?”—is checked only in the finalize transaction, where the authority provides all UTxOs as inputs and the script can verify completeness across the full input set.

This design pattern—per-user UTxOs at a shared script address, with a privileged party performing the aggregation step—is a general solution to eUTxO concurrency and is used in Cardano DEX order books, lending protocols, and governance systems. In the credential context, it maps naturally to the real-world workflow: professionals act independently, and an authority aggregates their actions into a final result.

6.4 SignerDatum Structure

Each deposit UTxO at the validator address carries a **SignerDatum**—a structured piece of data (a Plutus `PlutusData` type serialized as CBOR) that creates an immutable record of the signing act. The datum contains five fields:

Table 6.1: SignerDatum fields

Field	Type	Description
<code>signer_pkh</code>	PubKeyHash (28 bytes)	The public key hash of the signer. This is derived from the signer’s Ed25519 verification key and serves as the on-chain identity binding. Combined with the transaction signature, it proves that the holder of the corresponding private key authorized this deposit.
<code>document_hash</code>	ByteString (32 bytes)	The SHA-256 hash of the document being signed. This binds the signature to a specific, unalterable document version. Any party with the original document can recompute the hash and verify it matches.
<code>policy_id</code>	MintingPolicyHash (28 bytes)	The policy ID under which the signer’s credential tokens were minted. This allows verifiers to confirm which authority issued the signer’s credentials, establishing provenance.
<code>validity_expiry</code>	POSIXTime (slot number)	The expiry slot of the validity token deposited alongside this datum. Combined with <code>deposit_slot</code> , this field proves that the signer’s credentials were valid (not expired) at the time of signing.
<code>deposit_slot</code>	POSIXTime (slot number)	The slot at which this deposit transaction was included in a block. This provides a precise timestamp of the signing act, anchored to Cardano’s consensus-verified slot timeline.

The combination of these five fields creates a self-contained proof of credential-verified signing. A verifier needs nothing beyond the Cardano blockchain to confirm: (1) `signer_pkh` identifies who signed; (2) `document_hash` identifies what was signed; (3) `policy_id` identifies which authority vouches for the signer; (4) `validity_expiry` > `deposit_slot` proves the credential was valid at signing time.

The datum is attached to the UTxO when it is created (Phase 3) and becomes part of the blockchain’s permanent record when the UTxO is consumed (Phase 5). Because Cardano stores spent transaction data in the immutable ledger history, the `SignerDatum` is preserved indefinitely—even after the UTxO itself has been spent. This is the mechanism by which the system provides permanent, tamper-proof signing records.

Part B: Privacy and Compliance

6.5 The Privacy Architecture

A blockchain is, by design, a public, transparent, and immutable ledger. Every transaction, every token transfer, every datum is visible to anyone running a node or querying an explorer. This transparency is precisely what makes blockchain valuable for credential verification—but it creates an immediate tension with data privacy requirements. You cannot put a person’s name, license number, home address, and date of birth on a public blockchain and simultaneously comply with data protection laws.

The system resolves this tension through a strict **architectural separation** between on-chain and off-chain data. The principle is simple but rigorously applied: *anything that touches the blockchain must be incapable of identifying a natural person without access to a separate, private mapping.*

6.5.1 On-Chain Data (Public, Immutable)

The following data elements exist on the Cardano blockchain and are visible to any observer:

- **Policy IDs.** These are Blake2b-224 hashes of minting scripts. A policy ID like `a1b2c3...f4e5` reveals nothing about the authority that created it unless the observer already knows the mapping between policy IDs and authorities.
- **Token asset names.** Names like `LIC_PE_001` or `SIG_TOKEN` are visible on-chain but contain no personal data. The naming convention uses role-based prefixes, not personal identifiers.
- **Transaction signatures (PKHs).** Public key hashes are 28-byte values derived from Ed25519 verification keys. A PKH like `8a3f...c7d2` is a pseudonym—it identifies a *key pair*, not a *person*. Without the off-chain mapping, an observer cannot determine whose key it is.
- **CIP-25/68 metadata fields.** The on-chain metadata attached to license NFTs includes credential type, jurisdiction code, issue date, status, and policy-specific fields. These are designed to be *functionally useful* (a verifier can confirm “this is a PE license issued in California that is currently active”) without being *personally identifying* (the metadata does not include the licensee’s name or license number).
- **Slot timestamps.** Cardano’s slot numbers provide temporal ordering and can be converted to UTC timestamps. These record *when* events occurred but not *who* was involved (beyond the pseudonymous PKH).

6.5.2 Off-Chain Data (Private, Erasable)

Personal data that connects blockchain pseudonyms to real-world identities is stored exclusively in an off-chain database:

- **Full names** of licensees, signers, and authority contacts
- **License numbers** as assigned by the issuing jurisdiction
- **Physical addresses** and contact information
- **Email addresses** and phone numbers
- **PKH-to-identity mappings** that link on-chain pseudonyms to real people

This data is stored in an **AES-256-GCM encrypted SQLite database**. The encryption

ensures that even if an attacker gains access to the database file, the contents are unreadable without the encryption key.

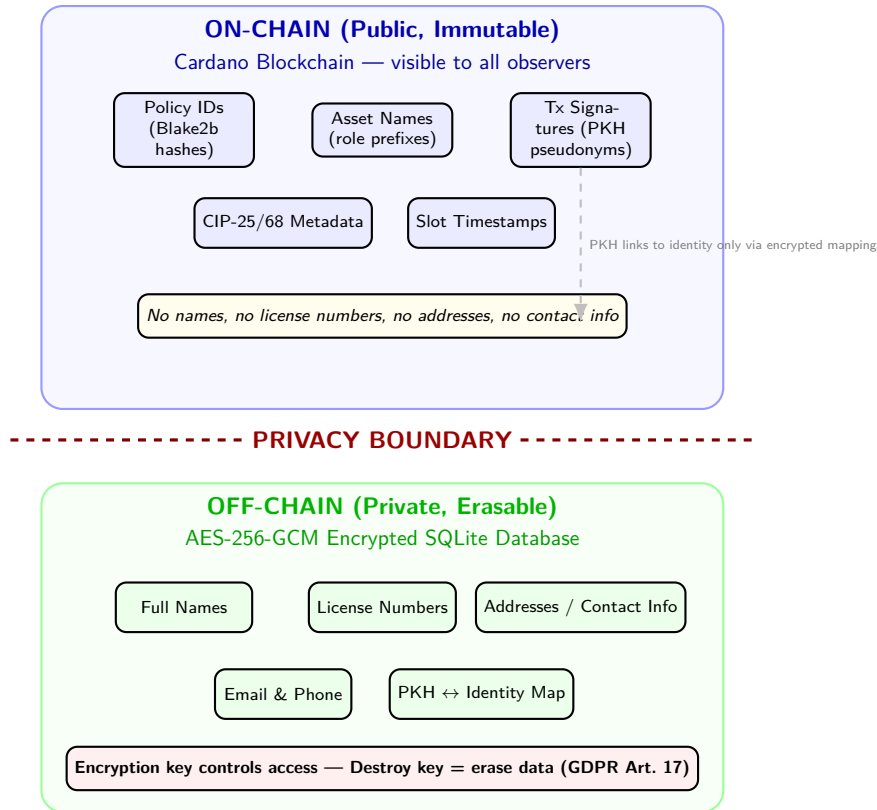


Figure 6.3: Privacy architecture: strict separation between public on-chain data (policy IDs, asset names, PKHs, metadata, timestamps) and private off-chain data (names, license numbers, addresses, contact information). The PKH-to-identity mapping exists only in the encrypted off-chain database.

6.6 AES-256-GCM Encryption

The off-chain database and the authority’s wallet keys at rest are protected by **AES-256-GCM** encryption. Understanding what this means requires unpacking the acronym:

AES stands for *Advanced Encryption Standard*, a symmetric-key encryption algorithm adopted by NIST in 2001 (FIPS 197). “Symmetric-key” means the same key is used for both encryption and decryption. AES operates on 128-bit blocks of data, transforming plaintext into ciphertext through a series of substitution, permutation, and mixing operations.

256 refers to the key length in bits. A 256-bit key provides 2^{256} possible key values—a number so large ($\approx 1.16 \times 10^{77}$) that brute-force enumeration is considered computationally infeasible for the foreseeable future, including against quantum computers using Grover’s algorithm (which reduces the effective security to 128 bits, still considered sufficient).

GCM stands for *Galois/Counter Mode*, an authenticated encryption mode that provides two guarantees simultaneously:

1. **Confidentiality.** The ciphertext reveals nothing about the plaintext to anyone who does not possess the key. GCM achieves this using CTR (Counter) mode, which turns AES into a stream cipher by encrypting sequential counter values and XOR-ing the result with the plaintext.
2. **Integrity and Authentication.** GCM produces an authentication tag (typically 128 bits) alongside the ciphertext. This tag is computed using Galois field multiplication over the ciphertext blocks. When decrypting, the recipient recomputes the tag and compares it with the stored tag. If even a single bit of the ciphertext has been tampered with, the tags will not match, and the decryption is rejected.

The combination of confidentiality and integrity is critical for the credential system. Without integrity protection, an attacker who gains access to the encrypted database could modify ciphertext bytes (even without knowing the key) and potentially cause the decrypted data to contain corrupted-but-accepted values. GCM prevents this: any modification to the ciphertext is detected and rejected.

In the credential system, AES-256-GCM is used in two contexts:

- **Authority wallet keys at rest.** The authority’s Ed25519 signing key is encrypted before being stored in the SQLite database. The encryption key is derived from a passphrase or stored in a separate key management system. This ensures that database compromise alone does not expose the authority’s signing capability.
- **Off-chain PII database.** Personal data (names, license numbers, addresses) is encrypted at the field level or database level. Per-credential encryption keys enable granular erasure: destroying one key renders one person’s data permanently inaccessible without affecting other records.

6.7 GDPR and Crypto-Shredding

6.7.1 The GDPR Dilemma

The General Data Protection Regulation (GDPR), enforced across the European Union since May 2018, establishes a comprehensive framework for personal data protection. Article 17 is the provision most directly in tension with blockchain technology:

“The data subject shall have the right to obtain from the controller the erasure of personal data concerning him or her without undue delay, and the controller shall have the obligation to erase personal data without undue delay. . . .” — GDPR Article 17(1)

This is the **right to erasure**, commonly called the “right to be forgotten.” It requires that upon request, a data controller must delete all personal data associated with the requesting individual.

Blockchain, however, is *immutable by design*. Once a transaction is confirmed and finalized, it cannot be altered or deleted. This is not a bug—it is the core value proposition. Credential verification depends on the guarantee that a licensing board’s attestation, once published, cannot be retroactively modified or erased.

The tension is stark: GDPR says “delete it,” and blockchain says “it cannot be deleted.” The European Data Protection Board (EDPB) has made clear in its April 2025 guidelines that

technical impossibility is not an acceptable excuse for non-compliance, with penalties reaching up to 20 million EUR or 4% of global annual turnover, whichever is higher.

6.7.2 The Solution: Crypto-Shredding

The credential system resolves this dilemma through a technique called **crypto-shredding** (also known as cryptographic erasure). The approach is architecturally simple but legally significant:

1. **Personal data is never stored on-chain.** As described in Section 6.5, names, license numbers, addresses, and other personally identifiable information (PII) exist only in the encrypted off-chain database.
2. **Each credential’s PII is encrypted with a per-credential key.** Rather than encrypting the entire database with a single key, the system uses individual encryption keys for each person’s data. This enables selective erasure without affecting other records.
3. **To “erase” data, destroy the encryption key.** When a data subject exercises their Article 17 right, the system destroys the encryption key associated with their data. The encrypted ciphertext remains in the database, but without the key, it is computationally indistinguishable from random noise. The data is effectively and permanently erased.
4. **On-chain hashes become meaningless.** The on-chain data consists of cryptographic hashes (policy IDs, PKHs, asset names) and metadata that does not contain PII. A PKH like `8a3f...c7d2` cannot be reversed to recover the original public key (hash functions are one-way), and even if it could, the public key does not reveal the person’s identity without the destroyed off-chain mapping.

This approach aligns with guidance from two regulatory bodies:

- **French CNIL (2018).** The Commission Nationale de l’Informatique et des Libertés published guidelines recognizing that storing only hashed or encrypted data on-chain, combined with off-chain key management and destruction capabilities, constitutes a GDPR-compliant architecture.
- **EDPB Framework (2025).** The European Data Protection Board’s updated guidelines acknowledge crypto-shredding as a valid technical measure for blockchain systems, provided that the on-chain data cannot be used to re-identify individuals after key destruction.

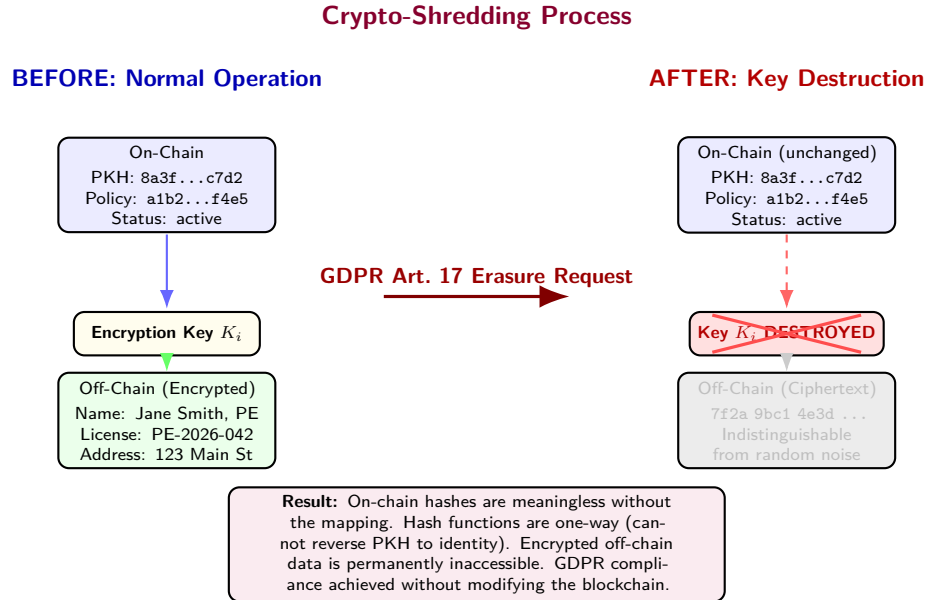


Figure 6.4: Crypto-shredding process. Left: during normal operation, the encryption key K_i connects on-chain pseudonyms to off-chain personal data. Right: after a GDPR erasure request, destroying K_i renders the off-chain data permanently unreadable while on-chain hashes remain but become unlinkable to any natural person.

6.7.3 Why Hashes Cannot Be Reversed

A common concern is: “If the PKH is derived from a public key, can’t someone reverse the hash to recover the public key, and then use the public key to identify the person?” The answer is no, for two reasons:

First, Blake2b-224 (the hash function used for Cardano PKHs) is a cryptographic hash function with preimage resistance. Given a hash output h , finding any input x such that $\text{Blake2b-224}(x) = h$ requires approximately 2^{224} operations—a number vastly exceeding the computational capacity of all computers ever built.

Second, even if an attacker could somehow recover the public key from the PKH, the public key is a 32-byte Ed25519 verification key. This key does not inherently contain any personal information. The mapping from public key to person exists only in the off-chain database—which has been crypto-shredded.

The combination of one-way hash functions and off-chain key destruction creates what privacy engineers call **unlinkability**: the on-chain data points cannot be linked to a natural person after crypto-shredding, satisfying the GDPR’s erasure requirement in substance.

Part C: Legal Framework

6.8 ESIGN Act (US Federal)

The Electronic Signatures in Global and National Commerce Act (15 U.S.C. §§ 7001–7031), enacted in 2000, is the foundational US federal law governing electronic signatures. Its central provision is deceptively simple: *an electronic signature cannot be denied legal effect solely because it is in electronic form*. The Act is deliberately technology-neutral—it does not prescribe *how* an electronic signature must be implemented, only that it must satisfy four requirements.

6.8.1 Requirement 1: Intent to Sign

The signer must demonstrate a deliberate intent to sign the document. In traditional e-signature tools, this is typically satisfied by clicking an “I agree” button. In the credential system, intent is demonstrated by the deliberate invocation of the `sign_document()` function, which requires the signer to:

- Identify the specific document (by hash) they are signing
- Decrypt their private signing key (requiring passphrase entry or hardware wallet interaction)
- Construct and submit an on-chain transaction that deposits tokens to the validator address

This multi-step process—far more deliberate than clicking a button—provides strong evidence of intent. A signer cannot accidentally deposit tokens to a validator address; the process requires explicit, informed action at multiple stages.

6.8.2 Requirement 2: Consent to Conduct Business Electronically

The signer must have consented to conducting business electronically. The credential system satisfies this through the onboarding process: when a professional registers for the system and receives their wallet, credential tokens, and access credentials, they explicitly accept terms of service that include consent to electronic transactions. This consent is recorded in the off-chain database with a timestamp and is presented to the signer before any wallet generation or token minting occurs.

6.8.3 Requirement 3: Association of Signature with Record

The electronic signature must be logically associated with the record being signed. The credential system achieves this through cryptographic binding at three levels:

1. **SHA-256 hash binding.** The `SignerDatum` includes the SHA-256 hash of the document, creating a mathematical link between the signature and the exact document version.
2. **Transaction binding.** The deposit transaction is signed with the signer’s Ed25519 private key, cryptographically proving that the holder of that key authorized this specific transaction.
3. **UTxO binding.** The deposited tokens (signature token + validity token) are locked at the validator address with the `SignerDatum`, creating an inseparable on-chain association between the signer’s identity, the document, and the signing act.

This triple binding is significantly stronger than the “email address clicked a link” association provided by most commercial e-signature tools.

6.8.4 Requirement 4: Record Retention

The signed record must be retained and reproducible. The credential system satisfies this through blockchain immutability: once the finalize transaction is confirmed, the complete signing record—who signed, what they signed, when they signed, and what credentials they held—is stored across thousands of Cardano nodes worldwide. The data cannot be altered, deleted, or lost (short of a catastrophic failure of the entire Cardano network). This level of retention surpasses any centralized storage system in both durability and tamper-resistance.

6.9 UETA (US State Law)

The Uniform Electronic Transactions Act (UETA) was drafted by the Uniform Law Commission in 1999 and has been adopted by **49 states plus the District of Columbia** (New York is the sole non-adopter, though it has its own Electronic Signatures and Records Act). UETA provides the state-level legal foundation for electronic signatures, complementing the federal ESIGN Act.

UETA’s key contribution is Section 9, which establishes that an electronic signature is “attributable to a person” if it can be shown by a “preponderance of the evidence” that the person performed the act. Cardano’s Ed25519 cryptography provides exceptionally strong attribution: possessing the private key corresponding to a PKH is, in practice, conclusive evidence of authorship. The probability of forging an Ed25519 signature without the private key is approximately 2^{-128} —effectively zero.

6.9.1 Blockchain-Specific State Amendments

Nine states have gone beyond basic UETA adoption to enact **blockchain-specific amendments** that explicitly recognize blockchain signatures and smart contracts. Two are particularly significant for the credential system:

Arizona HB 2417 (2017). Arizona was the first state to explicitly recognize blockchain signatures. A.R.S. §44-7061 provides that “a signature that is secured through blockchain technology is considered to be in an electronic form and to be an electronic signature” and that “a record or contract that is secured through blockchain technology is considered to be in an electronic form and to be an electronic record.” This statute directly validates the credential system’s on-chain signing mechanism: token deposits at a validator address, secured by Ed25519 cryptography on the Cardano blockchain, are electronic signatures under Arizona law.

Vermont 12 V.S.A. §1913 (2016). Vermont’s statute creates a **rebuttable presumption of authenticity** for blockchain records. Specifically, a “digital record electronically registered in a blockchain” is “self-authenticating” under the Vermont Rules of Evidence, provided that the record is accompanied by a “written declaration of a qualified person” stating that the record is accurate. This is an evidentiary advantage: in court, the party introducing a blockchain-verified credential does not bear the initial burden of proving its authenticity—the opposing party must affirmatively rebut the presumption. For the credential system, this means that a finalized work product with on-chain signing records carries automatic evidentiary weight in Vermont courts.

The remaining seven states with blockchain-specific provisions—Nevada, Tennessee, South Dakota, Wyoming, Illinois, Colorado, and others—provide varying degrees of explicit recognition for blockchain records and smart contracts. While the specific language differs, the trend is

clear: US state law is moving toward explicit statutory recognition of blockchain as a valid medium for legally binding records and signatures.

6.10 eIDAS 2.0 (European Union)

The European Union’s electronic identification and trust services regulation, known as eIDAS, is the most comprehensive legal framework for electronic signatures globally. The revised regulation, eIDAS 2.0 (Regulation (EU) 2024/1183), entered into force in 2024 and introduces provisions specifically relevant to blockchain-based systems.

6.10.1 Three Signature Tiers

eIDAS 2.0 defines three tiers of electronic signatures, each with increasing legal weight and technical requirements:

Tier 1: Simple Electronic Signature (SES). The broadest category, encompassing any “data in electronic form which is attached to or logically associated with other data in electronic form and which is used by the signatory to sign.” A typed name at the bottom of an email qualifies. There are no technical requirements beyond basic logical association. SES signatures are admissible as evidence but carry no presumption of validity.

Tier 2: Advanced Electronic Signature (AES). An AES must satisfy four requirements defined in Article 26:

1. **Uniquely linked to the signatory.** Each signer’s Ed25519 key pair is unique—no two signers share the same key. The PKH derived from the verification key is a unique 28-byte identifier.
2. **Capable of identifying the signatory.** The PKH-to-identity mapping in the off-chain database links the cryptographic identity to a natural person. The CIP-68 Reference Token’s metadata provides additional identification context.
3. **Created using data under the signatory’s sole control.** The Ed25519 private key, stored encrypted in the signer’s wallet (or in a hardware wallet via CIP-30), is under the signer’s exclusive control. No other party—including the authority—possesses the signer’s private key.
4. **Linked to the data signed in such a way that any subsequent change is detectable.** The SHA-256 document hash in the `SignerDatum`, combined with the transaction signature, ensures that any modification to the signed document would produce a different hash, immediately detectable by any verifier.

The credential system natively satisfies **all four AES requirements** without requiring any modifications or additional infrastructure. This is a significant legal result: AES signatures carry a presumption of validity across all EU member states and are admissible as evidence with enhanced legal weight.

Tier 3: Qualified Electronic Signature (QES). A QES is an AES that is additionally:

- Created by a **Qualified Electronic Signature Creation Device (QSCD)**—a certified hardware device that protects the signing key
- Based on a **Qualified Certificate** issued by a **Qualified Trust Service Provider (QTSP)** that has been audited and approved by an EU member state’s supervisory body

QES signatures have the *legal equivalent of handwritten signatures* across all EU member states. The credential system does not natively achieve QES status because it does not use a certified QSCD or a QTSP-issued certificate. However, a QTSP could integrate the credential system’s on-chain verification with its own qualified certificate infrastructure to provide QES-level signatures.

6.10.2 Qualified Electronic Ledger (Article 45l)

eIDAS 2.0 introduces a new trust service category: the **Qualified Electronic Ledger**. Article 45l defines this as a distributed ledger operated by a QTSP that provides “proof of the existence and integrity of electronic data recorded therein.” This provision was specifically designed to accommodate blockchain technology within the EU trust services framework.

For the credential system, Article 45l provides a clear pathway: if the Cardano blockchain (or a specific Cardano-based service) obtains QTSP status for ledger operation, then credentials recorded on it would carry the full legal weight of a qualified trust service. Cardano’s formally verified Ouroboros consensus protocol, with its published security proofs in peer-reviewed venues, provides a strong technical foundation for such qualification.

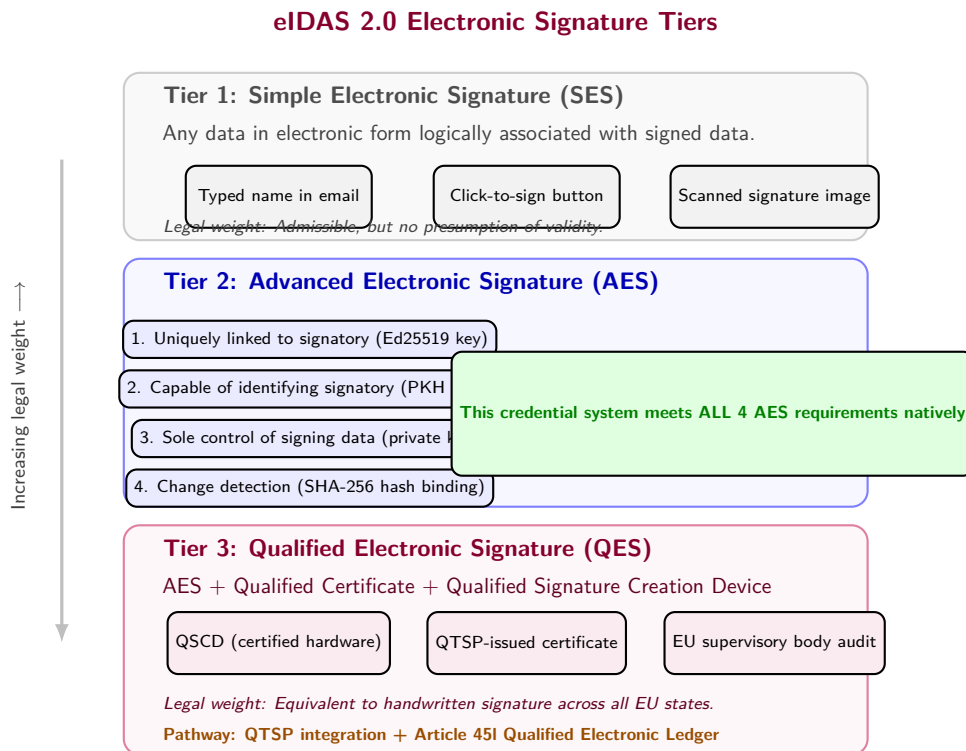


Figure 6.5: eIDAS 2.0 electronic signature tiers. The credential system natively meets all four Advanced Electronic Signature (AES) requirements. Qualified Electronic Signature (QES) status requires additional QTSP certification and hardware security modules, providing a clear upgrade pathway via Article 45l.

6.11 Putting It All Together: Compliance Summary

The multi-party signing workflow, privacy architecture, and legal framework are not independent concerns—they are three facets of a single design problem. The signing workflow must produce legally admissible records. The privacy architecture must not undermine the evidentiary value of those records. The legal framework must be satisfied without compromising the technical integrity of the system.

Table 6.2 summarizes how each component of the system maps to specific legal requirements across all three major frameworks.

Table 6.2: Compliance matrix: system components mapped to legal requirements

System Component	ESIGN Act	UETA / State Law	eIDAS 2.0
Ed25519 key pair <code>sign_document()</code> invocation	Association (Req. 3) Intent (Req. 1)	Attribution (Sec. 9) Intent to sign	AES Reqs. 1, 3 Signatory action
Onboarding consent	Consent (Req. 2)	Agreement to e-records	N/A
SHA-256 hash in datum	Association (Req. 3)	Record integrity	AES Req. 4
Blockchain immutability	Retention (Req. 4)	Record preservation	Art. 45l ledger
CIP-68 credential status	N/A	Real-time verification	Trust service
Crypto-shredding	N/A	N/A	GDPR Art. 17
PKH-to-identity mapping	N/A	N/A	AES Req. 2

The credential system achieves a rare balance: it provides stronger cryptographic guarantees than traditional e-signature tools (SHA-256 hash binding, Ed25519 signatures, immutable on-chain records), stronger credential verification than any existing signing platform (real-time on-chain license status), and GDPR compliance through architectural separation and crypto-shredding—all while satisfying the requirements of three major legal frameworks governing electronic signatures.

This compliance is not incidental; it is the result of architectural decisions made at the system’s foundation. The separation of on-chain and off-chain data, the use of PKHs as pseudonyms, the per-credential encryption keys, and the independent-UTxO signing pattern were all designed with legal requirements as first-class constraints alongside technical requirements.

Chapter Summary

- Multi-party document signing uses a six-phase workflow: Setup, Verify, Sign, Collect, Finalize, Verify. Each signer deposits tokens independently, and the authority finalizes atomically.
- The eUTxO concurrency problem—where multiple signers competing for the same UTxO causes all but one to fail—is solved by having each signer create an independent UTxO at the validator address.

- The **SignerDatum** structure records five fields (signer PKH, document hash, policy ID, validity expiry, deposit slot) that together prove who signed, what they signed, when, and with what credentials.
- Privacy is achieved through strict architectural separation: only pseudonymous hashes and policy IDs go on-chain; all personal data stays in an AES-256-GCM encrypted off-chain database.
- GDPR compliance is achieved through crypto-shredding: destroying per-credential encryption keys renders off-chain personal data permanently inaccessible while on-chain hashes remain meaningless without the mapping.
- The ESIGN Act requires intent, consent, association, and retention—all satisfied by the system’s deliberate signing process, onboarding flow, hash binding, and blockchain immutability.
- UETA, adopted by 49 states plus DC, provides the state-level legal foundation; nine states have blockchain-specific amendments with Arizona and Vermont being the most significant.
- eIDAS 2.0 defines three signature tiers (SES, AES, QES). The credential system natively meets all four AES requirements. Article 45l provides a pathway to qualified trust service status through the new “Qualified Electronic Ledger” category.

Chapter 7

The Software and the v4.0 Roadmap

This chapter is divided into three parts. Part A describes the `cardano-license` Python package—its architecture, API surface, database schema, and testing infrastructure. Part B presents the v4.0 roadmap: Aiken smart contracts, CIP-33 reference scripts, validity-token consolidation, HSM key storage, and KERI/multi-sig governance. Part C collects a comprehensive glossary of every technical term used throughout the study guide.

Part A — The Software

7.1 Package Overview

The `cardano-license` package is an open-source Python library released under the MIT license. As of this writing, the current release is version 2.1.0. The package provides a complete toolkit for professional credential management on the Cardano blockchain, encompassing wallet generation, license NFT minting, signature-token operations, validity management, CIP-68 reference-token revocation, Plutus V2 smart-contract deployment, and dues enforcement.

Installation is straightforward via the Python Package Index:

```
pip install cardano-license
```

Listing 7.1: Installing cardano-license

The package requires Python 3.10 or later and depends on four core libraries. **PyCardano** serves as the Cardano SDK, providing HD wallet generation, transaction building, Plutus V2 script support, and address derivation. **Blockfrost** provides the chain-query API, enabling the package to interact with the Cardano blockchain without running a full node. **aiosqlite** provides asynchronous SQLite database access, allowing the package to maintain local state (wallets, licenses, signatures, contracts) without blocking the event loop. **cryptography** provides AES-256-GCM authenticated encryption for wallet key files, using PBKDF2-HMAC-SHA256 key derivation with 100 000 iterations.

Table 7.1: Core dependencies of the `cardano-license` package.

Library	Role	Usage
PyCardano	Cardano SDK	HD wallets, transaction building, Plutus V2 scripts, CIP-1852 derivation
Blockfrost	Chain API	UTxO queries, transaction submission, balance lookups
aiosqlite	Async DB	Local persistence of wallets, licenses, signatures, contracts
cryptography	Encryption	AES-256-GCM key-file encryption with PBKDF2 derivation

The package follows a layered design philosophy: all on-chain interactions flow through PyCardano and Blockfrost, all local state is managed through aiosqlite, and all key material is protected by the cryptography module. This separation of concerns means that swapping the chain backend (e.g., from Blockfrost to Ogmios) or the database engine (e.g., from SQLite to PostgreSQL) would require changes to only one layer.

7.2 Architecture

The package is organized into four Python modules, each with a clearly defined responsibility. Figure 7.1 illustrates the dependency relationships between modules and external libraries.

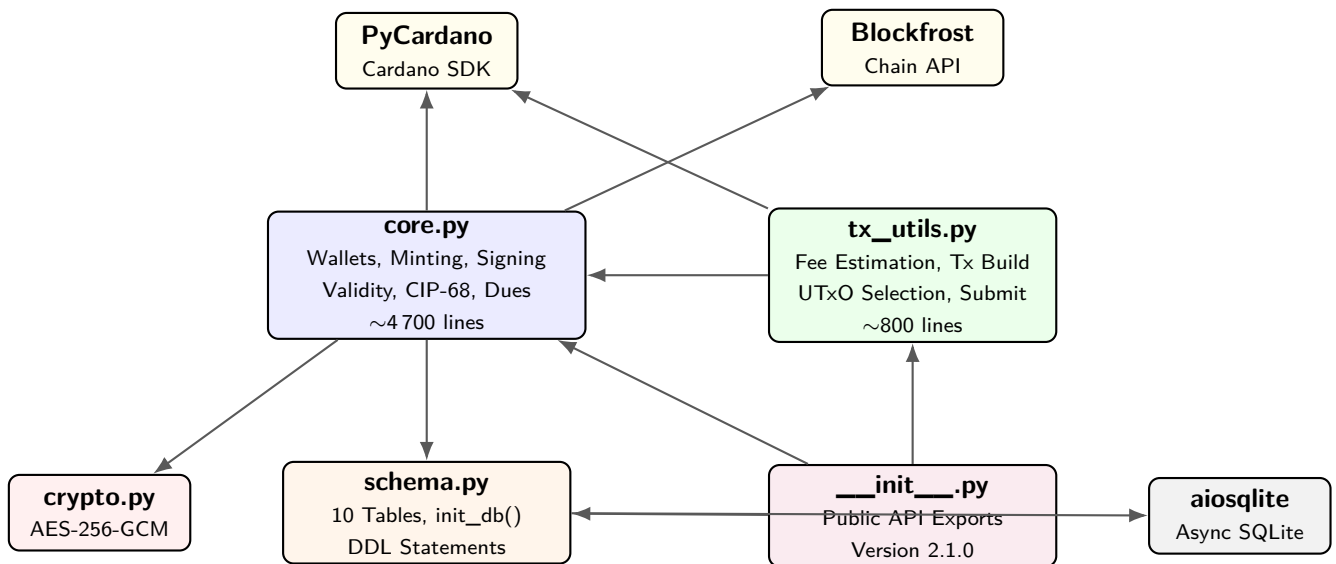


Figure 7.1: Module dependency diagram for `cardano-license`. Arrows point from the dependent module to the dependency. Yellow boxes are external libraries; colored boxes are internal modules.

7.2.1 `core.py` — The Main Module

The `core.py` module is the heart of the package, weighing in at approximately 4 700 lines of Python. It implements every domain operation in the credential lifecycle: HD wallet generation

following the CIP-1852 derivation standard, authority-restricted minting-policy creation, CIP-25-compliant license NFT minting, CIP-68 reference-token management for authority-controlled revocation, signature-token minting and transfer, validity-token minting and renewal, document signing with on-chain metadata, multi-signer work-product coordination, Plutus V2 signature-collection validators with atomic finalization, and dues-enforcement contracts with grace-period logic.

The module is organized into logical sections, each corresponding to a phase of the credential lifecycle. Wallet operations come first, followed by chain-context helpers, minting-policy management, license NFT operations, signature-token operations, validity-token operations, CIP-68 revocation, document signing and verification, work-product management, Plutus V2 validators, and dues enforcement. All database-touching functions are `async` and use `aiosqlite` for non-blocking I/O.

7.2.2 tx_utils.py — Transaction Utilities

The `tx_utils.py` module contains approximately 800 lines of transaction-building utilities that abstract away the lower-level details of Cardano transaction construction. Its primary exports include fee estimation (`estimate_fee`, `estimate_fee_from_context`), UTxO selection (`select_utxos`), and high-level transaction builders (`build_mint_tx`, `build_transfer_tx`, `build_multisig_tx`). It also provides `submit_tx` for transaction submission and `wait_for_confirmation` for polling until a transaction appears on-chain.

The module defines two important data classes. `TxResult` captures the outcome of any transaction operation—hash, fee, input/output counts, minted assets, confirmation status, and any error. `UTxOSelection` encapsulates the result of UTxO selection, including the selected inputs, total value, and change amount.

7.2.3 schema.py — Database Schema

The `schema.py` module defines the SQLite schema for the package’s local state. It contains ten `CREATE TABLE IF NOT EXISTS` statements and exposes a single public function, `init_db()`, which ensures all tables exist. The ten tables are described in detail in Section 7.4.

7.2.4 __init__.py — Package Exports

The `__init__.py` module re-exports the public API from `core.py`, `tx_utils.py`, and `schema.py`. It also defines the package version (`__version__ = "2.1.0"`) and the `__all__` list, which enumerates every public symbol. This module serves as the single entry point for consumers: `from cardano_license import generate_wallet, mint_license_nft`.

7.3 Key API Functions

This section presents the six most important functions in the `cardano-license` API, with usage examples. Every function is `async` and must be called within an `asyncio` event loop.

7.3.1 generate_wallet

The `generate_wallet` function creates a new HD wallet following the CIP-1852 derivation standard. It generates a 24-word BIP-39 mnemonic, derives payment and stake key pairs using

hardened derivation paths (`m/1852'/1815'/0'/0/0` for payment, `m/1852'/1815'/0'/2/0` for stake), constructs a base address that combines both keys, encrypts the private keys with AES-256-GCM, saves them to disk, and records the wallet metadata in the `blockchain_wallets` table.

The function accepts a `wallet_type` parameter that must be one of `authority`, `licensee`, `signer`, or `observer`. This type determines the wallet's role in the credential system and is enforced by a CHECK constraint in the database schema.

```
import asyncio
from cardano_license import generate_wallet

async def main():
    wallet = await generate_wallet(
        wallet_type="authority",
        label="state_board_of_engineers",
    )
    print(f"Address: {wallet['address']}")
    print(f"Mnemonic: {wallet['mnemonic']}")
    # Mnemonic: 24-word BIP-39 phrase (store securely!)
    # Keys saved to ~/.cardano-license/wallets/

asyncio.run(main())
```

Listing 7.2: Generating an authority wallet.

7.3.2 mint_license_nft

The `mint_license_nft` function mints a CIP-25-compliant license NFT and sends it to a licensee's address. Under the hood, it loads the authority wallet's keys from encrypted storage, creates a `ScriptPubkey` minting policy restricted to the authority's verification key hash, builds CIP-25 metadata containing the license fields, constructs and signs a minting transaction, submits it to the network via Blockfrost, and records the mint in the `blockchain_licenses` table.

The function validates all required metadata fields before performing any on-chain operation, failing fast with a `ValueError` if any field is missing. The minting policy ensures that only the authority's signing key can authorize the creation (or destruction) of tokens under that policy.

```
import asyncio
from cardano_license import mint_license_nft

async def main():
    result = await mint_license_nft(
        authority_wallet_label="state_board_of_engineers",
        licensee_address="addr_test1qz...",
        license_metadata={
            "name": "PE License - Jane Smith",
            "license_type": "Professional Engineer",
            "license_number": "PE-2026-00142",
            "jurisdiction": "New York",
            "issued_date": "2026-01-15",
            "expiry_date": "2028-01-15",
        },
    )
```

```

        "holder_name": "Jane Smith",
    },
)
print(f"Policy ID: {result['policy_id']}")
print(f"Token Name: {result['token_name']}")
print(f"Tx Hash: {result['tx_hash']}")

asyncio.run(main())

```

Listing 7.3: Minting a license NFT for a professional engineer.

7.3.3 update_license_status

The `update_license_status` function implements CIP-68 authority-controlled revocation. In the CIP-68 model, the authority holds a Reference Token (label 100) whose inline datum contains the license status. To revoke a license, the authority updates this datum to `status='revoked'`, requiring only the authority's signature and *zero interaction* with the licensee's wallet.

This is a critical design property: the licensee's User Token (label 222) remains in their wallet, but any verifier who checks the Reference Token's datum will see the revoked status. The authority retains unilateral control over credential status without needing to burn or recall tokens from the licensee.

```

import asyncio
from cardano_license import update_license_status

async def main():
    result = await update_license_status(
        license_id=42,
        new_status="revoked",
        authority_wallet_label="state_board_of_engineers",
    )
    print(f"Old status: {result['old_status']}")
    print(f"New status: {result['new_status']}")
    print(f"Tx Hash: {result['tx_hash']}")

asyncio.run(main())

```

Listing 7.4: Revoking a license via CIP-68 datum update.

7.3.4 sign_document

The `sign_document` function records a professional's signature on a document by transferring signature and validity tokens to a contract wallet address. The function builds a transaction that transfers one signature token and one validity token to the specified contract address, attaches the document's SHA-256 hash as transaction metadata (label 367), and records the timestamp in the `blockchain_signatures` table.

Each signer deposits tokens to an independent UTxO at the contract address, avoiding the eUTxO concurrency bottleneck that would arise if all signers tried to spend the same UTxO simultaneously. This design enables true parallel signing.

```

import asyncio
from cardano_license import sign_document

async def main():
    result = await sign_document(
        signer_wallet_label="jane_smith_pe",
        document_hash="a1b2c3d4e5f6...64-char-sha256-hex",
        contract_address="addr_test1qr...",
        license_ref=42,
    )
    print(f"Signature ID: {result['signature_id']}")
    print(f"Tx Hash:      {result['tx_hash']}")
    print(f"Timestamp:     {result['timestamp']}")

asyncio.run(main())

```

Listing 7.5: Signing a document with credential tokens.

7.3.5 verify_signature

The `verify_signature` function checks whether a document has been signed by querying the local signature records and verifying each signer's validity-token status. It returns a comprehensive result including the verification status, a list of all signatures, the signature count, and work-product information if the document is part of a multi-signer work product.

The verification process cross-references the `blockchain_signatures` table against the `blockchain_validity_token` table to ensure that each signer held a valid (unexpired, unrevoked) validity token at the time of signing.

```

import asyncio
from cardano_license import verify_signature

async def main():
    result = await verify_signature(
        contract_address="addr_test1qr...",
        document_hash="a1b2c3d4e5f6...64-char-sha256-hex",
    )
    print(f"Verified:      {result['is_verified']}")
    print(f"Signature count: {result['signature_count']}")
    for sig in result["signatures"]:
        print(f"  Signer: {sig['signer_address']}")
        print(f"  Time:   {sig['timestamp']}")

asyncio.run(main())

```

Listing 7.6: Verifying a document's signatures.

7.3.6 pay_dues

The `pay_dues` function records a dues payment against a dues-enforcement contract and renews the associated validity token. It validates the payment amount against the contract's

`annual_dues_lovelace` requirement, records the payment in the `dues_payments` table, and updates the validity expiry.

The dues-enforcement system links credential validity to ongoing financial obligations. If a licensee fails to pay annual dues within the grace period, the authority can invoke the `RevokeValidityRedeemer` to revoke the validity token on-chain, effectively disabling the credential.

```
import asyncio
from cardano_license import pay_dues

async def main():
    result = await pay_dues(
        contract_id=7,
        payer_address="addr_test1qz...",
        payment_lovelace=50_000_000,    # 50 ADA
        new_expiry="2027-03-01T00:00:00Z",
    )
    print(f"Payment ID: {result['payment_id']}")
    print(f"New Expiry: {result['new_expiry']}")

asyncio.run(main())
```

Listing 7.7: Paying annual dues to maintain credential validity.

7.4 Database Schema

The `cardano-license` package maintains local state in a SQLite database with ten tables. This database is *not* a replacement for the blockchain—it serves as a local index and cache that enables fast queries without hitting the Blockfrost API for every operation. The on-chain state remains the authoritative source of truth for all token ownership, transaction finality, and datum values.

Table 7.2 summarizes all ten tables. The entity-relationship diagram in Figure 7.2 shows the key foreign-key relationships.

The schema enforces data integrity through CHECK constraints on enumerated columns. For example, the `blockchain_wallets.wallet_type` column is constrained to ('authority', 'licensee', 'signer', 'observer'), and the `blockchain_licenses.status` column is constrained to ('pending', 'active', 'revoked', 'expired', 'burned'). These constraints prevent invalid state transitions at the database level, providing a safety net independent of application-level validation.

7.5 Configuration

The `cardano-license` package is configured entirely through environment variables, following the twelve-factor app methodology. This design enables the same codebase to target mainnet, preprod, or preview networks without code changes.

The configuration module (`config.py`) automatically creates the required directory structure on import: the base directory, the `wallets/` subdirectory for encrypted key files, and the

Table 7.2: Database schema: all ten tables in `cardano-license`.

Table	Key Columns	Purpose
<code>blockchain_wallets</code>	<code>id</code> , <code>wallet_type</code> , <code>address</code> , <code>public_key_hash</code> , <code>network</code> , <code>label</code>	Stores wallet metadata for all four roles (authority, licensee, signer, observer)
<code>blockchain_licenses</code>	<code>id</code> , <code>token_name</code> , <code>policy_id</code> , <code>licensee_address</code> , <code>authority_address</code> , <code>status</code>	Tracks minted license NFTs with metadata, status, and validity dates
<code>blockchain_reference_tokens</code>	<code>token_id</code> , <code>license_id</code> , <code>policy_id</code> , <code>user_token_name</code> , <code>ref_token_name</code> , <code>datum_json</code> , <code>status</code>	CIP-68 reference tokens with mutable datum for authority-controlled revocation
<code>blockchain_signature_tokens</code>	<code>token_id</code> , <code>policy_id</code> , <code>token_name</code> , <code>licensee_address</code> , <code>quantity</code> , <code>status</code>	Signature token inventory per licensee
<code>blockchain_validity_tokens</code>	<code>token_id</code> , <code>policy_id</code> , <code>token_name</code> , <code>licensee_address</code> , <code>valid_until</code> , <code>status</code>	Time-bounded validity tokens linked to licenses
<code>blockchain_signatures</code>	<code>id</code> , <code>document_hash</code> , <code>signer_address</code> , <code>signature_tx_hash</code> , <code>signature_datum</code>	Individual signature records with document hash and timestamp
<code>blockchain_work_products</code>	<code>id</code> , <code>title</code> , <code>wp_address</code> , <code>document_hash</code> , <code>required_signers_json</code> , <code>status</code>	Multi-signer work products with signature collection tracking
<code>blockchain_minting_policies</code>	<code>policy_id</code> , <code>authority_address</code> , <code>policy_cbor_hex</code> , <code>script_type</code>	Minting policy registry (native script and Plutus V2)
<code>dues_contracts</code>	<code>id</code> , <code>authority_address</code> , <code>license_ref</code> , <code>annual_dues_lovelace</code> , <code>grace_period_slots</code> , <code>status</code>	Dues-enforcement contracts linking credential validity to financial obligations
<code>dues_payments</code>	<code>id</code> , <code>contract_id</code> , <code>payer_address</code> , <code>amount_lovelace</code> , <code>new_expiry</code> , <code>status</code>	Payment records against dues contracts with confirmation tracking

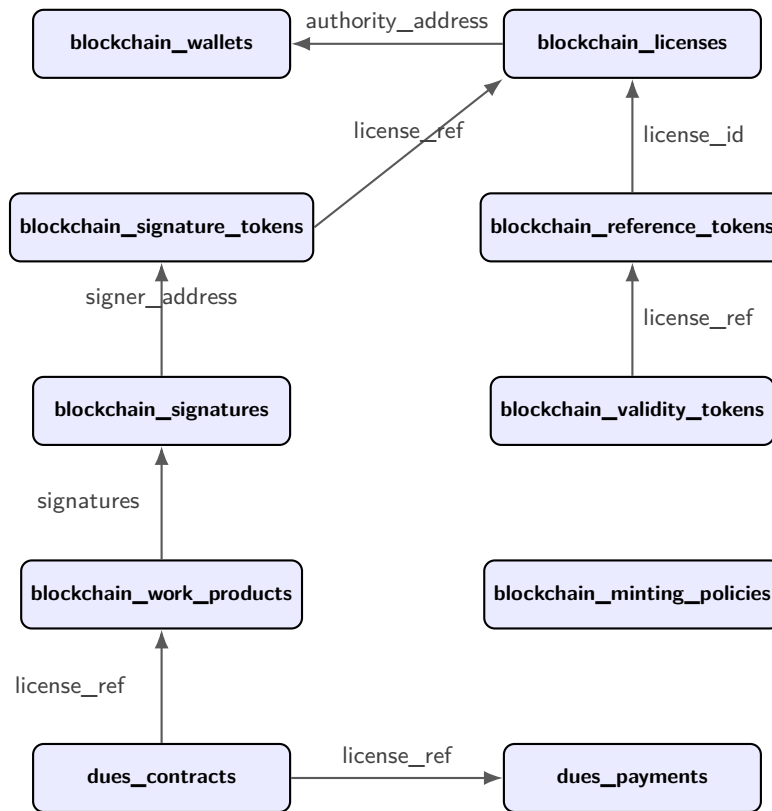


Figure 7.2: Entity-relationship diagram for the `cardano-license` database. Arrows indicate referential relationships between tables.

Table 7.3: Environment variables for `cardano-license` configuration.

Variable	Default	Purpose
CARDANO_NETWORK	testnet	Target network: <code>mainnet</code> , <code>testnet</code> (preprod), or <code>preview</code>
BLOCKFROST_PROJECT_ID	(empty)	Blockfrost API key for chain queries and transaction submission
CARDANO_LICENSE_PASSWORD	(empty)	Password for AES-256-GCM wallet key encryption; if empty, prompts interactively
CARDANO_LICENSE_DIR	<code>~/.cardano-license</code>	Base directory for wallets, policies, and database
CARDANO_LICENSE_DB	<code><DIR>/license.db</code>	SQLite database file path

`policies/` subdirectory for serialized minting-policy scripts. This means that a fresh installation requires no manual setup beyond setting the environment variables.

For production deployments, the `CARDANO_NETWORK` variable should be set to `mainnet` and the `BLOCKFROST_PROJECT_ID` should contain a mainnet API key obtained from the Blockfrost dashboard. For development and testing, the default `testnet` setting targets the Cardano preprod network, where test ADA is freely available from faucets.

7.6 Testing

The `cardano-license` package includes a comprehensive test suite with 660 passing tests. The tests are organized into four test modules covering the full spectrum of package functionality.

`test_license.py` covers wallet management (generation, key encryption/decryption, label lookup), license NFT operations (minting, metadata validation, status queries), CIP-68 reference-token management (store, update, revocation), signature-token operations (mint, balance, transfer), validity-token operations (mint, check, renew, revoke), document signing and verification, work-product lifecycle (create, sign, finalize), and authority registration.

`test_contracts.py` covers Plutus V2 smart-contract operations: signature-collection validator construction, datum and redeemer serialization, dues-enforcement contract deployment, payment validation, and grace-period logic.

`test_tx_utils.py` covers transaction-building utilities: fee estimation, UTxO selection, mint/-transfer/multisig transaction construction, submission, and confirmation polling.

`test_testnet.py` contains integration tests designed to run against the Cardano preprod testnet with a live Blockfrost connection. These tests are marked with `@pytest.mark.testnet` and are skipped by default in CI environments where testnet access is unavailable.

The test suite uses `pytest` with the `pytest-asyncio` plugin for async test support. All database-touching tests use isolated temporary databases to prevent cross-test contamination. Running the full suite requires only:

```
pytest tests/ -v
# 660 passed in ~45s (excluding testnet tests)
```

Listing 7.8: Running the test suite.

Part B — The v4.0 Roadmap

7.7 Aiken Smart Contracts

The current implementation uses Plutus V2 validators written as `PyCardano PlutusV2Script` objects with serialized CBOR bytecode. While functional, this approach inherits the well-known limitations of the PlutusTx/Haskell compilation pipeline: large compiled UPLC (Untyped

Plutus Lambda Calculus) output, high CPU and memory execution-unit (exUnit) consumption, and a steep learning curve for developers unfamiliar with Haskell.

Aiken is a modern smart-contract language designed specifically for Cardano. It uses a Rust-like syntax that is far more accessible to the broader developer community while maintaining the same security guarantees as Haskell-compiled Plutus scripts. The Aiken compiler produces significantly smaller UPLC bytecode—often 5–10× smaller than equivalent PlutusTx output—which directly translates to lower exUnit consumption and lower transaction fees.

The v4.0 roadmap calls for rewriting all three of the credential system’s validators in Aiken: the signature-collection validator (which manages multi-signer deposits, finalization sweeps, and reclaim operations), the dues-enforcement validator (which links credential validity to payment obligations), and the planned validity-consolidation validator (described in Section 7.9).

The benefits of the Aiken migration are fourfold. First, the compiled script sizes will shrink dramatically, reducing the on-chain footprint. Second, the smaller scripts consume fewer exUnits, lowering per-transaction costs. Third, the Rust-like syntax lowers the barrier to community contributions and third-party auditing. Fourth, the Aiken toolchain includes built-in testing and property-based fuzzing, improving validator correctness assurance.

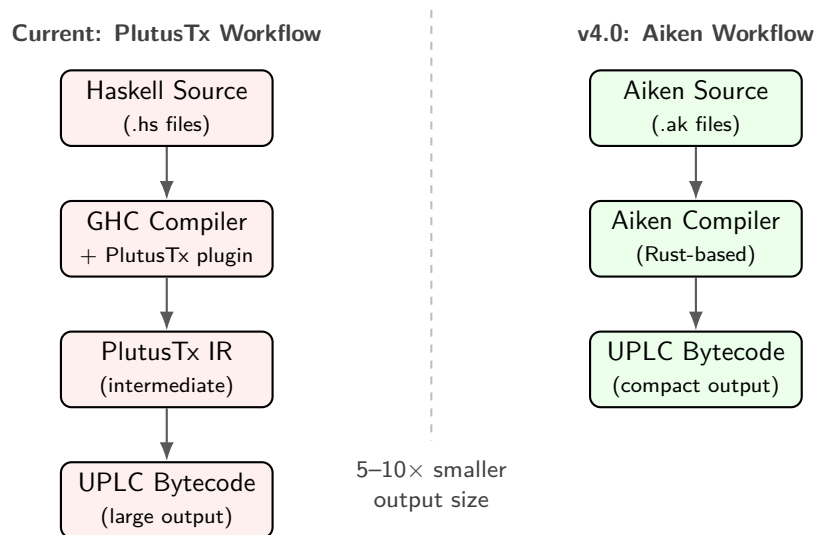


Figure 7.3: Compilation pipeline comparison: PlutusTx (left) requires three stages through the GHC/Haskell toolchain, producing large UPLC. Aiken (right) compiles directly to compact UPLC in a single step.

7.8 CIP-33 Reference Scripts

In the current architecture, every transaction that invokes a Plutus validator must attach the full compiled script bytecode. For the signature-collection validator, this adds several kilobytes to each signing transaction. For the dues-enforcement validator, it adds similar overhead to every payment transaction. This per-transaction overhead increases both transaction size and fees.

CIP-33 (Reference Scripts) solves this problem by allowing validators to be deployed to the blockchain *once* as reference scripts. The deploying transaction creates a UTxO that carries the

compiled script in its output. Every subsequent transaction that needs to invoke that validator simply references the UTxO containing the script, rather than attaching the full bytecode.

The impact on transaction economics is substantial. A typical signature-deposit transaction currently weighs approximately 8–12 KB due to the attached validator script. With CIP-33 reference scripts, the same transaction would weigh only 400–800 bytes—the script reference adds just a 32-byte transaction hash and a UTxO index. This reduction translates directly to lower fees: roughly 80–90% savings on script-bearing transactions.

The v4.0 implementation plan is straightforward. First, deploy each validator (signature collection, dues enforcement, validity consolidation) as a reference script in a dedicated UTxO. Second, update the transaction builders in `tx_utils.py` to reference the script UTxO instead of attaching the full bytecode. Third, store the reference-script UTxO identifiers in the `blockchain_minting_policies` table for retrieval during transaction construction.

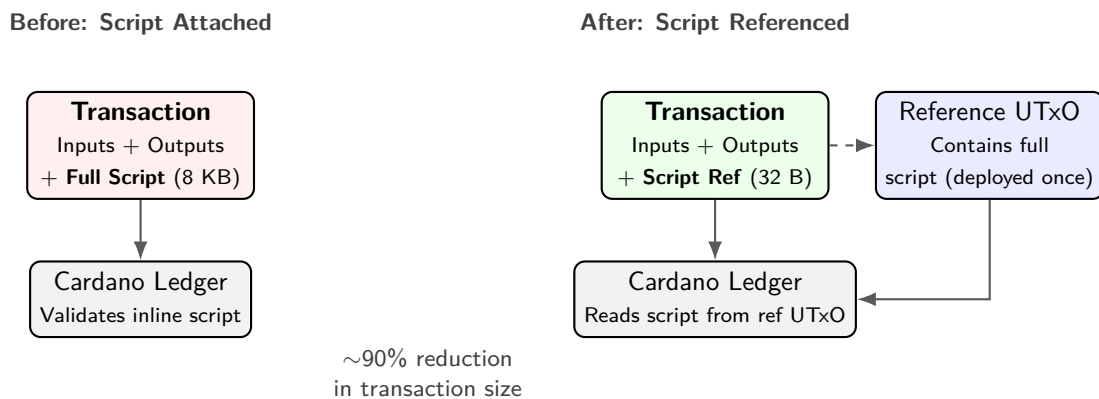


Figure 7.4: CIP-33 Reference Scripts: before (left), each transaction carries the full validator bytecode. After (right), the script is deployed once and subsequent transactions reference it by UTxO, dramatically reducing size and fees.

7.9 Validity Token Consolidation

The current three-token architecture presents a dual-state consistency risk that was identified during the v3.0 security audit (see Section 7.12). The problem: the CIP-68 Reference Token’s datum may say “revoked,” but the licensee may still hold an unexpired Validity Token (Token 3) in their wallet. A naïve verifier that checks only the Validity Token would incorrectly conclude that the credential is still valid. Although the system documentation specifies that verifiers must check *both* the Reference Token datum and the Validity Token, this dual-check requirement introduces a class of verification errors that a simpler architecture would eliminate.

The v4.0 solution is to deprecate Token 3 (the standalone Validity Token) entirely. Instead, validity information will be encoded directly into the CIP-68 Reference Token’s datum as a `valid_until` field containing a POSIX timestamp. Any verifier—or any on-chain validator—can read this datum via a CIP-31 Reference Input (a read-only input that does not consume the UTxO) and determine both the credential’s status and its validity window from a *single source of truth*.

This consolidation eliminates the dual-state risk entirely. The authority controls the Reference

Token’s datum, and the datum contains both the status (**active/revoked/suspended**) and the validity window (**valid_until**). There is no separate token whose state could contradict the Reference Token.

The Plutus validator for the new consolidated design is straightforward: it reads the Reference Token’s datum via CIP-31 Reference Input, extracts the **valid_until** field, compares it against the current slot (converted to POSIX time), and checks the **status** field—all in a single on-chain read with no UTxO consumption.

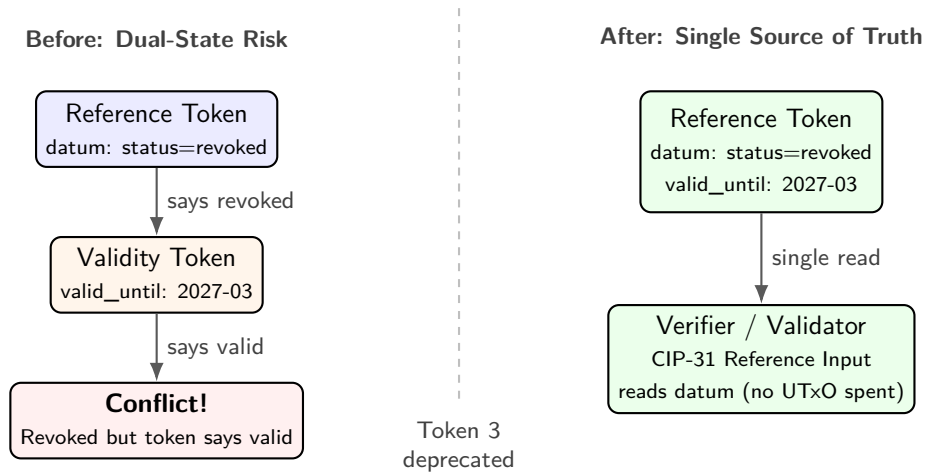


Figure 7.5: Validity token consolidation: before (left), the Reference Token and Validity Token can contradict each other. After (right), all status and validity information resides in the Reference Token’s datum, readable via CIP-31 Reference Input.

7.10 HSM Authority Keys

In the current implementation, authority wallet private keys are encrypted with AES-256-GCM using a password-derived key (PBKDF2-HMAC-SHA256 with 100 000 iterations) and stored as files on the server’s filesystem. While this provides strong cryptographic protection against offline attacks, the keys are still *software-managed*: they are decrypted into process memory during signing operations, and a sufficiently privileged attacker with server access could potentially extract them.

Hardware Security Modules (HSMs) provide a fundamentally stronger security model. An HSM is a dedicated cryptographic processor that generates, stores, and uses keys entirely within a tamper-resistant hardware enclave. The critical property is that private keys *never leave the HSM*—not even during signing. When the application needs to sign a transaction, it sends the transaction hash to the HSM, and the HSM returns the signature. The private key material is never exposed to the host operating system, the application process, or any software layer.

For the credential verification system, HSM integration means that the authority’s signing key—the key that controls who can mint license NFTs, revoke credentials, and deploy smart contracts—would be protected at the hardware level. Even if an attacker gained root access to the server, they could not extract the authority key. They could potentially *use* the HSM to sign transactions (if they gained access to the HSM’s authentication credentials), but they could not *steal* the key for use elsewhere.

The v4.0 HSM integration plan targets PKCS#11-compatible HSMs, which include cloud-hosted options (AWS CloudHSM, Azure Dedicated HSM, Google Cloud HSM) and on-premises devices (Thales Luna, Utimaco CryptoServer). The `crypto.py` module will be extended with an HSM backend that implements the same `encrypt_key/decrypt_key` interface, allowing the rest of the codebase to use HSM-backed keys without modification.

7.11 KERI / Multi-Sig Upgrade

The current system architecture concentrates all authority operations behind a single signing key. Whoever controls that key can mint credentials, revoke licenses, deploy smart contracts, and collect dues payments. This single-key design was identified as finding AS-6 in the v3.0 security audit: a single point of failure that, if compromised, could undermine the entire credential system for that authority.

The v4.0 roadmap addresses this with two alternative approaches to distributed governance, either of which eliminates the single-key risk.

Option A: KERI (Key Event Receipt Infrastructure). KERI, standardized through the Veridian project, provides a decentralized key-management framework where key authority is established through a cryptographically linked sequence of key events (inception, rotation, delegation) rather than through a single static key. In a KERI-based authority model, the credential-issuing authority would be identified by its KERI Autonomic Identifier (AID), and key rotation events would allow the authority to update its signing keys without invalidating previously issued credentials. This provides resilience against key compromise: if a key is suspected of being compromised, the authority can rotate to a new key pair, and all future operations use the new key while past operations remain verifiable through the key-event log.

Option B: Cardano ScriptNofK Multi-Signature. Cardano's native scripting system supports N -of- K multi-signature policies through the `ScriptNofK` constructor. A 3-of-5 configuration, for example, would require any three out of five designated board members to co-sign every authority operation. This distributes trust across multiple individuals and eliminates the single-key bottleneck.

In practical terms, a 3-of-5 multi-sig policy means that minting a new license NFT requires three board members to independently sign the transaction. Revoking a credential requires three signatures. Deploying a new smart contract requires three signatures. No single individual—and no single compromised key—can perform any authority action alone.

The choice between KERI and multi-sig depends on the deploying organization's requirements. KERI provides more sophisticated key management (rotation, delegation, recovery) but requires infrastructure for maintaining the key-event log. Multi-sig is simpler to implement using Cardano's native scripting but does not support key rotation without creating a new policy (and thus new policy ID). Both approaches achieve the primary goal: eliminating the single-key centralization risk identified in AS-6.

7.12 Security Audit Results

The credential verification system underwent three rounds of architectural security audit in February 2026, progressing from initial finding identification through remediation verification

to final clearance.

7.12.1 v3.0 Audit: Initial Findings

The first audit round examined the v3.0 architecture and identified five findings across three severity levels:

- **2 Critical findings:**

1. *Single-key authority risk (AS-6)*: All authority operations depend on a single signing key. Compromise of this key would allow an attacker to mint fraudulent credentials, revoke legitimate ones, or deploy malicious smart contracts.
2. *Dual-state validity inconsistency*: The CIP-68 Reference Token datum and the standalone Validity Token can hold contradictory state, creating a class of verification errors.

- **1 High finding:**

1. *Software-managed authority keys*: Private keys decrypted into process memory during signing operations are vulnerable to memory-extraction attacks by privileged processes.

- **2 Medium findings:**

1. *Per-transaction script attachment overhead*: Attaching full validator bytecode to every transaction increases fees and exposes the compiled script to analysis.
2. *PlutusTx compilation inefficiency*: The Haskell-based compilation pipeline produces unnecessarily large UPLC output.

7.12.2 v3.1 Audit: Remediation Verification

The second audit round verified that all five findings from the v3.0 audit had been addressed with concrete remediation plans:

- AS-6 (single-key risk): addressed by the KERI/multi-sig upgrade (Section 7.11)
- Dual-state inconsistency: addressed by validity-token consolidation (Section 7.9)
- Software-managed keys: addressed by HSM integration (Section 7.10)
- Script attachment overhead: addressed by CIP-33 reference scripts (Section 7.8)
- PlutusTx inefficiency: addressed by Aiken migration (Section 7.7)

7.12.3 Final Audit: Testnet Clearance

The third and final audit round confirmed that the remediation plans were architecturally sound and that the v3.1 implementation (with the v4.0 roadmap items scheduled for implementation) was cleared for testnet deployment. All three audit reports are published in the project repository.

The audit progression demonstrates a disciplined approach to security engineering: identify weaknesses, design mitigations, verify the mitigations address the root causes, and document everything. The v4.0 roadmap items described in this chapter are the direct result of this audit process.

7.13 Glossary of Terms

The following glossary defines every significant technical term used throughout this study guide, organized alphabetically.

ADA The native cryptocurrency of the Cardano blockchain. One ADA equals 1 000 000 lovelace. ADA is used to pay transaction fees, satisfy minimum UTxO requirements, and transfer value between addresses.

AES-256-GCM Advanced Encryption Standard with 256-bit keys in Galois/Counter Mode. An authenticated encryption algorithm that provides both confidentiality and integrity. Used by `cardano-license` to encrypt wallet private keys at rest, with key derivation via PBKDF2-HMAC-SHA256 (100 000 iterations).

Aiken A modern smart-contract language for Cardano with Rust-like syntax. Compiles directly to UPLC bytecode, producing significantly smaller output than the PlutusTx/Haskell pipeline. Planned for v4.0 to replace Haskell-compiled validators.

Asset Name The human-readable identifier for a native token within a minting policy. Combined with the Policy ID to form a globally unique token identifier. In the credential system, asset names encode the license holder's identifier (e.g., PE-2026-00142).

Base Address A Cardano address that combines both a payment credential (derived from the payment key) and a staking credential (derived from the stake key). This is the standard address format used by the credential system, enabling both spending and staking from the same address.

BIP-39 Bitcoin Improvement Proposal 39: a standard for generating deterministic wallet seeds from mnemonic word sequences. The credential system uses 24-word BIP-39 mnemonics as the root entropy for all wallet derivation.

BIP-44 Bitcoin Improvement Proposal 44: a standard for hierarchical deterministic (HD) wallet derivation paths. Superseded by CIP-1852 for Cardano-specific derivation, but the underlying multi-level derivation concept remains the same.

Blake2b A cryptographic hash function used extensively in Cardano for address derivation, script hashing, and transaction identification. Cardano uses Blake2b-224 (28 bytes) for key hashes and Blake2b-256 (32 bytes) for transaction hashes.

Blockfrost A hosted API service that provides access to the Cardano blockchain without running a full node. Supports UTxO queries, transaction submission, metadata retrieval, and block monitoring. Used by `cardano-license` as the default chain backend.

- CIP-25** Cardano Improvement Proposal 25: the metadata standard for NFTs on Cardano. Defines how token metadata (name, image, description, attributes) is attached to minting transactions under metadata label 721. Used for license NFT metadata in the credential system.
- CIP-30** Cardano Improvement Proposal 30: the wallet connector standard that defines a JavaScript API for web applications to interact with Cardano wallets (e.g., Nami, Eternl). Enables browser-based credential verification interfaces.
- CIP-31** Cardano Improvement Proposal 31: Reference Inputs. Allows transactions to read the datum of a UTxO without consuming it. Critical for the validity-token consolidation design, where verifiers read the Reference Token’s datum without spending the authority’s UTxO.
- CIP-32** Cardano Improvement Proposal 32: Inline Datums. Allows datums to be stored directly in transaction outputs rather than as hashes. Eliminates the need for datum witnesses in spending transactions and enables CIP-31 reference-input reads.
- CIP-33** Cardano Improvement Proposal 33: Reference Scripts. Allows Plutus scripts to be deployed once on-chain and referenced by subsequent transactions, eliminating the need to attach full script bytecode to every transaction. See Section 7.8.
- CIP-68** Cardano Improvement Proposal 68: an NFT standard that splits each token into a Reference Token (label 100, held by the issuer, mutable datum) and a User Token (label 222, held by the recipient, immutable). The credential system uses CIP-68 for authority-controlled revocation: the authority updates the Reference Token’s datum without touching the licensee’s wallet.
- CIP-1852** Cardano Improvement Proposal 1852: the HD wallet derivation standard for Cardano. Defines derivation paths using purpose 1852, coin type 1815 (Cardano), and separates payment keys (role 0) from stake keys (role 2). Path format: `m/1852'/1815'/0'/{role}/{index}`.

Crypto-shredding

A data-destruction technique where encrypted data is rendered permanently inaccessible by destroying the encryption key rather than the ciphertext. Used in the credential system’s GDPR compliance design: personal data stored off-chain under AES-256-GCM can be “erased” by deleting the encryption key, while the on-chain attestation hash remains.

- Datum** Data attached to a UTxO on the Cardano blockchain. In the eUTxO model, datums serve as the state of smart contracts. The credential system uses datums to store license status, validity windows, signer deposits, and dues-contract parameters. See also: Inline Datum, CIP-32.
- DID** Decentralized Identifier: a W3C standard for self-sovereign digital identities. DIDs are URI-formatted identifiers that can be resolved to DID Documents containing public keys, service endpoints, and authentication methods. Related to KERI’s approach to decentralized identity.
- eIDAS** Electronic Identification, Authentication and Trust Services: the European Union regulation establishing a legal framework for electronic signatures, seals,

timestamps, and identification. eIDAS 2.0 introduces the European Digital Identity Wallet. The credential system’s on-chain signatures align with eIDAS requirements for advanced electronic signatures.

ESIGN Act	The Electronic Signatures in Global and National Commerce Act (2000): a United States federal law that grants electronic signatures the same legal standing as handwritten signatures. Together with UETA, establishes the legal foundation for blockchain-based professional credential signatures.
eUTxO	Extended Unspent Transaction Output: Cardano’s transaction model that extends Bitcoin’s UTxO model with datums (state), redeemers (actions), and script contexts (validators). The eUTxO model provides deterministic transaction validation and enables the credential system’s per-signer deposit design for concurrent signing.
Epoch	A time period in the Cardano blockchain, lasting approximately 5 days (432 000 slots on mainnet). Epochs are relevant to the credential system for stake-reward distribution and protocol-parameter updates that affect transaction fees and minUTxO values.
exUnits	Execution units: a two-dimensional measure of Plutus script resource consumption on Cardano, comprising CPU steps and memory units. Each transaction has maximum exUnit budgets. Smaller exUnit consumption (e.g., from Aiken-compiled scripts) translates to lower fees and higher throughput.
GDPR	General Data Protection Regulation: the European Union’s data-privacy law that includes the “right to erasure” (Article 17). The credential system’s off-chain personal-data storage with crypto-shredding is designed to satisfy GDPR requirements while maintaining on-chain attestation immutability.
Halo2	A zero-knowledge proof system based on polynomial commitments, developed by the Electric Coin Company. Relevant to future privacy-preserving credential verification where a holder can prove possession of a valid credential without revealing the credential’s details.
HD Wallet	Hierarchical Deterministic wallet: a wallet architecture where a single seed (mnemonic) can derive an unlimited tree of key pairs using deterministic derivation paths (BIP-32/BIP-44/CIP-1852). The credential system uses HD wallets for all participant roles.
HSM	Hardware Security Module: a dedicated cryptographic processor that generates, stores, and uses keys within a tamper-resistant hardware enclave. Private keys never leave the HSM. Planned for v4.0 to protect authority signing keys. See Section 7.10 .
Inline Datum	A datum stored directly in a transaction output (enabled by CIP-32), rather than as a hash that must be provided as a witness during spending. Inline datums enable CIP-31 reference-input reads, which are essential for the validity-token consolidation design.
KERI	Key Event Receipt Infrastructure: a decentralized key-management protocol standardized through the Veridian project. KERI establishes key authority

through a cryptographically linked sequence of key events (inception, rotation, delegation, interaction). Planned for v4.0 as one option for distributed authority governance. See Section [7.11](#).

Lovelace The smallest unit of ADA. One ADA equals 1 000 000 lovelace. Named after Ada Lovelace. All on-chain amounts in the credential system are denominated in lovelace.

minUTxO The protocol-enforced minimum amount of ADA that must accompany any UTxO. The value depends on the UTxO's size (datum size, number of native tokens). The credential system uses a conservative default of 2 ADA (2 000 000 lovelace) for token-bearing outputs.

Minting Policy

A script that governs the creation and destruction of native tokens on Cardano. The credential system uses authority-key-restricted **ScriptPubkey** policies (Native Script) that permit only the authority's signing key to mint or burn tokens under that policy.

Native Script

A simple scripting language on Cardano (predating Plutus) that supports key-signature requirements, time locks, and Boolean combinations (all-of, any-of, *N-of-K*). The credential system uses Native Scripts for minting policies. Multi-sig governance would use **ScriptNofK**.

NFT Non-Fungible Token: a unique token with quantity 1. On Cardano, NFTs are native multi-asset tokens with an enforced quantity of one. License NFTs in the credential system are NFTs carrying CIP-25 metadata that attests to a professional's credential.

Ouroboros The family of proof-of-stake consensus protocols used by Cardano. Ouroboros Praos (the current production variant) provides probabilistic finality and is formally verified for security properties. Relevant to the credential system because consensus determines when minted credentials become final.

PKH Public Key Hash: the Blake2b-224 hash of a verification (public) key. Used as the payment credential in Cardano addresses and as the identity parameter in minting policies. The credential system's authority-restricted policies require the authority's PKH to authorize minting.

Plutus The smart-contract platform on Cardano, based on the eUTxO model. Plutus scripts (validators) run on-chain to enforce spending conditions. The credential system uses Plutus V2 for the signature-collection validator and the dues-enforcement contract.

Policy ID The hash of a minting policy script. Serves as the unique, immutable identifier for all tokens minted under that policy. In the credential system, the Policy ID cryptographically binds every license NFT, signature token, and validity token to the authority that created the minting policy.

PyCardano A Python library for building Cardano transactions, managing wallets, and interacting with the blockchain. Provides HD wallet generation, Plutus V2

script support, and CIP-1852 derivation. The primary Cardano SDK used by `cardano-license`.

Redeemer Data provided by a transaction to a Plutus validator, specifying the intended action. In the credential system, redeemers include `CollectRedeemer` (deposit a signature), `FinalizeRedeemer` (sweep all deposits), `ReclaimRedeemer` (reclaim a deposit), `PayDuesRedeemer` (pay dues), and `RevokeValidityRedeemer` (revoke for non-payment).

Reference Input

A transaction input (enabled by CIP-31) that reads a UTxO's datum without consuming the UTxO. Essential for the validity-consolidation design, where verifiers read the Reference Token's datum without spending the authority's UTxO. See CIP-31.

Reference Script

A Plutus script stored on-chain in a UTxO (enabled by CIP-33) that can be referenced by subsequent transactions instead of being attached inline. Reduces transaction size and fees. See Section 7.8.

Reference Token

In the CIP-68 standard, the token with label 100 that is held by the issuing authority and carries a mutable inline datum. The credential system uses Reference Tokens for authority-controlled status management: the authority updates the datum to revoke or suspend a credential without interacting with the licensee's wallet.

Script Context

The information available to a Plutus validator during execution: the transaction being validated (inputs, outputs, minting, fees, validity range, signatories), the script's own purpose (spending, minting, certifying), and the datum and redeemer. Validators use the Script Context to enforce spending conditions.

Slot The fundamental time unit on the Cardano blockchain, lasting 1 second on mainnet. Slots are used in validity ranges (`InvalidBefore`, `InvalidHereAfter`) and in time-locked minting policies. The credential system converts between POSIX timestamps and slot numbers for validity-window enforcement.

Stake Key The key pair used for staking operations in a Cardano HD wallet. Derived from path `m/1852'/1815'/0'/2/0`. Combined with the payment key to form a base address. The credential system derives stake keys for all wallet types but uses them primarily for address construction.

UETA Uniform Electronic Transactions Act (1999): a U.S. model law (adopted by 47 states) that establishes the legal validity of electronic records and signatures. Together with the E-SIGN Act, provides the legal foundation for blockchain-based credential signatures.

UPLC Untyped Plutus Lambda Calculus: the low-level execution language for Plutus smart contracts on Cardano. Both PlutusTx (Haskell) and Aiken compile down to UPLC bytecode. Smaller UPLC output means lower exUnit consumption and lower fees.

- User Token** In the CIP-68 standard, the token with label 222 that is held by the recipient (licensee). The User Token is immutable and serves as the holder’s proof of credential issuance. Its counterpart, the Reference Token (label 100), is held by the authority and carries the mutable status datum.
- UTxO** Unspent Transaction Output: the fundamental unit of value on the Cardano blockchain. Each UTxO contains an address, a value (ADA and/or native tokens), and optionally a datum. Spending a UTxO consumes it entirely and creates new UTxOs. The credential system’s per-signer deposit design creates independent UTxOs to avoid concurrency conflicts.
- Validator** A Plutus script that governs the conditions under which a UTxO can be spent. The credential system uses two validators: the signature-collection validator (manages multi-signer deposits and finalization) and the dues-enforcement validator (links credential validity to payment obligations).
- Veridian** The open-source project (formerly under the Trust over IP Foundation) that develops the KERI protocol and ACDC (Authentic Chained Data Container) standards for decentralized identity and verifiable credentials. The v4.0 roadmap considers Veridian/KERI as one option for distributed authority governance.